EAST WEST UNIVERSITY

Course Code: CSE423

Course Title: Software Architecture

Section: 01

Semester: Spring26

Group No: 01

Report

Project Name: Hospital Management System

Submitted To:

Ahmed Adnan

Lecturer

Department of Computer Science and Engineering

Submitted By:

Name:	ID:
Sababa Hoque Saba	2022-3-60-060
Afia Ayub Samirah	2022-3-60-012
Kha. Mo. Syeed Asif	2022-3-60-030
Md. Omor Faruq Shikdere	2022-1-60-335

Date of Submission: 10-02-2026

SOLID Principles Applied to Hospital Management System - AddDoctor Module

Link of Commit ([Applied SOLID Principle])

1. Single Responsibility Principle (SRP)

Before: **AddDoctor** class handled **UI layout, event handling, and doctor data storage**.

```
import java.lang.*;
```

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;



public class AddDoctor extends JFrame implements
MouseListener,ActionListener{



    JLabel userLabel,passLabel,genderLabel,programLabel,frame,imgLabel;

    JTextField userTF;

    JPasswordField passTF;

    JButton add, back;

    ButtonGroup bg1;

    JRadioButton r1,r2,r3;

    JComboBox dept;

    Color color1,color2;

    Font font1, font2, font3, font4;

    ImageIcon img;

    JPanel panel;
```

```
public AddDoctor(){

    super("Doctor Information");
    this.setSize(900,600);
    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    this.setLocationRelativeTo(null);



    color1= new Color(143,207,225);
    color2= new Color(39,83,23);

    font1 = new Font("Biome",Font.BOLD, 30);
    font2 = new Font("Cascadia Code SemiBold",Font.PLAIN, 20);
    font3 = new Font("Cascadia Mono SemiBold",Font.PLAIN, 15);
    font4 = new Font("Biome",Font.BOLD, 18);



    panel = new JPanel();
    panel.setLayout(null);

    userLabel = new JLabel("HealthMate ");
    userLabel.setBounds(300,10,400,100);
```

```
        userLabel.setFont(font1);

        panel.add(userLabel);




        userLabel = new JLabel("Add Doctor Form ");

        userLabel.setBounds(130,80,400,50);

        userLabel.setFont(font4);

        panel.add(userLabel);


        userLabel = new JLabel("Doctor Name: ");

        userLabel.setBounds(75,120,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,160,150,20);

        panel.add(userTF);


        userLabel = new JLabel("Mobile Number: ");

        userLabel.setBounds(75,150,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,190,150,20);
```

```
        panel.add(userTF);


        genderLabel = new JLabel("Gender:");

        genderLabel.setBounds(75,220,115,20);

        panel.add(genderLabel);


        r1 = new JRadioButton("Male");

        r1.setBounds(150,220,80,20);

        r1.setBackground(color1);

        panel.add(r1);


        r2 = new JRadioButton("Female");

        r2.setBounds(230,220,80,20);

        r2.setBackground(color1);

        panel.add(r2);


        r3 = new JRadioButton("Other");

        r3.setBounds(320,220,80,20);

        r3.setBackground(color1);

        panel.add(r3);


        bg1 = new ButtonGroup();

        bg1.add(r1);
```

```
        bg1.add(r2);

        bg1.add(r3);




        programLabel = new JLabel("Department:");

        programLabel.setBounds(75,210,100,100);

        panel.add(programLabel);


        String items[] = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};

        dept = new JComboBox(items);

        dept.setBounds(170,250,150,20);

        panel.add(dept);


        userLabel = new JLabel("Joining Date: ");

        userLabel.setBounds(75,240,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,280,150,20);

        panel.add(userTF);


        userLabel = new JLabel("Salary: ");

        userLabel.setBounds(75,270,100,100);
```

```
        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,310,150,20);

        panel.add(userTF);


        userLabel = new JLabel("Room NO: ");

        userLabel.setBounds(75,300,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,340,150,20);

        panel.add(userTF);


        userLabel = new JLabel("BM&DC REG: ");

        userLabel.setBounds(75,330,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,370,150,20);

        panel.add(userTF);
```

```
add = new JButton("ADD");
add.setBounds(500,470,100,40);
add.setBackground(Color.GREEN);
add.setForeground(Color.WHITE);
add.setOpaque(true);
add.addMouseListener(this);
add.addActionListener(this);
panel.add(add);

back = new JButton("BACK");
back.setBounds(620,470,100,40);
back.setBackground(Color.RED);
back.setForeground(Color.WHITE);
back.setOpaque(true);
back.addMouseListener(this);
back.addActionListener(this);
panel.add(back);

img = new ImageIcon("images/emp-info.png");

imgLabel = new JLabel(img);
imgLabel.setBounds(485,100,300,300);
```

```
        panel.add(imgLabel);


        panel.setBackground(color1);
        this.add(panel);
    }


    public void mouseClicked(MouseEvent me){}
    public void mousePressed(MouseEvent me){}
    public void mouseReleased(MouseEvent me){}
    public void mouseEntered(MouseEvent me){
        if(me.getSource() == add){
            add.setBackground(Color.RED);
    add.setForeground(Color.WHITE);
        }else if(me.getSource() == back){
            back.setBackground(Color.GREEN);
            back.setForeground(Color.BLACK);
        }
    }
    public void mouseExited(MouseEvent me){
        if(me.getSource() == add){
            add.setBackground(Color.GREEN);
            add.setForeground(Color.BLACK);
        }else if(me.getSource() == back){
```

```
            back.setBackground(Color.RED);

            back.setForeground(Color.WHITE);

        }

    }



    public void actionPerformed(ActionEvent ae){

        String command = ae.getActionCommand();

        if(add.getText().equals(command));{

            DoctorList dl = new DoctorList();

            dl.setVisible(true);

            this.setVisible(false);



        }

        //public void actionPerformed(ActionEvent ae){

        //String command = ae.getActionCommand();

            if(ae.getSource() == back) {



                    JOptionPane.showMessageDialog(this, "Added doctor
successfully");

                     DrMenu dm = new DrMenu();

                    dm.setVisible(true);

                    this.setVisible(false);

                }
```

```
    }



}
```

After:

Doctor - handles **data only**

DoctorService - handles **doctor-related operations** (adding, storing)

AddDoctorUI - handles **UI only**

Doctor.java

```
public class Doctor {

    private String name;

    private String mobile;

    private String gender;

    private String department;

    private String joiningDate;

    private double salary;

    private String roomNo;

    private String bmdcReg;
```

```java
    public Doctor(String name, String mobile, String gender, String
department,
                  String joiningDate, double salary, String roomNo, String
bmdcReg) {
        this.name = name;
        this.mobile = mobile;
        this.gender = gender;
        this.department = department;
        this.joiningDate = joiningDate;
        this.salary = salary;
        this.roomNo = roomNo;
        this.bmdcReg = bmdcReg;
    }

    // Getters
    public String getName() { return name; }
    public String getMobile() { return mobile; }
    public String getGender() { return gender; }
    public String getDepartment() { return department; }
    public String getJoiningDate() { return joiningDate; }
    public double getSalary() { return salary; }
    public String getRoomNo() { return roomNo; }
    public String getBmdcReg() { return bmdcReg; }
}
```

DoctorService.java

```
import java.util.ArrayList;

import java.util.List;


public class DoctorService implements IDoctorService {

    private List<Doctor> doctors = new ArrayList<>();


    @Override

    public void addDoctor(Doctor doctor) {

        doctors.add(doctor);


    }


    public List<Doctor> getDoctors() {

        return doctors;

    }

}
```

AddDoctorUI.java

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;


public class AddDoctorUI extends JFrame {


    private IDoctorService doctorService;


    private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF,
bmdcTF;

    private JRadioButton maleRB, femaleRB, otherRB;

    private JComboBox<String> deptCB;

    private JButton addButton, backButton;

    private ButtonGroup genderGroup;


    public AddDoctorUI(IDoctorService doctorService) {

        super("Add Doctor Form");

        this.doctorService = doctorService;


        setSize(900, 600);

        setLocationRelativeTo(null);
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);


        initUI();
    }


    private void initUI() {
        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBackground(new Color(143,207,225));


        Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN,
18);


        // Labels & TextFields
        JLabel titleLabel = new JLabel("Add Doctor Form");
        titleLabel.setBounds(130, 20, 300, 40);
        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
        panel.add(titleLabel);


        JLabel nameLabel = new JLabel("Doctor Name:");
        nameLabel.setBounds(75, 80, 120, 25);
        nameLabel.setFont(labelFont);
        panel.add(nameLabel);
```

```
        nameTF = new JTextField();
        nameTF.setBounds(200, 80, 150, 25);
        panel.add(nameTF);

        JLabel mobileLabel = new JLabel("Mobile Number:");
        mobileLabel.setBounds(75, 120, 120, 25);
        mobileLabel.setFont(labelFont);
        panel.add(mobileLabel);

        mobileTF = new JTextField();
        mobileTF.setBounds(200, 120, 150, 25);
        panel.add(mobileTF);

        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75, 160, 100, 25);
        genderLabel.setFont(labelFont);
        panel.add(genderLabel);

        maleRB = new JRadioButton("Male");
        maleRB.setBounds(150, 160, 80, 25);
        maleRB.setBackground(panel.getBackground());
        femaleRB = new JRadioButton("Female");
        femaleRB.setBounds(230, 160, 80, 25);
```

```
        femaleRB.setBackground(panel.getBackground());

        otherRB = new JRadioButton("Other");

        otherRB.setBounds(310, 160, 80, 25);

        otherRB.setBackground(panel.getBackground());



        genderGroup = new ButtonGroup();

        genderGroup.add(maleRB);

        genderGroup.add(femaleRB);

        genderGroup.add(otherRB);



        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);



        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);



        String[] departments = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);

        deptCB.setBounds(200, 200, 150, 25);

        panel.add(deptCB);
```

```
JLabel joiningLabel = new JLabel("Joining Date:");
joiningLabel.setBounds(75, 240, 120, 25);
joiningLabel.setFont(labelFont);
panel.add(joiningLabel);

joiningTF = new JTextField();
joiningTF.setBounds(200, 240, 150, 25);
panel.add(joiningTF);

JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);
```

```
roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);

bmdcTF = new JTextField();
bmdcTF.setBounds(200, 360, 150, 25);
panel.add(bmdcTF);

// Buttons
addButton = new JButton("ADD");
addButton.setBounds(500, 470, 100, 40);
addButton.setBackground(Color.GREEN);
addButton.setForeground(Color.WHITE);
addButton.addActionListener(e -> onAdd());
panel.add(addButton);

backButton = new JButton("BACK");
```

```
        backButton.setBounds(620, 470, 100, 40);

        backButton.setBackground(Color.RED);

        backButton.setForeground(Color.WHITE);

        backButton.addActionListener(e -> onBack());

        panel.add(backButton);



        this.add(panel);

    }



    private void onAdd() {

        String name = nameTF.getText();

        String mobile = mobileTF.getText();

        String gender = maleRB.isSelected() ? "Male" :

                        femaleRB.isSelected() ? "Female" : "Other";

        String department = (String) deptCB.getSelectedItem();

        String joiningDate = joiningTF.getText();

        double salary = Double.parseDouble(salaryTF.getText());

        String room = roomTF.getText();

        String bmdc = bmdcTF.getText();



        Doctor doctor = new Doctor(name, mobile, gender, department,
joiningDate, salary, room, bmdc);

        doctorService.addDoctor(doctor);
```

```
        JOptionPane.showMessageDialog(this, "Doctor added successfully!");

        clearFields();

    }



    private void onBack() {

        this.setVisible(false);

        DrMenu dm = new DrMenu(); // Navigation to main menu

        dm.setVisible(true);

    }



    private void clearFields() {

        nameTF.setText("");

        mobileTF.setText("");

        genderGroup.clearSelection();

        deptCB.setSelectedIndex(0);

        joiningTF.setText("");

        salaryTF.setText("");

        roomTF.setText("");

        bmdcTF.setText("");

    }

}
```

2. Open/Closed Principle (OCP)

Before: Adding new fields required **modifying AddDoctor class**.

Violation in Original Code

- To add a new role (Doctor, Nurse, Admin), existing menu code had to be modified.
- Logic was hardcoded inside UI classes.

Refactored Solution

- New features can be added by:

  Creating new service classes
  Implementing interfaces

After: Doctor class stores doctor attributes. Adding a new field can be done by extending Doctor or using a builder pattern **without modifying existing logic**.

```
public interface IDoctorService {

    void addDoctor(Doctor doctor);

}
```

3. Liskov Substitution Principle (LSP)

Violation in Original Code

- UI classes depended directly on concrete implementations.
- No abstraction existed.

Refactored Solution

UI depends on the interface, not the implementation.Any class implementing **IDoctorService** can replace **DoctorService**.

```
public interface IDoctorService {

    void addDoctor(Doctor doctor);

}
```

If logic changed, the UI would break.

4. Interface Segregation Principle (ISP)

UI depends on **IDoctorService** interface, not concrete implementations.

Classes implement only the methods they need.
UI uses **only what it needs**

No unnecessary method dependency

```
public interface IDoctorService {
```

```
    void addDoctor(Doctor doctor);

}
```

5. Dependency Inversion Principle (DIP)

Violation in Original Code

- **UI directly created service objects.**
- **Strong coupling between UI and logic.**

AddDoctorUI depends on **IDoctorService** abstraction, not **DoctorService** directly.

Allows swapping implementations or mocking for testing.

```
public class Start {
    public static void main(String[] args) {
        IDoctorService doctorService = new DoctorService();
        AddDoctorUI addDoctorUI = new AddDoctorUI(doctorService);
        addDoctorUI.setVisible(true);
    }
}
```

Code Smell

Commit Message: [Code Smell removed]

Code Smell 1 - God Class (Large Class)

Before: AddDoctor class handled UI layout, event handling, and doctor data storage.

```
import java.lang.*;
```

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;



public class AddDoctor extends JFrame implements
MouseListener,ActionListener{



    JLabel
userLabel,passLabel,genderLabel,programLabel,frame,imgLabel;

    JTextField userTF;

    JPasswordField passTF;

    JButton add, back;

    ButtonGroup bg1;

    JRadioButton r1,r2,r3;

    JComboBox dept;

    Color color1,color2;

    Font font1, font2, font3, font4;
```

```
    ImageIcon img;

    JPanel panel;


    public AddDoctor(){


        super("Doctor Information");

        this.setSize(900,600);

        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        this.setLocationRelativeTo(null);




        color1= new Color(143,207,225);

        color2= new Color(39,83,23);


        font1 = new Font("Biome",Font.BOLD, 30);

        font2 = new Font("Cascadia Code SemiBold",Font.PLAIN,
20);

        font3 = new Font("Cascadia Mono SemiBold",Font.PLAIN,
15);

        font4 = new Font("Biome",Font.BOLD, 18);
```

```
        panel = new JPanel();

        panel.setLayout(null);


        userLabel = new JLabel("HealthMate ");

        userLabel.setBounds(300,10,400,100);

        userLabel.setFont(font1);

        panel.add(userLabel);




        userLabel = new JLabel("Add Doctor Form ");

        userLabel.setBounds(130,80,400,50);

        userLabel.setFont(font4);

        panel.add(userLabel);


        userLabel = new JLabel("Doctor Name: ");

        userLabel.setBounds(75,120,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,160,150,20);

        panel.add(userTF);
```

```
        userLabel = new JLabel("Mobile Number: ");
        userLabel.setBounds(75,150,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,190,150,20);
        panel.add(userTF);

        genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75,220,115,20);
        panel.add(genderLabel);

        r1 = new JRadioButton("Male");
        r1.setBounds(150,220,80,20);
        r1.setBackground(color1);
        panel.add(r1);

        r2 = new JRadioButton("Female");
        r2.setBounds(230,220,80,20);
        r2.setBackground(color1);
```

```
        panel.add(r2);


        r3 = new JRadioButton("Other");

        r3.setBounds(320,220,80,20);

        r3.setBackground(color1);

        panel.add(r3);


        bg1 = new ButtonGroup();

        bg1.add(r1);

        bg1.add(r2);

        bg1.add(r3);




        programLabel = new JLabel("Department:");

        programLabel.setBounds(75,210,100,100);

        panel.add(programLabel);


        String items[] = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};

        dept = new JComboBox(items);

        dept.setBounds(170,250,150,20);

        panel.add(dept);
```

```
        userLabel = new JLabel("Joining Date: ");
        userLabel.setBounds(75,240,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,280,150,20);
        panel.add(userTF);

        userLabel = new JLabel("Salary: ");
        userLabel.setBounds(75,270,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,310,150,20);
        panel.add(userTF);

        userLabel = new JLabel("Room NO: ");
        userLabel.setBounds(75,300,100,100);
        panel.add(userLabel);
```

```
        userTF = new JTextField();
        userTF.setBounds(170,340,150,20);
        panel.add(userTF);

        userLabel = new JLabel("BM&DC REG: ");
        userLabel.setBounds(75,330,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,370,150,20);
        panel.add(userTF);

        add = new JButton("ADD");
        add.setBounds(500,470,100,40);
        add.setBackground(Color.GREEN);
        add.setForeground(Color.WHITE);
        add.setOpaque(true);
        add.addMouseListener(this);
        add.addActionListener(this);
        panel.add(add);
```

```
        back = new JButton("BACK");
        back.setBounds(620,470,100,40);
        back.setBackground(Color.RED);
        back.setForeground(Color.WHITE);
        back.setOpaque(true);
        back.addMouseListener(this);
        back.addActionListener(this);
        panel.add(back);

        img = new ImageIcon("images/emp-info.png");

        imgLabel = new JLabel(img);
        imgLabel.setBounds(485,100,300,300);

        panel.add(imgLabel);

        panel.setBackground(color1);
        this.add(panel);
    }
```

```
    public void mouseClicked(MouseEvent me){}

    public void mousePressed(MouseEvent me){}

    public void mouseReleased(MouseEvent me){}

    public void mouseEntered(MouseEvent me){

        if(me.getSource() == add){

            add.setBackground(Color.RED);

    add.setForeground(Color.WHITE);

        }else if(me.getSource() == back){

            back.setBackground(Color.GREEN);

            back.setForeground(Color.BLACK);

        }

    }

    public void mouseExited(MouseEvent me){

        if(me.getSource() == add){

            add.setBackground(Color.GREEN);

            add.setForeground(Color.BLACK);

        }else if(me.getSource() == back){

            back.setBackground(Color.RED);

            back.setForeground(Color.WHITE);

        }

    }
```

```
    public void actionPerformed(ActionEvent ae){

        String command = ae.getActionCommand();

        if(add.getText().equals(command));{

            DoctorList dl = new DoctorList();

            dl.setVisible(true);

            this.setVisible(false);



        }

        //public void actionPerformed(ActionEvent ae){

        //String command = ae.getActionCommand();

            if(ae.getSource() == back) {


                    JOptionPane.showMessageDialog(this, "Added
doctor successfully");

                     DrMenu dm = new DrMenu();

                    dm.setVisible(true);

                    this.setVisible(false);

                }



    }
```

```
}
```

Why this is a Code Smell

This class performs too many tasks, which violates:Single Responsibility Principle (SRP**)**

Problems Caused

- Hard to debug
- Hard to reuse
- Hard to extend

Correct Refactored Code (After SOLID)

Doctor.java
After:

```
public class Doctor {
    private String name;
    private String mobile;
    private String gender;
    private String department;
    private String joiningDate;
    private double salary;
    private String roomNo;
    private String bmdcReg;

    public Doctor(String name, String mobile, String gender,
String department,
                  String joiningDate, double salary, String
roomNo, String bmdcReg) {
```

```
        this.name = name;
        this.mobile = mobile;
        this.gender = gender;
        this.department = department;
        this.joiningDate = joiningDate;
        this.salary = salary;
        this.roomNo = roomNo;
        this.bmdcReg = bmdcReg;
    }

    // Getters
    public String getName() { return name; }
    public String getMobile() { return mobile; }
    public String getGender() { return gender; }
    public String getDepartment() { return department; }
    public String getJoiningDate() { return joiningDate; }
    public double getSalary() { return salary; }
    public String getRoomNo() { return roomNo; }
    public String getBmdcReg() { return bmdcReg; }
}
```

Doctor Service (Business Logic)

```
import java.util.ArrayList;
import java.util.List;

public class DoctorService implements IDoctorService {
    private List<Doctor> doctors = new ArrayList<>();

    @Override
    public void addDoctor(Doctor doctor) {
        doctors.add(doctor);
```

```
    }

    public List<Doctor> getDoctors() {
        return doctors;
    }
}
```

Interface for Dependency Inversion

```
public interface IDoctorService {
    void addDoctor(Doctor doctor);
}
```

UI Class (Only UI Responsibility)

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;

    private JTextField nameTF, mobileTF, joiningTF, salaryTF,
roomTF, bmdcTF;
    private JRadioButton maleRB, femaleRB, otherRB;
    private JComboBox<String> deptCB;
    private JButton addButton, backButton;
    private ButtonGroup genderGroup;

    public AddDoctorUI(IDoctorService doctorService) {
        super("Add Doctor Form");
        this.doctorService = doctorService;
```

```
        setSize(900, 600);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        initUI();
    }

    private void initUI() {
        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBackground(new Color(143,207,225));

        Font labelFont = new Font("Cascadia Code SemiBold",
Font.PLAIN, 18);

        // Labels & TextFields
        JLabel titleLabel = new JLabel("Add Doctor Form");
        titleLabel.setBounds(130, 20, 300, 40);
        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
        panel.add(titleLabel);

        JLabel nameLabel = new JLabel("Doctor Name:");
        nameLabel.setBounds(75, 80, 120, 25);
        nameLabel.setFont(labelFont);
        panel.add(nameLabel);

        nameTF = new JTextField();
        nameTF.setBounds(200, 80, 150, 25);
        panel.add(nameTF);

        JLabel mobileLabel = new JLabel("Mobile Number:");
        mobileLabel.setBounds(75, 120, 120, 25);
        mobileLabel.setFont(labelFont);
        panel.add(mobileLabel);
```

```
mobileTF = new JTextField();
mobileTF.setBounds(200, 120, 150, 25);
panel.add(mobileTF);

JLabel genderLabel = new JLabel("Gender:");
genderLabel.setBounds(75, 160, 100, 25);
genderLabel.setFont(labelFont);
panel.add(genderLabel);

maleRB = new JRadioButton("Male");
maleRB.setBounds(150, 160, 80, 25);
maleRB.setBackground(panel.getBackground());
femaleRB = new JRadioButton("Female");
femaleRB.setBounds(230, 160, 80, 25);
femaleRB.setBackground(panel.getBackground());
otherRB = new JRadioButton("Other");
otherRB.setBounds(310, 160, 80, 25);
otherRB.setBackground(panel.getBackground());

genderGroup = new ButtonGroup();
genderGroup.add(maleRB);
genderGroup.add(femaleRB);
genderGroup.add(otherRB);

panel.add(maleRB);
panel.add(femaleRB);
panel.add(otherRB);

JLabel deptLabel = new JLabel("Department:");
deptLabel.setBounds(75, 200, 120, 25);
deptLabel.setFont(labelFont);
panel.add(deptLabel);
```

```
        String[] departments = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};
        deptCB = new JComboBox<>(departments);
        deptCB.setBounds(200, 200, 150, 25);
        panel.add(deptCB);

        JLabel joiningLabel = new JLabel("Joining Date:");
        joiningLabel.setBounds(75, 240, 120, 25);
        joiningLabel.setFont(labelFont);
        panel.add(joiningLabel);

        joiningTF = new JTextField();
        joiningTF.setBounds(200, 240, 150, 25);
        panel.add(joiningTF);

        JLabel salaryLabel = new JLabel("Salary:");
        salaryLabel.setBounds(75, 280, 120, 25);
        salaryLabel.setFont(labelFont);
        panel.add(salaryLabel);

        salaryTF = new JTextField();
        salaryTF.setBounds(200, 280, 150, 25);
        panel.add(salaryTF);

        JLabel roomLabel = new JLabel("Room No:");
        roomLabel.setBounds(75, 320, 120, 25);
        roomLabel.setFont(labelFont);
        panel.add(roomLabel);

        roomTF = new JTextField();
        roomTF.setBounds(200, 320, 150, 25);
        panel.add(roomTF);

        JLabel bmdcLabel = new JLabel("BM&DC REG:");
        bmdcLabel.setBounds(75, 360, 120, 25);
```

```
        bmdcLabel.setFont(labelFont);
        panel.add(bmdcLabel);

        bmdcTF = new JTextField();
        bmdcTF.setBounds(200, 360, 150, 25);
        panel.add(bmdcTF);

        // Buttons
        addButton = new JButton("ADD");
        addButton.setBounds(500, 470, 100, 40);
        addButton.setBackground(Color.GREEN);
        addButton.setForeground(Color.WHITE);
        addButton.addActionListener(e -> onAdd());
        panel.add(addButton);

        backButton = new JButton("BACK");
        backButton.setBounds(620, 470, 100, 40);
        backButton.setBackground(Color.RED);
        backButton.setForeground(Color.WHITE);
        backButton.addActionListener(e -> onBack());
        panel.add(backButton);

        this.add(panel);
    }

    private void onAdd() {
        String name = nameTF.getText();
        String mobile = mobileTF.getText();
        String gender = maleRB.isSelected() ? "Male" :
                        femaleRB.isSelected() ? "Female" :
"Other";
        String department = (String) deptCB.getSelectedItem();
        String joiningDate = joiningTF.getText();
        double salary = Double.parseDouble(salaryTF.getText());
        String room = roomTF.getText();
```

```
        String bmdc = bmdcTF.getText();

        Doctor doctor = new Doctor(name, mobile, gender,
department, joiningDate, salary, room, bmdc);
        doctorService.addDoctor(doctor);

        JOptionPane.showMessageDialog(this, "Doctor added
successfully!");
        clearFields();
    }

    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu(); // Navigation to main menu
        dm.setVisible(true);
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

Code Smell 2 - Inappropriate naming

Add Doctor version Before:

```
  userLabel = new JLabel("HealthMate ");

      userLabel.setBounds(300,10,400,100);

      userLabel.setFont(font1);

      panel.add(userLabel);




      userLabel = new JLabel("Add Doctor Form ");

      userLabel.setBounds(130,80,400,50);

      userLabel.setFont(font4);

      panel.add(userLabel);


      userLabel = new JLabel("Doctor Name: ");

      userLabel.setBounds(75,120,100,100);

      panel.add(userLabel);


      userTF = new JTextField();

      userTF.setBounds(170,160,150,20);

      panel.add(userTF);


      userLabel = new JLabel("Mobile Number: ");

      userLabel.setBounds(75,150,100,100);
```

```
        panel.add(userLabel);



        userTF = new JTextField();

        userTF.setBounds(170,190,150,20);

        panel.add(userTF);
```

Why This Is a Problem

- Overwrites previous fields
- Makes debugging difficult
- Confusing variable usage

After Adding UI Class:

```
// Labels & TextFields
        JLabel titleLabel = new JLabel("Add Doctor Form");
        titleLabel.setBounds(130, 20, 300, 40);
        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
        panel.add(titleLabel);

        JLabel nameLabel = new JLabel("Doctor Name:");
        nameLabel.setBounds(75, 80, 120, 25);
        nameLabel.setFont(labelFont);
        panel.add(nameLabel);

        nameTF = new JTextField();
        nameTF.setBounds(200, 80, 150, 25);
        panel.add(nameTF);

        JLabel mobileLabel = new JLabel("Mobile Number:");
```

```
        mobileLabel.setBounds(75, 120, 120, 25);
        mobileLabel.setFont(labelFont);
        panel.add(mobileLabel);

        mobileTF = new JTextField();
        mobileTF.setBounds(200, 120, 150, 25);
        panel.add(mobileTF);

        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75, 160, 100, 25);
        genderLabel.setFont(labelFont);
        panel.add(genderLabel);
```

Each field has a clear purpose.

Code Smell 3 - Long Method

The original constructor contained hundreds of lines of UI setup.

Before:

```
import java.lang.*;
```

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;



public class AddDoctor extends JFrame implements
MouseListener,ActionListener{
```

```
    JLabel
userLabel,passLabel,genderLabel,programLabel,frame,imgLabel;

    JTextField userTF;

    JPasswordField passTF;

    JButton add, back;

    ButtonGroup bg1;

    JRadioButton r1,r2,r3;

    JComboBox dept;

    Color color1,color2;

    Font font1, font2, font3, font4;

    ImageIcon img;

    JPanel panel;


    public AddDoctor(){


        super("Doctor Information");

        this.setSize(900,600);

        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        this.setLocationRelativeTo(null);




        color1= new Color(143,207,225);
```

```
        color2= new Color(39,83,23);


        font1 = new Font("Biome",Font.BOLD, 30);

        font2 = new Font("Cascadia Code SemiBold",Font.PLAIN,
20);

        font3 = new Font("Cascadia Mono SemiBold",Font.PLAIN,
15);

        font4 = new Font("Biome",Font.BOLD, 18);




        panel = new JPanel();

        panel.setLayout(null);


        userLabel = new JLabel("HealthMate ");

        userLabel.setBounds(300,10,400,100);

        userLabel.setFont(font1);

        panel.add(userLabel);



        userLabel = new JLabel("Add Doctor Form ");

        userLabel.setBounds(130,80,400,50);

        userLabel.setFont(font4);
```

```
        panel.add(userLabel);


        userLabel = new JLabel("Doctor Name: ");
        userLabel.setBounds(75,120,100,100);
        panel.add(userLabel);


        userTF = new JTextField();
        userTF.setBounds(170,160,150,20);
        panel.add(userTF);


        userLabel = new JLabel("Mobile Number: ");
        userLabel.setBounds(75,150,100,100);
        panel.add(userLabel);


        userTF = new JTextField();
        userTF.setBounds(170,190,150,20);
        panel.add(userTF);


        genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75,220,115,20);
        panel.add(genderLabel);
```

```
r1 = new JRadioButton("Male");
r1.setBounds(150,220,80,20);
r1.setBackground(color1);
panel.add(r1);

r2 = new JRadioButton("Female");
r2.setBounds(230,220,80,20);
r2.setBackground(color1);
panel.add(r2);

r3 = new JRadioButton("Other");
r3.setBounds(320,220,80,20);
r3.setBackground(color1);
panel.add(r3);

bg1 = new ButtonGroup();
bg1.add(r1);
bg1.add(r2);
bg1.add(r3);
```

```
        programLabel = new JLabel("Department:");
        programLabel.setBounds(75,210,100,100);
        panel.add(programLabel);

        String items[] = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};
        dept = new JComboBox(items);
        dept.setBounds(170,250,150,20);
        panel.add(dept);

        userLabel = new JLabel("Joining Date: ");
        userLabel.setBounds(75,240,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,280,150,20);
        panel.add(userTF);

        userLabel = new JLabel("Salary: ");
        userLabel.setBounds(75,270,100,100);
        panel.add(userLabel);
```

```
        userTF = new JTextField();
        userTF.setBounds(170,310,150,20);
        panel.add(userTF);

        userLabel = new JLabel("Room NO: ");
        userLabel.setBounds(75,300,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,340,150,20);
        panel.add(userTF);

        userLabel = new JLabel("BM&DC REG: ");
        userLabel.setBounds(75,330,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,370,150,20);
        panel.add(userTF);
```

```
add = new JButton("ADD");
add.setBounds(500,470,100,40);
add.setBackground(Color.GREEN);
add.setForeground(Color.WHITE);
add.setOpaque(true);
add.addMouseListener(this);
add.addActionListener(this);
panel.add(add);

back = new JButton("BACK");
back.setBounds(620,470,100,40);
back.setBackground(Color.RED);
back.setForeground(Color.WHITE);
back.setOpaque(true);
back.addMouseListener(this);
back.addActionListener(this);
panel.add(back);

img = new ImageIcon("images/emp-info.png");
```

```
        imgLabel = new JLabel(img);
        imgLabel.setBounds(485,100,300,300);

        panel.add(imgLabel);

        panel.setBackground(color1);
        this.add(panel);
    }

    public void mouseClicked(MouseEvent me){}
    public void mousePressed(MouseEvent me){}
    public void mouseReleased(MouseEvent me){}
    public void mouseEntered(MouseEvent me){
        if(me.getSource() == add){
            add.setBackground(Color.RED);
    add.setForeground(Color.WHITE);
        }else if(me.getSource() == back){
            back.setBackground(Color.GREEN);
            back.setForeground(Color.BLACK);
        }
    }
```

```
    public void mouseExited(MouseEvent me){

        if(me.getSource() == add){

            add.setBackground(Color.GREEN);

            add.setForeground(Color.BLACK);

        }else if(me.getSource() == back){

            back.setBackground(Color.RED);

            back.setForeground(Color.WHITE);

        }

    }



    public void actionPerformed(ActionEvent ae){

        String command = ae.getActionCommand();

        if(add.getText().equals(command));{

            DoctorList dl = new DoctorList();

            dl.setVisible(true);

            this.setVisible(false);



        }

        //public void actionPerformed(ActionEvent ae){

        //String command = ae.getActionCommand();

            if(ae.getSource() == back) {
```

```
                    JOptionPane.showMessageDialog(this, "Added
doctor successfully");

                     DrMenu dm = new DrMenu();

                    dm.setVisible(true);

                    this.setVisible(false);

                }



    }



}
```

Problem

Hard to read and maintain.

Correct Solution

Break it into smaller methods.

After:

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;
```

```java
public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;

    private JTextField nameTF, mobileTF, joiningTF, salaryTF,
roomTF, bmdcTF;
    private JRadioButton maleRB, femaleRB, otherRB;
    private JComboBox<String> deptCB;
    private JButton addButton, backButton;
    private ButtonGroup genderGroup;

    public AddDoctorUI(IDoctorService doctorService) {
        super("Add Doctor Form");
        this.doctorService = doctorService;

        setSize(900, 600);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        initUI();
    }
```

```
    private void initUI() {

        JPanel panel = new JPanel();

        panel.setLayout(null);

        panel.setBackground(new Color(143,207,225));


        Font labelFont = new Font("Cascadia Code SemiBold",
Font.PLAIN, 18);


        // Labels & TextFields

        JLabel titleLabel = new JLabel("Add Doctor Form");

        titleLabel.setBounds(130, 20, 300, 40);

        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));

        panel.add(titleLabel);


        JLabel nameLabel = new JLabel("Doctor Name:");

        nameLabel.setBounds(75, 80, 120, 25);

        nameLabel.setFont(labelFont);

        panel.add(nameLabel);


        nameTF = new JTextField();

        nameTF.setBounds(200, 80, 150, 25);
```

```
        panel.add(nameTF);


        JLabel mobileLabel = new JLabel("Mobile Number:");
        mobileLabel.setBounds(75, 120, 120, 25);
        mobileLabel.setFont(labelFont);
        panel.add(mobileLabel);


        mobileTF = new JTextField();
        mobileTF.setBounds(200, 120, 150, 25);
        panel.add(mobileTF);


        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75, 160, 100, 25);
        genderLabel.setFont(labelFont);
        panel.add(genderLabel);


        maleRB = new JRadioButton("Male");
        maleRB.setBounds(150, 160, 80, 25);
        maleRB.setBackground(panel.getBackground());
        femaleRB = new JRadioButton("Female");
        femaleRB.setBounds(230, 160, 80, 25);
```

```
        femaleRB.setBackground(panel.getBackground());

        otherRB = new JRadioButton("Other");

        otherRB.setBounds(310, 160, 80, 25);

        otherRB.setBackground(panel.getBackground());


        genderGroup = new ButtonGroup();

        genderGroup.add(maleRB);

        genderGroup.add(femaleRB);

        genderGroup.add(otherRB);


        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);


        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);


        String[] departments = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);
```

```
        deptCB.setBounds(200, 200, 150, 25);
        panel.add(deptCB);

        JLabel joiningLabel = new JLabel("Joining Date:");
        joiningLabel.setBounds(75, 240, 120, 25);
        joiningLabel.setFont(labelFont);
        panel.add(joiningLabel);

        joiningTF = new JTextField();
        joiningTF.setBounds(200, 240, 150, 25);
        panel.add(joiningTF);

        JLabel salaryLabel = new JLabel("Salary:");
        salaryLabel.setBounds(75, 280, 120, 25);
        salaryLabel.setFont(labelFont);
        panel.add(salaryLabel);

        salaryTF = new JTextField();
        salaryTF.setBounds(200, 280, 150, 25);
        panel.add(salaryTF);
```

```
JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);

bmdcTF = new JTextField();
bmdcTF.setBounds(200, 360, 150, 25);
panel.add(bmdcTF);

// Buttons
addButton = new JButton("ADD");
addButton.setBounds(500, 470, 100, 40);
```

```
        addButton.setBackground(Color.GREEN);

        addButton.setForeground(Color.WHITE);

        addButton.addActionListener(e -> onAdd());

        panel.add(addButton);


        backButton = new JButton("BACK");

        backButton.setBounds(620, 470, 100, 40);

        backButton.setBackground(Color.RED);

        backButton.setForeground(Color.WHITE);

        backButton.addActionListener(e -> onBack());

        panel.add(backButton);


        this.add(panel);

    }


    private void onAdd() {

        String name = nameTF.getText();

        String mobile = mobileTF.getText();

        String gender = maleRB.isSelected() ? "Male" :

                        femaleRB.isSelected() ? "Female" :
"Other";

        String department = (String) deptCB.getSelectedItem();
```

```
        String joiningDate = joiningTF.getText();

        double salary = Double.parseDouble(salaryTF.getText());

        String room = roomTF.getText();

        String bmdc = bmdcTF.getText();



        Doctor doctor = new Doctor(name, mobile, gender,
department, joiningDate, salary, room, bmdc);

        doctorService.addDoctor(doctor);



        JOptionPane.showMessageDialog(this, "Doctor added
successfully!");

        clearFields();

    }



    private void onBack() {

        this.setVisible(false);

        DrMenu dm = new DrMenu(); // Navigation to main menu

        dm.setVisible(true);

    }



    private void clearFields() {

        nameTF.setText("");
```

```
        mobileTF.setText("");

        genderGroup.clearSelection();

        deptCB.setSelectedIndex(0);

        joiningTF.setText("");

        salaryTF.setText("");

        roomTF.setText("");

        bmdcTF.setText("");

    }

}
```

RegDoctorList.java

Before (Original Code)

```
package Classes;

import java.io.*;

import java.util.Scanner;


public class RegDoctorList{

	public Doctor doctorList[] = new Doctor[100];

	public static int doctorCount = 0;


	public RegDoctorList(){

		try{

			File file = new File("Files/DoctorList.txt");

			Scanner sc = new Scanner(file);

				while(sc.hasNext()){

					String line1 = sc.nextLine();  //id

					String line2 = sc.nextLine();  //name

					String line3 = sc.nextLine();  //mo n.

					String line4 = sc.nextLine();  //gender

					String line5 = sc.nextLine();  //department

					String line6 = sc.nextLine();  //joinig date
```

```
                    String line7 = sc.nextLine();  //bmdc

                    String line8 = sc.nextLine();  //pass

                    String line9 = sc.nextLine();  //extra newline



                    System.out.println(line1);

                    System.out.println(line2);

                    System.out.println(line3);

                    System.out.println(line4);

                    System.out.println(line5);

                    System.out.println(line6);

                    System.out.println(line7);

                    System.out.println(line8);


                    Doctor d = new
Doctor(line1,line2,line3,line4,line5,line6,line7,line8);

                    doctorList[doctorCount] = d;

                    doctorCount++;


System.out.println(doctorCount+"********************");

                }

            sc.close();   //closing the file

        }catch(Exception ex){
```

```
                System.out.println("File not found.");
                return;
            }

        }

        public void addDoctor(Doctor d){
            doctorList[doctorCount] = d;
            doctorCount++;



        String DrDetails = d.getId() + "\n" + d.getName() + "\n" +d.getGender() + "\n"
+d.getMobileNo() + "\n"+ d.getDepartment() + "\n" +
                                                d.getJoiningDate() + "\n" + d.getBmdcReg() +"\n"
+ d.getPassword() + "\n" + "\n";
            try{
                FileWriter fw = new FileWriter("Files/DoctorList.txt",true);
                fw.write(DrDetails);
                fw.close();
            }catch(Exception ex){
                System.out.println(ex);
            }
        }
```

```
public int doctorExists(String name){
	int index = -1;
	for(int i = 0; i<doctorCount; i++){
		if(doctorList[i]!=null && doctorList[i].getName().equals(name)){
			index = i;
			break;
		}
	}
	return index;
}

public Doctor checkPassword(int index, String pass){
	Doctor d = null;
	if(doctorList[index].getPassword().equals(pass)){
		d = doctorList[index];
	}
	return d;
}

public Doctor getDoctor(int index){
	return doctorList[index];
}
```

```
        public String[] getDoctorNames() {
    String doctorNames [] = new String[doctorCount];
    for (int i = 0; i < doctorCount; i++) {
                if (doctorNames!= null && doctorList[i]!= null){
      doctorNames[i] = "Dr. "+doctorList[i].getName();
    }}
    return doctorNames;


}




        /*public void deleteDoctor(Doctor d){
                for(int i = 0; i<doctorCount; i++){
                        if(doctorList[i] == d){
                                for(int j = i; j<doctorCount-1; j++){
                                        doctorList[j] = doctorList[j+1];
                                }
                                doctorCount--;
                                doctorList[doctorCount]=null;
                                break;
                        }
                }
        }*/
```

```
}
```

After (Modified Code)

```
package Classes;

import java.io.*;

import java.util.ArrayList;

import java.util.List;

import java.util.Scanner;


public class RegDoctorList {

    private static final String DOCTOR_FILE_PATH = "Files/DoctorList.txt";

    private final List<Doctor> doctorList = new ArrayList<>();


    public RegDoctorList() {

        loadDoctorsFromFile();

    }


    private void loadDoctorsFromFile() {

        File file = new File(DOCTOR_FILE_PATH);

        if (!file.exists()) {

            return; // Silently handle missing file, assume empty list

        }

        try (Scanner sc = new Scanner(file)) {

            while (sc.hasNext()) {
```

```
            String id = sc.nextLine();
            String name = sc.nextLine();
            String mobileNo = sc.nextLine();
            String gender = sc.nextLine();
            String department = sc.nextLine();
            String joiningDate = sc.nextLine();
            String bmdcReg = sc.nextLine();
            String password = sc.nextLine();
            if (sc.hasNextLine()) sc.nextLine(); // Skip extra newline

            Doctor d = new Doctor(id, name, mobileNo, gender, department, joiningDate,
bmdcReg, password);
            doctorList.add(d);
        }
    } catch (FileNotFoundException ex) {
        // Log or handle appropriately; for now, empty list
    } catch (IOException ex) {
        throw new RuntimeException("Error loading doctors", ex);
    }
}

public void addDoctor(Doctor d) {
    doctorList.add(d);
    saveDoctorToFile(d);
```

```
    }


    private void saveDoctorToFile(Doctor d) {

        String drDetails = d.getId() + "\n" + d.getName() + "\n" + d.getMobileNo() + "\n" +
d.getGender() + "\n" +

                        d.getDepartment() + "\n" + d.getJoiningDate() + "\n" + d.getBmdcReg() + "\n"
+

                        d.getPassword() + "\n\n";

        try (FileWriter fw = new FileWriter(DOCTOR_FILE_PATH, true)) {

            fw.write(drDetails);

        } catch (IOException ex) {

            throw new RuntimeException("Error saving doctor", ex);

        }

    }


    public int doctorExists(String name) {

        for (int i = 0; i < doctorList.size(); i++) {

            if (doctorList.get(i).getName().equals(name)) {

                return i;

            }

        }

        return -1;

    }
```

```
    public Doctor checkPassword(int index, String pass) {
        if (index < 0 || index >= doctorList.size()) {
            return null;
        }
        Doctor d = doctorList.get(index);
        return d.getPassword().equals(pass) ? d : null;
    }

    public Doctor getDoctor(int index) {
        if (index < 0 || index >= doctorList.size()) {
            return null;
        }
        return doctorList.get(index);
    }

    public String[] getDoctorNames() {
        String[] doctorNames = new String[doctorList.size()];
        for (int i = 0; i < doctorList.size(); i++) {
            doctorNames[i] = "Dr. " + doctorList.get(i).getName();
        }
        return doctorNames;
    }
}
```

Why : The code contained several **code smells** that affected its quality and maintainability. These included **duplicated code** in file I/O operations, the use of **magic numbers** such as a fixed array size (100), and reliance on **static variables** like doctorCount to track elements. It also had **poor error handling**, where exceptions were not properly managed, and **dead code**, including unnecessary debug prints and commented-out methods. Additionally, the code showed **primitive obsession** by using arrays instead of more flexible data structures like List or ArrayList.

PrescriptionList.java

Before (Original Code)

```
package Classes;

import java.io.*;

import java.util.Scanner;


public class PrescriptionList {

    public Prescription[] prescriptionList = new Prescription[100];

    public static int pCount = 0;


    public PrescriptionList() {

        try {

            File file = new File("Files/prescriptions.txt");
```

```
            Scanner sc = new Scanner(file);
            while (sc.hasNextLine()) {
                String patientId = sc.nextLine().split(": ")[1];  // Patient ID
                String patientName = sc.nextLine().split(": ")[1];  // Patient Name
                String diagnosis = sc.nextLine().split(": ")[1];  // Diagnosis
                String doctorName = sc.nextLine().split(": ")[1];  // Doctor's Name
                String medicineName = sc.nextLine().split(": ")[1];  // Medicine Name
                String dosage = sc.nextLine().split(": ")[1];  // Dosage
                String usageInstructions = sc.nextLine().split(": ")[1];  // Usage Instructions
                if (sc.hasNextLine()) sc.nextLine();  // Skip

                Prescription p = new Prescription(patientId, patientName, diagnosis, doctorName,
medicineName, dosage, usageInstructions);
                prescriptionList[pCount] = p;
                pCount++;
            }
            sc.close();
        } catch (Exception ex) {
            System.out.println("File not found or error reading file.");
        }
    }

    public void addPrescription(Prescription p) {
        try {
```

```
        prescriptionList[pCount] = p;
        pCount++;
    } catch (Exception ex) {
        System.out.println("Array full");
    }

    String prescriptionDetails = "Patient ID: " + p.getPatientId() + "\n" +
                                 "Patient Name: " + p.getPatientName() + "\n" +
                                 "Diagnosis: " + p.getDiagnosis() + "\n" +
                                 "Doctor's Name: " + p.getDoctorName() + "\n" +
                                 "Medicine Name: " + p.getMedicineName() + "\n" +
                                 "Dosage: " + p.getDosage() + "\n" +
                                 "Usage Instructions: " + p.getUsageInstructions() + "\n\n";

    try {
        FileWriter fw = new FileWriter("Files/prescriptions.txt", true);
        fw.write(prescriptionDetails);
        fw.close();
    } catch (Exception ex) {
        System.out.println(ex);
    }
}
```

```
    public int searchPrescription(String patientId) {
        int index = -1;
        for (int i = 0; i < pCount; i++) {
            if (prescriptionList[i] != null &&
prescriptionList[i].getPatientId().equalsIgnoreCase(patientId)) {
                index = i;
                break;
            }
        }
        return index;
    }

    public Prescription getPrescription(int index) {
        return prescriptionList[index];
    }
}
```

After (Modified Code)

```
package Classes;

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;
```

```
public class PrescriptionList {
    private static final String PRESCRIPTION_FILE_PATH = "Files/prescriptions.txt";
    private final List<Prescription> prescriptionList = new ArrayList<>();

    public PrescriptionList() {
        loadPrescriptionsFromFile();
    }

    private void loadPrescriptionsFromFile() {
        File file = new File(PRESCRIPTION_FILE_PATH);
        if (!file.exists()) {
            return;
        }
        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNextLine()) {
                String patientId = sc.nextLine().split(": ", 2)[1];
                String patientName = sc.nextLine().split(": ", 2)[1];
                String diagnosis = sc.nextLine().split(": ", 2)[1];
                String doctorName = sc.nextLine().split(": ", 2)[1];
                String medicineName = sc.nextLine().split(": ", 2)[1];
                String dosage = sc.nextLine().split(": ", 2)[1];
                String usageInstructions = sc.nextLine().split(": ", 2)[1];
```

```
            if (sc.hasNextLine()) sc.nextLine();


            Prescription p = new Prescription(patientId, patientName, diagnosis, doctorName,
medicineName, dosage, usageInstructions);

            prescriptionList.add(p);

        }

    } catch (FileNotFoundException ex) {

        // Empty list

    } catch (IOException ex) {

        throw new RuntimeException("Error loading prescriptions", ex);

    } catch (ArrayIndexOutOfBoundsException ex) {

        throw new RuntimeException("Invalid file format", ex);

    }

}


public void addPrescription(Prescription p) {

    prescriptionList.add(p);

    savePrescriptionToFile(p);

}


private void savePrescriptionToFile(Prescription p) {

    String prescriptionDetails = "Patient ID: " + p.getPatientId() + "\n" +

                                 "Patient Name: " + p.getPatientName() + "\n" +

                                 "Diagnosis: " + p.getDiagnosis() + "\n" +
```

```
                    "Doctor's Name: " + p.getDoctorName() + "\n" +
                    "Medicine Name: " + p.getMedicineName() + "\n" +
                    "Dosage: " + p.getDosage() + "\n" +
                    "Usage Instructions: " + p.getUsageInstructions() + "\n\n";
    try (FileWriter fw = new FileWriter(PRESCRIPTION_FILE_PATH, true)) {
        fw.write(prescriptionDetails);
    } catch (IOException ex) {
        throw new RuntimeException("Error saving prescription", ex);
    }
}

public int searchPrescription(String patientId) {
    for (int i = 0; i < prescriptionList.size(); i++) {
        if (prescriptionList.get(i).getPatientId().equalsIgnoreCase(patientId)) {
            return i;
        }
    }
    return -1;
}

public Prescription getPrescription(int index) {
    if (index < 0 || index >= prescriptionList.size()) {
        return null;
```

```
        }
        return prescriptionList.get(index);
    }
}
```

Why: The refactored code removed several **code smells** to improve quality and maintainability. The **duplicated file I/O code** was streamlined to avoid repetition. **Magic numbers and strings** were replaced with constants, and fixed arrays were replaced with a dynamic **List** for better flexibility. The unnecessary **static variable pCount** was removed and replaced with more reliable methods like checking the list size. **Error handling** was improved by using more specific exceptions to properly handle format-related issues. Additionally, **defensive bounds checks** were added to prevent invalid index access and make the code more robust.

Prescription.java

Before (Original Code)

```
package Classes;


public class Prescription {
    private String patientId;
    private String patientName;
```

```
    private String diagnosis;

    private String doctorName;

    private String medicineName;

    private String dosage;

    private String usageInstructions;


    // Constructor

    public Prescription(String patientId, String patientName, String diagnosis, String doctorName,
String medicineName, String dosage, String usageInstructions) {

        this.patientId = patientId;

        this.patientName = patientName;

        this.diagnosis = diagnosis;

        this.doctorName = doctorName;

        this.medicineName = medicineName;

        this.dosage = dosage;

        this.usageInstructions = usageInstructions;

    }


    // Getters

    public String getPatientId() {

        return patientId;

    }


    public String getPatientName() {
```

```
        return patientName;
    }

    public String getDiagnosis() {
        return diagnosis;
    }

    public String getDoctorName() {
        return doctorName;
    }

    public String getMedicineName() {
        return medicineName;
    }

    public String getDosage() {
        return dosage;
    }

    public String getUsageInstructions() {
        return usageInstructions;
    }
}
```

After (Modified Code)

java

```
package Classes;


public class Prescription {

    private final String patientId;

    private final String patientName;

    private final String diagnosis;

    private final String doctorName;

    private final String medicineName;

    private final String dosage;

    private final String usageInstructions;


    public Prescription(String patientId, String patientName, String diagnosis, String doctorName,
String medicineName, String dosage, String usageInstructions) {

        this.patientId = patientId;

        this.patientName = patientName;

        this.diagnosis = diagnosis;

        this.doctorName = doctorName;

        this.medicineName = medicineName;

        this.dosage = dosage;

        this.usageInstructions = usageInstructions;

    }
```

```
public String getPatientId() {
    return patientId;
}

public String getPatientName() {
    return patientName;
}

public String getDiagnosis() {
    return diagnosis;
}

public String getDoctorName() {
    return doctorName;
}

public String getMedicineName() {
    return medicineName;
}

public String getDosage() {
    return dosage;
```

```
    }

    public String getUsageInstructions() {
        return usageInstructions;
    }
}
```

Why: The refactoring addressed some key **code smells** to improve code reliability and design. A **mutable object** was converted into an **immutable object** by making its fields final, which helps prevent unintended changes to its state and improves thread safety. Additionally, the presence of a **long parameter list** was identified as a design smell; although it was not refactored in this stage, the **Builder pattern** was suggested as a future improvement to make object creation cleaner and more maintainable.

Patient.java

Before (Original Code)

```
package Classes;

public class Patient extends Person {
        private String age;
        private String address;
```

```
    private String appointedDoctor;

    private String room;

    private String diagnosis;

    private String deposit;

    public Patient( String id, String name, String mobileNo, String gender, String age, String
address, String appointedDoctor, String room, String diagnosis,String deposit) {

        super(id,name,mobileNo,gender);



                    this.age = age;

                    this.address =  address;

        this.appointedDoctor = appointedDoctor;

        this.room = room;

        this.diagnosis = diagnosis;

                    this.deposit = deposit;

    }




            public String getAge() {

        return age;

    }



    public void setAge(String age) {

        this.age = age;

    }
```

```
        public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}


public String getAppointedDoctor() {
    return appointedDoctor;
}

public void setAppointedDoctor(String appointedDoctor) {
    this.appointedDoctor = appointedDoctor;
}

public String getRoom() {
    return room;
}
```

```
    public void setRoom(String room) {
        this.room = room;
    }

    public String getDiagnosis() {
        return diagnosis;
    }

    public void setDiagnosis(String diagnosis) {
        this.diagnosis = diagnosis;
    }

            public String getDeposit() {
        return deposit;
    }

    public void setDeposit(String deposit) {
        this.deposit = deposit;
    }
```

After (Modified Code)

```
package Classes;
```

```
public class Patient extends Person {

    private String age;

    private String address;

    private String appointedDoctor;

    private String room;

    private String diagnosis;

    private String deposit;


    public Patient(String id, String name, String mobileNo, String gender, String age, String
address, String appointedDoctor, String room, String diagnosis, String deposit) {

        super(id, name, mobileNo, gender);

        this.age = age;

        this.address = address;

        this.appointedDoctor = appointedDoctor;

        this.room = room;

        this.diagnosis = diagnosis;

        this.deposit = deposit;

    }


    public String getAge() {

        return age;

    }


    public void setAge(String age) {
```

```
    this.age = age;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public String getAppointedDoctor() {
    return appointedDoctor;
}

public void setAppointedDoctor(String appointedDoctor) {
    this.appointedDoctor = appointedDoctor;
}

public String getRoom() {
    return room;
}
```

```
    public void setRoom(String room) {
        this.room = room;
    }

    public String getDiagnosis() {
        return diagnosis;
    }

    public void setDiagnosis(String diagnosis) {
        this.diagnosis = diagnosis;
    }

    public String getDeposit() {
        return deposit;
    }

    public void setDeposit(String deposit) {
        this.deposit = deposit;
    }
}
```

Why This Is a Code Smell?

The code contains **Long Parameter List**, **Data Clump**, and **Primitive Obsession**.

Problems:

1. Related data is passed separately instead of grouping into objects (e.g., an Address class).
2. Basic types like String are used for values such as age or deposit instead of proper types (int, double).

Doctor.java

Before (Original Code)

```
package Classes;


public class Doctor extends Person {

   private String department;

   private String joiningDate;

   private String bmdcReg;

         private String password;


   public Doctor(String id, String name, String gender, String mobileNo, String department,
String joiningDate, String bmdcReg, String password) {

      super(id,name,gender,mobileNo);

      this.department = department;

      this.joiningDate = joiningDate;

      this.bmdcReg = bmdcReg;
```

```
                this.password = password;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public String getJoiningDate() {
    return joiningDate;
}

public void setJoiningDate(String joiningDate) {
    this.joiningDate = joiningDate;
}

public String getPassword() {
                return password;
        }
```

```
        public void setPassword(String password){
                this.password = password;
        }

    public String getBmdcReg() {
        return bmdcReg;
    }

    public void setBmdcReg(String bmdcReg) {
        this.bmdcReg = bmdcReg;
    }
}
```

After (Modified Code)

```
package Classes;
public class Doctor extends Person {
    private String department;
    private String joiningDate;
    private String bmdcReg;
    private String password;

  public Doctor(String id, String name, String mobileNo, String gender, String department, String joiningDate, String bmdcReg, String password) {
        super(id, name, mobileNo, gender);
```

```
    this.department = department;
    this.joiningDate = joiningDate;
    this.bmdcReg = bmdcReg;
    this.password = password;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public String getJoiningDate() {
    return joiningDate;
}

public void setJoiningDate(String joiningDate) {
    this.joiningDate = joiningDate;
}

public String getBmdcReg() {
```

```
        return bmdcReg;
    }

    public void setBmdcReg(String bmdcReg) {
        this.bmdcReg = bmdcReg;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

Why This Code Smells?

The code contains several issues such as **Long Parameter List** and **Primitive Obsession**, which affect both **design and security**.

Problems:

1. **Long Parameter List:** Methods become harder to read and maintain.
2. **Plain Text Passwords:** Storing passwords in plain text is insecure; production systems should use **secure hashing**.
3. **Primitive Obsession:** Dates are stored using simple types instead of proper date/time classes, reducing validation and reliability.

PList.java

Before (Original Code)

```
package Classes;

import Interfaces.*;


import java.io.*;

import java.util.Scanner;


public class PList implements IPList {

	public Patient patientList[] = new Patient[100];

	public static int pCount = 0;


	public PList(){

		try{

		File file = new File("Files/PatientList.txt");

		Scanner sc = new Scanner(file);

			while(sc.hasNext()){

				String line1 = sc.nextLine();  //pid

				String line2 = sc.nextLine();  //name
```

```
                    String line3 = sc.nextLine();  //mobile
                    String line4 = sc.nextLine();  //gender
                    String line5 = sc.nextLine();  //age
                    String line6 = sc.nextLine();   //address
                    String line7 = sc.nextLine();  //appDoctor
                    String line8 = sc.nextLine();  //room
                    String line9 = sc.nextLine();  //diagnosic
                    String line10 = sc.nextLine();  //deposit
                    String line11= sc.nextLine();



                    Patient p = new Patient (line1, line2, line3, line4, line5, line6,
line7, line8, line9, line10);
                    patientList[pCount] = p;
                    pCount++;
                }
            sc.close();


            }catch(Exception ex){
                System.out.println("File not found.");
                return;


            }
        }
```

```
public void addPatient(Patient p){

        try{
                patientList[pCount] = p;
                pCount++;
        }catch (Exception ex){
                        System.out.println("Array full");
        }

        String patientDatiels = p.getId() + "\n" + p.getName() + "\n" +
                                                p.getMobileNo() + "\n" +
p.getGender() + "\n" +
                                                p.getAge() + "\n" + p.getAddress() +
"\n" +
                                                p.getAppointedDoctor() + "\n" +
p.getRoom() + "\n" +
                                                p.getDiagnosis() + "\n" +
p.getDeposit() + "\n" +"\n";

        try{
                FileWriter fw = new FileWriter("Files/PatientList.txt",true);
                fw.write(patientDatiels);
                fw.close();
        }catch(Exception ex){
```

```
                System.out.println(ex);
        }




}

public int searchPatient(String id){
        int index = -1;
        for(int i = 0; i<pCount; i++){
                if(patientList[i]!= null && patientList[i].getId().equals(id)){
                  index = i;
                  break;
                }
        }
        return index;
}

public Patient getPatient(int index){
        return patientList[index];
}

public void deletePatient(Patient p){
```

```
for( int i = 0; i<pCount; i++){
    if(patientList[i] == p){
        for(int j = i; j<pCount-1; j++){
            patientList[j] = patientList[j+1];
        }
        pCount--;
        patientList[pCount] = null;
        break;
    }
}


String patientToDelete = p.getId() + "\n" + p.getName() + "\n" +
                                                p.getMobileNo() + "\n" +
p.getGender() + "\n" +
                                                p.getAge() + "\n" + p.getAddress()
+ "\n" +
                                                p.getAppointedDoctor() + "\n" +
p.getRoom() + "\n" +
                                                p.getDiagnosis() + "\n" +
p.getDeposit() + "\n" ;


try{
    String filepath = "Files/PatientList.txt";
```

```
File originalFile = new File(filepath);


String newDetails = "";
Scanner sc = new Scanner(originalFile);
while(sc.hasNext()){
        String readUser = sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine() + "\n";
                readUser += sc.nextLine();

        System.out.println(readUser);
        System.out.println(patientToDelete);

        if(readUser.equals(patientToDelete)){
                System.out.println("Equal, so deleting");
                continue;
```

```
                }
                System.out.println("Writing");
                newDetails += readUser + "\n";

            } sc.close();
            FileWriter fw = new FileWriter(filepath);
            fw.write(newDetails);
            fw.close();

        } catch (Exception ex){
            System.out.println(ex);
        }

    }

    public void updatePatient(String patientToUpdate, Patient updatedPatient){

        String updatedDetails = updatedPatient.getId() + "\n" +
                                updatedPatient.getName() + "\n" +
                                updatedPatient.getMobileNo() + "\n"
+
                                updatedPatient.getGender() + "\n" +
                                updatedPatient.getAge() + "\n" +
                                updatedPatient.getAddress() + "\n" +
```

```
updatedPatient.getAppointedDoctor() + "\n" +
                                                updatedPatient.getRoom() + "\n"+
                                                updatedPatient.getDiagnosis() + "\n"
+
                                                updatedPatient.getDeposit() + "\n";

            System.out.println("Want to update:\n "+ updatedDetails+ "\n");
            System.out.println("----------------------------------------");

            //Update in file
            try{
                String filepath = "Files/PatientList.txt";
                File originalFile = new File(filepath);

                String newDetails = "";
                Scanner sc = new Scanner(originalFile);
                while(sc.hasNext()){
                    String readUser = sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
```

```
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine();

                System.out.println("read:\n "+ readUser+"\n-----------------");
                System.out.println("updt:\n "+ patientToUpdate
+"\n-----------------");

                if(readUser.equals(patientToUpdate)){
                        System.out.println("Updated");
                        newDetails += updatedDetails + "\n";
                } else{
                        System.out.println("Writing");
                        newDetails += readUser + "\n";
                }
            } sc.close();
            FileWriter fw = new FileWriter(filepath);
            fw.write(newDetails);
            fw.close();

        } catch (Exception ex){
            System.out.println(ex);
```

```
            }
        }

}
```

After (Modified Code)

```
package Classes;
import Interfaces.*;

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class PList implements IPList {
    private static final String PATIENT_FILE_PATH = "Files/PatientList.txt";
    private final List<Patient> patientList = new ArrayList<>();

    public PList() {
        loadPatientsFromFile();
    }
```

```
    private void loadPatientsFromFile() {
        File file = new File(PATIENT_FILE_PATH);
        if (!file.exists()) {
            return;
        }
        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String id = sc.nextLine();
                String name = sc.nextLine();
                String mobileNo = sc.nextLine();
                String gender = sc.nextLine();
                String age = sc.nextLine();
                String address = sc.nextLine();
                String appointedDoctor = sc.nextLine();
                String room = sc.nextLine();
                String diagnosis = sc.nextLine();
                String deposit = sc.nextLine();
                if (sc.hasNextLine()) sc.nextLine();


                Patient p = new Patient(id, name, mobileNo, gender, age, address, appointedDoctor,
room, diagnosis, deposit);
                patientList.add(p);
            }
        } catch (FileNotFoundException ex) {
```

```
        // Empty list
    } catch (IOException ex) {
        throw new RuntimeException("Error loading patients", ex);
    }
}

public void addPatient(Patient p) {
    patientList.add(p);
    savePatientToFile(p);
}

private void savePatientToFile(Patient p) {
    String patientDetails = p.getId() + "\n" + p.getName() + "\n" +
                            p.getMobileNo() + "\n" + p.getGender() + "\n" +
                            p.getAge() + "\n" + p.getAddress() + "\n" +
                            p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
                            p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
    try (FileWriter fw = new FileWriter(PATIENT_FILE_PATH, true)) {
        fw.write(patientDetails);
    } catch (IOException ex) {
        throw new RuntimeException("Error saving patient", ex);
    }
}
```

```
public int searchPatient(String id) {
    for (int i = 0; i < patientList.size(); i++) {
        if (patientList.get(i).getId().equals(id)) {
            return i;
        }
    }
    return -1;
}

public Patient getPatient(int index) {
    if (index < 0 || index >= patientList.size()) {
        return null;
    }
    return patientList.get(index);
}

public void deletePatient(Patient p) {
    patientList.remove(p);
    removePatientFromFile(p);
}

private void removePatientFromFile(Patient p) {
```

```
    String patientToDelete = buildPatientFileString(p);
    try {
        List<String> lines = readAllLines(PATIENT_FILE_PATH);
        StringBuilder newContent = new StringBuilder();
        StringBuilder currentBlock = new StringBuilder();
        for (String line : lines) {
            currentBlock.append(line).append("\n");
            if (currentBlock.toString().endsWith("\n\n")) { // Block end
                if (!currentBlock.toString().equals(patientToDelete)) {
                    newContent.append(currentBlock);
                }
                currentBlock = new StringBuilder();
            }
        }
        writeAllText(PATIENT_FILE_PATH, newContent.toString());
    } catch (IOException ex) {
        throw new RuntimeException("Error deleting patient from file", ex);
    }
}

private String buildPatientFileString(Patient p) {
    return p.getId() + "\n" + p.getName() + "\n" +
            p.getMobileNo() + "\n" + p.getGender() + "\n" +
```

```
        p.getAge() + "\n" + p.getAddress() + "\n" +
        p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
        p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
}

public void updatePatient(String patientToUpdate, Patient updatedPatient) {
    // Assume update in memory first if needed; here focus on file
    updatePatientInFile(patientToUpdate, updatedPatient);
}

private void updatePatientInFile(String patientToUpdate, Patient updatedPatient) {
    String updatedDetails = buildPatientFileString(updatedPatient);
    try {
        List<String> lines = readAllLines(PATIENT_FILE_PATH);
        StringBuilder newContent = new StringBuilder();
        StringBuilder currentBlock = new StringBuilder();
        for (String line : lines) {
            currentBlock.append(line).append("\n");
            if (currentBlock.toString().endsWith("\n\n")) {
                if (currentBlock.toString().equals(patientToUpdate)) {
                    newContent.append(updatedDetails);
                } else {
                    newContent.append(currentBlock);
```

```
            }
            currentBlock = new StringBuilder();
        }
    }
    writeAllText(PATIENT_FILE_PATH, newContent.toString());
  } catch (IOException ex) {
    throw new RuntimeException("Error updating patient in file", ex);
  }
}

// Helper methods to reduce duplication in file ops
private List<String> readAllLines(String path) throws IOException {
  List<String> lines = new ArrayList<>();
  try (BufferedReader br = new BufferedReader(new FileReader(path))) {
    String line;
    while ((line = br.readLine()) != null) {
      lines.add(line);
    }
  }
  return lines;
}

private void writeAllText(String path, String content) throws IOException {
```

```
        try (FileWriter fw = new FileWriter(path)) {
            fw.write(content);
        }
    }
}
```

Why This Code Smells?

The code is full of **Duplicated Code**, **Long Method**, **Magic Number**, and **Dead Code**.

Improvements:

1. Duplicate logic in delete and update operations moved to helper methods (readAllLines, writeAllText).
2. Long methods split into smaller functions for better readability.
3. Magic numbers/strings replaced with constants.
4. Fixed arrays replaced with dynamic List for flexibility.
5. Unused debug code removed and error handling improved.

Person.java

Before (Original Code)

```
package Classes;

public abstract class Person {

    public String id;
```

```
private String name;

private String gender;

private String mobileNo;

private String address;


public Person(String id,String name,String mobileNo,String gender) {

    this.id = id;

    this.name = name;

    this.mobileNo = mobileNo;

    this.gender = gender;

}


public String getId() {

    return id;

}


public void setId(String id) {

    this.id = id;

}



public String getName() {

    return name;
```

```
}

public void setName(String name) {
    this.name = name;
}

        public String getMobileNo() {
    return mobileNo;
}

public void setMobileNo(String mobileNo) {
    this.mobileNo = mobileNo;
}

public String getGender() {
    return gender;
}

public void setGender(String gender) {
    this.gender = gender;
}
```

```
}
```

After (Modified Code)

```
package Classes;

public abstract class Person {

    private String id;

    private String name;

    private String gender;

    private String mobileNo;


    public Person(String id, String name, String mobileNo, String gender) {

        this.id = id;

        this.name = name;

        this.mobileNo = mobileNo;

        this.gender = gender;

    }


    public String getId() {

        return id;

    }
```

```
public void setId(String id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getMobileNo() {
    return mobileNo;
}

public void setMobileNo(String mobileNo) {
    this.mobileNo = mobileNo;
}

public String getGender() {
    return gender;
}
```

```
    public void setGender(String gender) {
        this.gender = gender;
    }
}
```

Why This Is Better ?

The refactoring removed Dead Code.

Improvements:

1. Unused address field (dead code) was removed to keep the code clean.
2. The public id field was changed to **private** and accessed through getter/setter methods.

Appointment.java

Before (Original Code)

```
package Classes;


public class Appointment {
    private String appID;
    private String patientName;
    private String diagnosis;
    private String doctorName;
    private String appDate;
    private String appTime;
```

```
    public Appointment(String appID, String patientName, String diagnosis, String doctorName,
String appDate, String appTime) {

        this.appID = appID;

        this.patientName = patientName;

        this.diagnosis = diagnosis;

        this.doctorName = doctorName;

        this.appDate = appDate;

        this.appTime = appTime;

    }


    public String getAppId() {

        return appID;

    }


    public String getPatientName() {

        return patientName;

    }


    public String getDiagnosis() {

        return diagnosis;

    }


    public String getDoctorName() {
```

```
        return doctorName;
    }

    public String getAppDate() {
        return appDate;
    }

    public String getAppTime() {
        return appTime;
    }
}
```

After (Modified Code)

```
package Classes;

public class Appointment {

    private final String appID;
    private final String patientName;
    private final String diagnosis;
    private final String doctorName;
    private final String appDate;
```

```
private final String appTime;

private Appointment(Builder builder) {
    this.appID       = builder.appID;
    this.patientName = builder.patientName;
    this.diagnosis   = builder.diagnosis;
    this.doctorName  = builder.doctorName;
    this.appDate     = builder.appDate;
    this.appTime     = builder.appTime;
}

// getters remain the same
public String getAppId()        { return appID; }
public String getPatientName()  { return patientName; }
public String getDiagnosis()    { return diagnosis; }
public String getDoctorName()   { return doctorName; }
public String getAppDate()      { return appDate; }
public String getAppTime()      { return appTime; }

// Static nested Builder class
public static class Builder {
    // required fields
    private final String appID;
```

```
private final String patientName;

// optional / nullable fields
private String diagnosis   = "";
private String doctorName  = "";
private String appDate     = "";
private String appTime     = "";

public Builder(String appID, String patientName) {
    this.appID = requireNonNull(appID, "appID is required");
    this.patientName = requireNonNull(patientName, "patientName is required");
}

public Builder diagnosis(String diagnosis) {
    this.diagnosis = diagnosis != null ? diagnosis : "";
    return this;
}

public Builder doctorName(String doctorName) {
    this.doctorName = doctorName != null ? doctorName : "";
    return this;
}
```

```
        public Builder appDate(String appDate) {
            this.appDate = appDate != null ? appDate : "";
            return this;
        }

        public Builder appTime(String appTime) {
            this.appTime = appTime != null ? appTime : "";
            return this;
        }

        public Appointment build() {
            return new Appointment(this);
        }
    }

    // Optional: helper method (requires static import of Objects.requireNonNull)
    private static <T> T requireNonNull(T obj, String message) {
        if (obj == null) throw new NullPointerException(message);
        return obj;
    }
}
```

Why Is This Better?

The refactoring addressed the Long Parameter List.

Improvements:

1. The **long parameter list** was identified as a design issue for future refactoring.

AppointmentList.java

Before (Original Code)

```
package Classes;


import Interfaces.*;
import java.io.*;
import java.util.Scanner;


public class AppointmentList {
    public Appointment[] appointmentList = new Appointment[100];
    public static int appCount = 0;


    public AppointmentList() {
        try {
            File file = new File("Files/AppointmentList.txt");
            Scanner sc = new Scanner(file);
            while (sc.hasNext()) {
```

```
            String line1 = sc.nextLine();  // appID
            String line2 = sc.nextLine();  // patientName
            String line3 = sc.nextLine();  // diagnosis
            String line4 = sc.nextLine();  // doctorName
            String line5 = sc.nextLine();  // appDate
            String line6 = sc.nextLine();  // appTime
            if (sc.hasNextLine()) sc.nextLine();  // Skip empty line

            Appointment app = new Appointment(line1, line2, line3, line4, line5, line6);
            appointmentList[appCount] = app;
            appCount++;
        }
        sc.close();
    } catch (Exception ex) {
        System.out.println("File not found.");
        return;
    }
}

public void addAppointment(Appointment app) {
    try {
        appointmentList[appCount] = app;
        appCount++;
```

```
    } catch (Exception ex) {
        System.out.println("Array full");
    }

    String appointmentDetails = app.getAppId() + "\n" +
                                app.getPatientName() + "\n" +
                                app.getDiagnosis() + "\n" +
                                app.getDoctorName() + "\n" +
                                app.getAppDate() + "\n" +
                                app.getAppTime() + "\n" + "\n";

    try {
        FileWriter fw = new FileWriter("Files/AppointmentList.txt", true);
        fw.write(appointmentDetails);
        fw.close();
    } catch (Exception ex) {
        System.out.println(ex);
    }
}

public int searchAppointment(String appId) {
    int index = -1;
    for (int i = 0; i < appCount; i++) {
```

```
        if (appointmentList[i] != null && appointmentList[i].getAppId().equals(appId)) {
            index = i;
            break;
        }
    }
    return index;
}

public Appointment getAppointment(int index) {
    return appointmentList[index];
}

public void deleteAppointment(Appointment app) {
    for (int i = 0; i < appCount; i++) {
        if (appointmentList[i] == app) {
            for (int j = i; j < appCount - 1; j++) {
                appointmentList[j] = appointmentList[j + 1];
            }
            appCount--;
            appointmentList[appCount] = null;
            break;
        }
    }
```

```
String appointmentToDelete = app.getAppId() + "\n" +
                        app.getPatientName() + "\n" +
                        app.getDiagnosis() + "\n" +
                        app.getDoctorName() + "\n" +
                        app.getAppDate() + "\n" +
                        app.getAppTime() + "\n";

try {
    String filepath = "Files/AppointmentList.txt";
    File originalFile = new File(filepath);

    String newDetails = "";
    Scanner sc = new Scanner(originalFile);
    while (sc.hasNext()) {
        String readUser = sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine();
        if (sc.hasNextLine()) readUser += "\n";
```

```
            if (readUser.equals(appointmentToDelete)) {
                continue;
            }
            newDetails += readUser + "\n";
        }
        sc.close();
        FileWriter fw = new FileWriter(filepath);
        fw.write(newDetails);
        fw.close();
    } catch (Exception ex) {
        System.out.println(ex);
    }
}


public void updateAppointment(String appointmentToUpdate, Appointment
updatedAppointment) {
    String updatedDetails = updatedAppointment.getAppId() + "\n" +
                    updatedAppointment.getPatientName() + "\n" +
                    updatedAppointment.getDiagnosis() + "\n" +
                    updatedAppointment.getDoctorName() + "\n" +
                    updatedAppointment.getAppDate() + "\n" +
                    updatedAppointment.getAppTime() + "\n";

    // Update in file
```

```
try {
    String filepath = "Files/AppointmentList.txt";
    File originalFile = new File(filepath);

    String newDetails = "";
    Scanner sc = new Scanner(originalFile);
    while (sc.hasNext()) {
        String readUser = sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine();
        if (sc.hasNextLine()) readUser += "\n";

        if (readUser.equals(appointmentToUpdate)) {
            newDetails += updatedDetails + "\n";
        } else {
            newDetails += readUser + "\n";
        }
    }
    sc.close();
    FileWriter fw = new FileWriter(filepath);
```

```
            fw.write(newDetails);
            fw.close();
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }
}
```

After (Modified Code)

```
package Classes;

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class AppointmentList {
    private static final String APPOINTMENT_FILE_PATH = "Files/AppointmentList.txt";
    private final List<Appointment> appointmentList = new ArrayList<>();

    public AppointmentList() {
        loadAppointmentsFromFile();
    }
```

```
    private void loadAppointmentsFromFile() {
        File file = new File(APPOINTMENT_FILE_PATH);
        if (!file.exists()) {
            return;
        }
        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String appID = sc.nextLine();
                String patientName = sc.nextLine();
                String diagnosis = sc.nextLine();
                String doctorName = sc.nextLine();
                String appDate = sc.nextLine();
                String appTime = sc.nextLine();
                if (sc.hasNextLine()) sc.nextLine();


                Appointment app = new Appointment(appID, patientName, diagnosis, doctorName,
appDate, appTime);
                appointmentList.add(app);
            }
        } catch (FileNotFoundException ex) {
            // Empty list
        } catch (IOException ex) {
            throw new RuntimeException("Error loading appointments", ex);
```

```
    }
}

public void addAppointment(Appointment app) {
    appointmentList.add(app);
    saveAppointmentToFile(app);
}

private void saveAppointmentToFile(Appointment app) {
    String appointmentDetails = app.getAppId() + "\n" +
                                app.getPatientName() + "\n" +
                                app.getDiagnosis() + "\n" +
                                app.getDoctorName() + "\n" +
                                app.getAppDate() + "\n" +
                                app.getAppTime() + "\n\n";
    try (FileWriter fw = new FileWriter(APPOINTMENT_FILE_PATH, true)) {
        fw.write(appointmentDetails);
    } catch (IOException ex) {
        throw new RuntimeException("Error saving appointment", ex);
    }
}

public int searchAppointment(String appId) {
```

```
    for (int i = 0; i < appointmentList.size(); i++) {
        if (appointmentList.get(i).getAppId().equals(appId)) {
            return i;
        }
    }
    return -1;
}

public Appointment getAppointment(int index) {
    if (index < 0 || index >= appointmentList.size()) {
        return null;
    }
    return appointmentList.get(index);
}

public void deleteAppointment(Appointment app) {
    appointmentList.remove(app);
    removeAppointmentFromFile(app);
}

private void removeAppointmentFromFile(Appointment app) {
    String appointmentToDelete = buildAppointmentFileString(app);
    try {
```

```
        List<String> lines = readAllLines(APPOINTMENT_FILE_PATH);
        StringBuilder newContent = new StringBuilder();
        StringBuilder currentBlock = new StringBuilder();
        for (String line : lines) {
            currentBlock.append(line).append("\n");
            if (currentBlock.toString().endsWith("\n\n")) {
                if (!currentBlock.toString().equals(appointmentToDelete)) {
                    newContent.append(currentBlock);
                }
                currentBlock = new StringBuilder();
            }
        }
        writeAllText(APPOINTMENT_FILE_PATH, newContent.toString());
    } catch (IOException ex) {
        throw new RuntimeException("Error deleting appointment from file", ex);
    }
}

private String buildAppointmentFileString(Appointment app) {
    return app.getAppId() + "\n" +
            app.getPatientName() + "\n" +
            app.getDiagnosis() + "\n" +
            app.getDoctorName() + "\n" +
```

```
            app.getAppDate() + "\n" +
            app.getAppTime() + "\n\n";
    }

    public void updateAppointment(String appointmentToUpdate, Appointment
updatedAppointment) {
        updateAppointmentInFile(appointmentToUpdate, updatedAppointment);
    }

    private void updateAppointmentInFile(String appointmentToUpdate, Appointment
updatedAppointment) {
        String updatedDetails = buildAppointmentFileString(updatedAppointment);
        try {
            List<String> lines = readAllLines(APPOINTMENT_FILE_PATH);
            StringBuilder newContent = new StringBuilder();
            StringBuilder currentBlock = new StringBuilder();
            for (String line : lines) {
                currentBlock.append(line).append("\n");
                if (currentBlock.toString().endsWith("\n\n")) {
                    if (currentBlock.toString().equals(appointmentToUpdate)) {
                        newContent.append(updatedDetails);
                    } else {
                        newContent.append(currentBlock);
                    }
```

```
                currentBlock = new StringBuilder();
            }
        }
        writeAllText(APPOINTMENT_FILE_PATH, newContent.toString());
    } catch (IOException ex) {
        throw new RuntimeException("Error updating appointment in file", ex);
    }
}

// Helpers
private List<String> readAllLines(String path) throws IOException {
    List<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        String line;
        while ((line = br.readLine()) != null) {
            lines.add(line);
        }
    }
    return lines;
}

private void writeAllText(String path, String content) throws IOException {
    try (FileWriter fw = new FileWriter(path)) {
```

```
            fw.write(content);
        }
    }
}
```

Why This Is Better?

The refactoring removed common issues like Duplicated Code, Long Method, and Magic Number.

Improvements:

1. Duplicate code moved to helper methods.
2. Long methods broken into smaller ones.
3. Magic numbers/strings replaced with constants.
4. Arrays changed to List for better data handling.
   Error handling improved for reliability.

Start.java

Before (Original Code)

```
import java.lang.*;

import Classes.*;

import Frames.*;

import Interfaces.*;
```

```
public class Start{
        public static void main(String []args){
                Login lg =  new Login();
                lg.setVisible(true);
        /*      Menu me =  new Menu();
                me.setVisible(true);*/
        }
}
```

After (Modified Code)

```
import Frames.Login;

public class Start {
    public static void main(String[] args) {
        createAndShowInitialFrame();
    }

    private static void createAndShowInitialFrame() {
        Login lg = new Login();
        lg.setVisible(true);
    }
```

}

Why: The refactoring process removed several minor code smells to keep the class clean and maintainable. Unnecessary imports were eliminated to reduce clutter and improve readability. Commented-out or dead code was also removed since it no longer served any purpose and could confuse future developers. Overall, only minimal changes were required because the class was already simple and well-structured, ensuring that the codebase remains clear, concise, and easy to maintain.

EAST WEST UNIVERSITY

Course Code: CSE423

Course Title: Software Architecture

Section: 01

Semester: Spring26

Group No: 01

Report

Project Name: Hospital Management System

Submitted To:

Ahmed Adnan

Lecturer

Department of Computer Science and Engineering

Submitted By:

Name:	ID:
Sababa Hoque Saba	2022-3-60-060
Afia Ayub Samirah	2022-3-60-012

Kha. Mo. Syeed Asif	2022-3-60-030
Md. Omor Faruq Shikdere	2022-1-60-335

Date of Submission: 10-03-2026

Table of Content

Code Smell..... 4

Code Smell 1 - God Class (Large Class)..... 4

Why this is a Code Smell..... 14

Problems Caused..... 14

Correct Refactored Code (After SOLID)..... 15

Code Smell 2 - Inappropriate naming..... 22

Add Doctor version Before:..... 22

Why This Is a Problem..... 23

After Adding UI Class:..... 24

Code Smell 3 - Long Method..... 25

Problem..... 35

Correct Solution..... 35

RegDoctorList.java..... 44

Before (Original Code)..... 44

After (Modified Code)..... 49

Why : The code contained several code smells that affected its quality and maintainability. These included duplicated code in file I/O operations, the use of magic numbers such as a fixed array size (100), and reliance on static variables like doctorCount to track elements. It also had poor error handling, where exceptions were not properly managed, and dead code, including unnecessary debug prints and commented-out methods. Additionally, the code showed primitive obsession by using arrays instead of more flexible data structures like List or ArrayList..... 53

PrescriptionList.java..... 54

Before (Original Code)..... 54

After (Modified Code)..... 57

Why: The refactored code removed several code smells to improve quality and maintainability. The duplicated file I/O code was streamlined to avoid repetition. Magic numbers and strings were replaced with constants, and fixed arrays were replaced with a dynamic List for better flexibility. The unnecessary static variable pCount was removed and replaced with more reliable methods like checking the list size. Error handling was improved by using more specific exceptions to properly handle format-related issues. Additionally, defensive bounds checks were added to prevent invalid index access and make the code more robust..... 61

Prescription.java..... 61
Before (Original Code)..... 61
After (Modified Code)..... 63
Why: The refactoring addressed some key code smells to improve code reliability and design. A mutable object was converted into an immutable object by making its fields final, which helps prevent unintended changes to its state and improves thread safety. Additionally, the presence of a long parameter list was identified as a design smell; although it was not refactored in this stage, the Builder pattern was suggested as a future improvement to make object creation cleaner and more maintainable.....66
Patient.java..... 66
Before (Original Code)..... 66
After (Modified Code)..... 69
Why This Is a Code Smell?.....72
Doctor.java.....73
Before (Original Code)..... 73
After (Modified Code)..... 75
Why This Code Smells?.....77
PList.java..... 77
Before (Original Code)..... 77
After (Modified Code)..... 87
Why This Code Smells?.....94
Person.java.....94
Before (Original Code)..... 94
After (Modified Code)..... 97
Why This Is Better ?..... 99
Appointment.java..... 99
Before (Original Code)..... 99
After (Modified Code)..... 101
Why Is This Better?..... 104
AppointmentList.java..... 105
Before (Original Code)..... 105
After (Modified Code)..... 112
Why This Is Better?.....119
Start.java..... 119
Before (Original Code).....119
After (Modified Code)..... 120
Why: The refactoring process removed several minor code smells to keep the class clean and maintainable. Unnecessary imports were eliminated to reduce clutter and improve readability. Commented-out or dead code was also removed since it no longer served any purpose and could confuse future developers. Overall, only minimal changes were required because the class was already simple and well-structured, ensuring that the codebase remains clear, concise, and easy to maintain.....121

Code Smell

Commit Message: [Code Smell removed]

Code Smell 1 - God Class (Large Class)

Before: AddDoctor class handled UI layout, event handling, and doctor data storage.

```
import java.lang.*;
```

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;



public class AddDoctor extends JFrame implements
MouseListener,ActionListener{



    JLabel
userLabel,passLabel,genderLabel,programLabel,frame,imgLabel;

    JTextField userTF;

    JPasswordField passTF;

    JButton add, back;

    ButtonGroup bg1;
```

```
    JRadioButton r1,r2,r3;

    JComboBox dept;

    Color color1,color2;

    Font font1, font2, font3, font4;

    ImageIcon img;

    JPanel panel;


    public AddDoctor(){


        super("Doctor Information");

        this.setSize(900,600);

        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        this.setLocationRelativeTo(null);




        color1= new Color(143,207,225);

        color2= new Color(39,83,23);


        font1 = new Font("Biome",Font.BOLD, 30);

        font2 = new Font("Cascadia Code SemiBold",Font.PLAIN,
20);
```

```
        font3 = new Font("Cascadia Mono SemiBold",Font.PLAIN,
15);

        font4 = new Font("Biome",Font.BOLD, 18);




        panel = new JPanel();

        panel.setLayout(null);


        userLabel = new JLabel("HealthMate ");

        userLabel.setBounds(300,10,400,100);

        userLabel.setFont(font1);

        panel.add(userLabel);




        userLabel = new JLabel("Add Doctor Form ");

        userLabel.setBounds(130,80,400,50);

        userLabel.setFont(font4);

        panel.add(userLabel);


        userLabel = new JLabel("Doctor Name: ");

        userLabel.setBounds(75,120,100,100);

        panel.add(userLabel);
```

```
userTF = new JTextField();
userTF.setBounds(170,160,150,20);
panel.add(userTF);

userLabel = new JLabel("Mobile Number: ");
userLabel.setBounds(75,150,100,100);
panel.add(userLabel);

userTF = new JTextField();
userTF.setBounds(170,190,150,20);
panel.add(userTF);

genderLabel = new JLabel("Gender:");
genderLabel.setBounds(75,220,115,20);
panel.add(genderLabel);

r1 = new JRadioButton("Male");
r1.setBounds(150,220,80,20);
r1.setBackground(color1);
panel.add(r1);
```

```
r2 = new JRadioButton("Female");
r2.setBounds(230,220,80,20);
r2.setBackground(color1);
panel.add(r2);

r3 = new JRadioButton("Other");
r3.setBounds(320,220,80,20);
r3.setBackground(color1);
panel.add(r3);

bg1 = new ButtonGroup();
bg1.add(r1);
bg1.add(r2);
bg1.add(r3);


programLabel = new JLabel("Department:");
programLabel.setBounds(75,210,100,100);
panel.add(programLabel);
```

```
        String items[] = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};

        dept = new JComboBox(items);

        dept.setBounds(170,250,150,20);

        panel.add(dept);


        userLabel = new JLabel("Joining Date: ");

        userLabel.setBounds(75,240,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,280,150,20);

        panel.add(userTF);


        userLabel = new JLabel("Salary: ");

        userLabel.setBounds(75,270,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,310,150,20);

        panel.add(userTF);
```

```
        userLabel = new JLabel("Room NO: ");
        userLabel.setBounds(75,300,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,340,150,20);
        panel.add(userTF);

        userLabel = new JLabel("BM&DC REG: ");
        userLabel.setBounds(75,330,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,370,150,20);
        panel.add(userTF);

        add = new JButton("ADD");
        add.setBounds(500,470,100,40);
        add.setBackground(Color.GREEN);
        add.setForeground(Color.WHITE);
```

```
        add.setOpaque(true);
        add.addMouseListener(this);
        add.addActionListener(this);
        panel.add(add);

        back = new JButton("BACK");
        back.setBounds(620,470,100,40);
        back.setBackground(Color.RED);
        back.setForeground(Color.WHITE);
        back.setOpaque(true);
        back.addMouseListener(this);
        back.addActionListener(this);
        panel.add(back);

        img = new ImageIcon("images/emp-info.png");

        imgLabel = new JLabel(img);
        imgLabel.setBounds(485,100,300,300);

        panel.add(imgLabel);
```

```
        panel.setBackground(color1);
        this.add(panel);
    }


    public void mouseClicked(MouseEvent me){}
    public void mousePressed(MouseEvent me){}
    public void mouseReleased(MouseEvent me){}
    public void mouseEntered(MouseEvent me){
        if(me.getSource() == add){
            add.setBackground(Color.RED);
    add.setForeground(Color.WHITE);
        }else if(me.getSource() == back){
            back.setBackground(Color.GREEN);
            back.setForeground(Color.BLACK);
        }
    }
    public void mouseExited(MouseEvent me){
        if(me.getSource() == add){
            add.setBackground(Color.GREEN);
            add.setForeground(Color.BLACK);
        }else if(me.getSource() == back){
```

```
            back.setBackground(Color.RED);

            back.setForeground(Color.WHITE);

        }

    }



    public void actionPerformed(ActionEvent ae){

        String command = ae.getActionCommand();

        if(add.getText().equals(command));{

            DoctorList dl = new DoctorList();

            dl.setVisible(true);

            this.setVisible(false);



        }

        //public void actionPerformed(ActionEvent ae){

        //String command = ae.getActionCommand();

            if(ae.getSource() == back) {


                    JOptionPane.showMessageDialog(this, "Added
doctor successfully");

                     DrMenu dm = new DrMenu();

                    dm.setVisible(true);

                    this.setVisible(false);
```

```
                    }



    }



}
```

Why this is a Code Smell

This class performs too many tasks, which violates:Single Responsibility Principle (SRP**)**

Problems Caused

- Hard to debug
- Hard to reuse
- Hard to extend

Correct Refactored Code (After SOLID)

Doctor.java
After:

```
public class Doctor {
    private String name;
    private String mobile;
    private String gender;
    private String department;
    private String joiningDate;
    private double salary;
```

```
    private String roomNo;
    private String bmdcReg;

    public Doctor(String name, String mobile, String gender,
String department,
                  String joiningDate, double salary, String
roomNo, String bmdcReg) {
        this.name = name;
        this.mobile = mobile;
        this.gender = gender;
        this.department = department;
        this.joiningDate = joiningDate;
        this.salary = salary;
        this.roomNo = roomNo;
        this.bmdcReg = bmdcReg;
    }

    // Getters
    public String getName() { return name; }
    public String getMobile() { return mobile; }
    public String getGender() { return gender; }
    public String getDepartment() { return department; }
    public String getJoiningDate() { return joiningDate; }
    public double getSalary() { return salary; }
    public String getRoomNo() { return roomNo; }
    public String getBmdcReg() { return bmdcReg; }
}
```

Doctor Service (Business Logic)

```
import java.util.ArrayList;
import java.util.List;
```

```java
public class DoctorService implements IDoctorService {
    private List<Doctor> doctors = new ArrayList<>();

    @Override
    public void addDoctor(Doctor doctor) {
        doctors.add(doctor);

    }

    public List<Doctor> getDoctors() {
        return doctors;
    }
}
```

Interface for Dependency Inversion

```java
public interface IDoctorService {
    void addDoctor(Doctor doctor);
}
```

UI Class (Only UI Responsibility)

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;

    private JTextField nameTF, mobileTF, joiningTF, salaryTF,
roomTF, bmdcTF;
    private JRadioButton maleRB, femaleRB, otherRB;
```

```java
    private JComboBox<String> deptCB;
    private JButton addButton, backButton;
    private ButtonGroup genderGroup;

    public AddDoctorUI(IDoctorService doctorService) {
        super("Add Doctor Form");
        this.doctorService = doctorService;

        setSize(900, 600);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        initUI();
    }

    private void initUI() {
        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBackground(new Color(143,207,225));

        Font labelFont = new Font("Cascadia Code SemiBold",
Font.PLAIN, 18);

        // Labels & TextFields
        JLabel titleLabel = new JLabel("Add Doctor Form");
        titleLabel.setBounds(130, 20, 300, 40);
        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
        panel.add(titleLabel);

        JLabel nameLabel = new JLabel("Doctor Name:");
        nameLabel.setBounds(75, 80, 120, 25);
        nameLabel.setFont(labelFont);
        panel.add(nameLabel);

        nameTF = new JTextField();
```

```
        nameTF.setBounds(200, 80, 150, 25);
        panel.add(nameTF);

        JLabel mobileLabel = new JLabel("Mobile Number:");
        mobileLabel.setBounds(75, 120, 120, 25);
        mobileLabel.setFont(labelFont);
        panel.add(mobileLabel);

        mobileTF = new JTextField();
        mobileTF.setBounds(200, 120, 150, 25);
        panel.add(mobileTF);

        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75, 160, 100, 25);
        genderLabel.setFont(labelFont);
        panel.add(genderLabel);

        maleRB = new JRadioButton("Male");
        maleRB.setBounds(150, 160, 80, 25);
        maleRB.setBackground(panel.getBackground());
        femaleRB = new JRadioButton("Female");
        femaleRB.setBounds(230, 160, 80, 25);
        femaleRB.setBackground(panel.getBackground());
        otherRB = new JRadioButton("Other");
        otherRB.setBounds(310, 160, 80, 25);
        otherRB.setBackground(panel.getBackground());

        genderGroup = new ButtonGroup();
        genderGroup.add(maleRB);
        genderGroup.add(femaleRB);
        genderGroup.add(otherRB);

        panel.add(maleRB);
        panel.add(femaleRB);
        panel.add(otherRB);
```

```
        JLabel deptLabel = new JLabel("Department:");
        deptLabel.setBounds(75, 200, 120, 25);
        deptLabel.setFont(labelFont);
        panel.add(deptLabel);

        String[] departments = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};
        deptCB = new JComboBox<>(departments);
        deptCB.setBounds(200, 200, 150, 25);
        panel.add(deptCB);

        JLabel joiningLabel = new JLabel("Joining Date:");
        joiningLabel.setBounds(75, 240, 120, 25);
        joiningLabel.setFont(labelFont);
        panel.add(joiningLabel);

        joiningTF = new JTextField();
        joiningTF.setBounds(200, 240, 150, 25);
        panel.add(joiningTF);

        JLabel salaryLabel = new JLabel("Salary:");
        salaryLabel.setBounds(75, 280, 120, 25);
        salaryLabel.setFont(labelFont);
        panel.add(salaryLabel);

        salaryTF = new JTextField();
        salaryTF.setBounds(200, 280, 150, 25);
        panel.add(salaryTF);

        JLabel roomLabel = new JLabel("Room No:");
        roomLabel.setBounds(75, 320, 120, 25);
        roomLabel.setFont(labelFont);
        panel.add(roomLabel);
```

```
        roomTF = new JTextField();
        roomTF.setBounds(200, 320, 150, 25);
        panel.add(roomTF);

        JLabel bmdcLabel = new JLabel("BM&DC REG:");
        bmdcLabel.setBounds(75, 360, 120, 25);
        bmdcLabel.setFont(labelFont);
        panel.add(bmdcLabel);

        bmdcTF = new JTextField();
        bmdcTF.setBounds(200, 360, 150, 25);
        panel.add(bmdcTF);

        // Buttons
        addButton = new JButton("ADD");
        addButton.setBounds(500, 470, 100, 40);
        addButton.setBackground(Color.GREEN);
        addButton.setForeground(Color.WHITE);
        addButton.addActionListener(e -> onAdd());
        panel.add(addButton);

        backButton = new JButton("BACK");
        backButton.setBounds(620, 470, 100, 40);
        backButton.setBackground(Color.RED);
        backButton.setForeground(Color.WHITE);
        backButton.addActionListener(e -> onBack());
        panel.add(backButton);

        this.add(panel);
    }

    private void onAdd() {
        String name = nameTF.getText();
        String mobile = mobileTF.getText();
        String gender = maleRB.isSelected() ? "Male" :
```

```
                        femaleRB.isSelected() ? "Female" :
"Other";
        String department = (String) deptCB.getSelectedItem();
        String joiningDate = joiningTF.getText();
        double salary = Double.parseDouble(salaryTF.getText());
        String room = roomTF.getText();
        String bmdc = bmdcTF.getText();

        Doctor doctor = new Doctor(name, mobile, gender,
department, joiningDate, salary, room, bmdc);
        doctorService.addDoctor(doctor);

        JOptionPane.showMessageDialog(this, "Doctor added
successfully!");
        clearFields();
    }

    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu(); // Navigation to main menu
        dm.setVisible(true);
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

Code Smell 2 - Inappropriate naming

Add Doctor version Before:

```
  userLabel = new JLabel("HealthMate ");

       userLabel.setBounds(300,10,400,100);

       userLabel.setFont(font1);

       panel.add(userLabel);




       userLabel = new JLabel("Add Doctor Form ");

       userLabel.setBounds(130,80,400,50);

       userLabel.setFont(font4);

       panel.add(userLabel);


       userLabel = new JLabel("Doctor Name: ");

       userLabel.setBounds(75,120,100,100);

       panel.add(userLabel);


       userTF = new JTextField();

       userTF.setBounds(170,160,150,20);

       panel.add(userTF);
```

```
        userLabel = new JLabel("Mobile Number: ");

        userLabel.setBounds(75,150,100,100);

        panel.add(userLabel);



        userTF = new JTextField();

        userTF.setBounds(170,190,150,20);

        panel.add(userTF);
```

Why This Is a Problem

- Overwrites previous fields
- Makes debugging difficult
- Confusing variable usage

After Adding UI Class:

```
// Labels & TextFields
        JLabel titleLabel = new JLabel("Add Doctor Form");
        titleLabel.setBounds(130, 20, 300, 40);
        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
        panel.add(titleLabel);

        JLabel nameLabel = new JLabel("Doctor Name:");
        nameLabel.setBounds(75, 80, 120, 25);
        nameLabel.setFont(labelFont);
        panel.add(nameLabel);
```

```
        nameTF = new JTextField();
        nameTF.setBounds(200, 80, 150, 25);
        panel.add(nameTF);

        JLabel mobileLabel = new JLabel("Mobile Number:");
        mobileLabel.setBounds(75, 120, 120, 25);
        mobileLabel.setFont(labelFont);
        panel.add(mobileLabel);

        mobileTF = new JTextField();
        mobileTF.setBounds(200, 120, 150, 25);
        panel.add(mobileTF);

        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(75, 160, 100, 25);
        genderLabel.setFont(labelFont);
        panel.add(genderLabel);
```

Each field has a clear purpose.

Code Smell 3 - Long Method

The original constructor contained hundreds of lines of UI setup.

Before:

```
import java.lang.*;
```

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;
```

```
public class AddDoctor extends JFrame implements
MouseListener,ActionListener{


    JLabel
userLabel,passLabel,genderLabel,programLabel,frame,imgLabel;

    JTextField userTF;

    JPasswordField passTF;

    JButton add, back;

    ButtonGroup bg1;

    JRadioButton r1,r2,r3;

    JComboBox dept;

    Color color1,color2;

    Font font1, font2, font3, font4;

    ImageIcon img;

    JPanel panel;


    public AddDoctor(){


        super("Doctor Information");

        this.setSize(900,600);

        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        this.setLocationRelativeTo(null);
```

```
        color1= new Color(143,207,225);

        color2= new Color(39,83,23);


        font1 = new Font("Biome",Font.BOLD, 30);

        font2 = new Font("Cascadia Code SemiBold",Font.PLAIN,
20);

        font3 = new Font("Cascadia Mono SemiBold",Font.PLAIN,
15);

        font4 = new Font("Biome",Font.BOLD, 18);



        panel = new JPanel();

        panel.setLayout(null);


        userLabel = new JLabel("HealthMate ");

        userLabel.setBounds(300,10,400,100);

        userLabel.setFont(font1);

        panel.add(userLabel);
```

```
        userLabel = new JLabel("Add Doctor Form ");
        userLabel.setBounds(130,80,400,50);
        userLabel.setFont(font4);
        panel.add(userLabel);

        userLabel = new JLabel("Doctor Name: ");
        userLabel.setBounds(75,120,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,160,150,20);
        panel.add(userTF);

        userLabel = new JLabel("Mobile Number: ");
        userLabel.setBounds(75,150,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,190,150,20);
        panel.add(userTF);
```

```
genderLabel = new JLabel("Gender:");
genderLabel.setBounds(75,220,115,20);
panel.add(genderLabel);

r1 = new JRadioButton("Male");
r1.setBounds(150,220,80,20);
r1.setBackground(color1);
panel.add(r1);

r2 = new JRadioButton("Female");
r2.setBounds(230,220,80,20);
r2.setBackground(color1);
panel.add(r2);

r3 = new JRadioButton("Other");
r3.setBounds(320,220,80,20);
r3.setBackground(color1);
panel.add(r3);

bg1 = new ButtonGroup();
bg1.add(r1);
```

```
        bg1.add(r2);

        bg1.add(r3);




        programLabel = new JLabel("Department:");

        programLabel.setBounds(75,210,100,100);

        panel.add(programLabel);


        String items[] = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};

        dept = new JComboBox(items);

        dept.setBounds(170,250,150,20);

        panel.add(dept);


        userLabel = new JLabel("Joining Date: ");

        userLabel.setBounds(75,240,100,100);

        panel.add(userLabel);


        userTF = new JTextField();

        userTF.setBounds(170,280,150,20);

        panel.add(userTF);
```

```
        userLabel = new JLabel("Salary: ");
        userLabel.setBounds(75,270,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,310,150,20);
        panel.add(userTF);

        userLabel = new JLabel("Room NO: ");
        userLabel.setBounds(75,300,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
        userTF.setBounds(170,340,150,20);
        panel.add(userTF);

        userLabel = new JLabel("BM&DC REG: ");
        userLabel.setBounds(75,330,100,100);
        panel.add(userLabel);

        userTF = new JTextField();
```

```
        userTF.setBounds(170,370,150,20);
        panel.add(userTF);



        add = new JButton("ADD");
        add.setBounds(500,470,100,40);
        add.setBackground(Color.GREEN);
        add.setForeground(Color.WHITE);
        add.setOpaque(true);
        add.addMouseListener(this);
        add.addActionListener(this);
        panel.add(add);

        back = new JButton("BACK");
        back.setBounds(620,470,100,40);
        back.setBackground(Color.RED);
        back.setForeground(Color.WHITE);
        back.setOpaque(true);
        back.addMouseListener(this);
        back.addActionListener(this);
        panel.add(back);
```

```
        img = new ImageIcon("images/emp-info.png");


        imgLabel = new JLabel(img);
        imgLabel.setBounds(485,100,300,300);


        panel.add(imgLabel);


        panel.setBackground(color1);
        this.add(panel);
    }


    public void mouseClicked(MouseEvent me){}
    public void mousePressed(MouseEvent me){}
    public void mouseReleased(MouseEvent me){}
    public void mouseEntered(MouseEvent me){
        if(me.getSource() == add){
            add.setBackground(Color.RED);
    add.setForeground(Color.WHITE);
        }else if(me.getSource() == back){
            back.setBackground(Color.GREEN);
```

```
            back.setForeground(Color.BLACK);
        }
    }
    public void mouseExited(MouseEvent me){
        if(me.getSource() == add){
            add.setBackground(Color.GREEN);
            add.setForeground(Color.BLACK);
        }else if(me.getSource() == back){
            back.setBackground(Color.RED);
            back.setForeground(Color.WHITE);
        }
    }

    public void actionPerformed(ActionEvent ae){
        String command = ae.getActionCommand();
        if(add.getText().equals(command));{
            DoctorList dl = new DoctorList();
            dl.setVisible(true);
            this.setVisible(false);

        }
```

```
        //public void actionPerformed(ActionEvent ae){

        //String command = ae.getActionCommand();

            if(ae.getSource() == back) {



                    JOptionPane.showMessageDialog(this, "Added
doctor successfully");

                     DrMenu dm = new DrMenu();

                    dm.setVisible(true);

                    this.setVisible(false);

                }



    }




}
```

Problem

Hard to read and maintain.

Correct Solution

Break it into smaller methods.

After:

```java
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;


public class AddDoctorUI extends JFrame {


    private IDoctorService doctorService;


    private JTextField nameTF, mobileTF, joiningTF, salaryTF,
roomTF, bmdcTF;

    private JRadioButton maleRB, femaleRB, otherRB;

    private JComboBox<String> deptCB;

    private JButton addButton, backButton;

    private ButtonGroup genderGroup;


    public AddDoctorUI(IDoctorService doctorService) {

        super("Add Doctor Form");

        this.doctorService = doctorService;


        setSize(900, 600);

        setLocationRelativeTo(null);

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        initUI();
    }

    private void initUI() {
        JPanel panel = new JPanel();
        panel.setLayout(null);
        panel.setBackground(new Color(143,207,225));

        Font labelFont = new Font("Cascadia Code SemiBold",
Font.PLAIN, 18);

        // Labels & TextFields
        JLabel titleLabel = new JLabel("Add Doctor Form");
        titleLabel.setBounds(130, 20, 300, 40);
        titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
        panel.add(titleLabel);

        JLabel nameLabel = new JLabel("Doctor Name:");
        nameLabel.setBounds(75, 80, 120, 25);
        nameLabel.setFont(labelFont);
        panel.add(nameLabel);
```

```
nameTF = new JTextField();
nameTF.setBounds(200, 80, 150, 25);
panel.add(nameTF);

JLabel mobileLabel = new JLabel("Mobile Number:");
mobileLabel.setBounds(75, 120, 120, 25);
mobileLabel.setFont(labelFont);
panel.add(mobileLabel);

mobileTF = new JTextField();
mobileTF.setBounds(200, 120, 150, 25);
panel.add(mobileTF);

JLabel genderLabel = new JLabel("Gender:");
genderLabel.setBounds(75, 160, 100, 25);
genderLabel.setFont(labelFont);
panel.add(genderLabel);

maleRB = new JRadioButton("Male");
maleRB.setBounds(150, 160, 80, 25);
```

```
        maleRB.setBackground(panel.getBackground());

        femaleRB = new JRadioButton("Female");

        femaleRB.setBounds(230, 160, 80, 25);

        femaleRB.setBackground(panel.getBackground());

        otherRB = new JRadioButton("Other");

        otherRB.setBounds(310, 160, 80, 25);

        otherRB.setBackground(panel.getBackground());


        genderGroup = new ButtonGroup();

        genderGroup.add(maleRB);

        genderGroup.add(femaleRB);

        genderGroup.add(otherRB);


        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);


        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);
```

```
        String[] departments = {"CARDIOLOGY", "NEUROSURGERY",
"ONCOLOGIST", "ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);

        deptCB.setBounds(200, 200, 150, 25);

        panel.add(deptCB);


        JLabel joiningLabel = new JLabel("Joining Date:");

        joiningLabel.setBounds(75, 240, 120, 25);

        joiningLabel.setFont(labelFont);

        panel.add(joiningLabel);


        joiningTF = new JTextField();

        joiningTF.setBounds(200, 240, 150, 25);

        panel.add(joiningTF);


        JLabel salaryLabel = new JLabel("Salary:");

        salaryLabel.setBounds(75, 280, 120, 25);

        salaryLabel.setFont(labelFont);

        panel.add(salaryLabel);


        salaryTF = new JTextField();
```

```
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);

bmdcTF = new JTextField();
bmdcTF.setBounds(200, 360, 150, 25);
panel.add(bmdcTF);
```

```
        // Buttons
        addButton = new JButton("ADD");
        addButton.setBounds(500, 470, 100, 40);
        addButton.setBackground(Color.GREEN);
        addButton.setForeground(Color.WHITE);
        addButton.addActionListener(e -> onAdd());
        panel.add(addButton);

        backButton = new JButton("BACK");
        backButton.setBounds(620, 470, 100, 40);
        backButton.setBackground(Color.RED);
        backButton.setForeground(Color.WHITE);
        backButton.addActionListener(e -> onBack());
        panel.add(backButton);

        this.add(panel);
    }

    private void onAdd() {
        String name = nameTF.getText();
        String mobile = mobileTF.getText();
```

```
        String gender = maleRB.isSelected() ? "Male" :
                        femaleRB.isSelected() ? "Female" :
"Other";
        String department = (String) deptCB.getSelectedItem();
        String joiningDate = joiningTF.getText();
        double salary = Double.parseDouble(salaryTF.getText());
        String room = roomTF.getText();
        String bmdc = bmdcTF.getText();


        Doctor doctor = new Doctor(name, mobile, gender,
department, joiningDate, salary, room, bmdc);
        doctorService.addDoctor(doctor);


        JOptionPane.showMessageDialog(this, "Doctor added
successfully!");
        clearFields();
    }


    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu(); // Navigation to main menu
        dm.setVisible(true);
    }
```

```
    private void clearFields() {

        nameTF.setText("");

        mobileTF.setText("");

        genderGroup.clearSelection();

        deptCB.setSelectedIndex(0);

        joiningTF.setText("");

        salaryTF.setText("");

        roomTF.setText("");

        bmdcTF.setText("");

    }

}
```

RegDoctorList.java

Before (Original Code)

```
package Classes;

import java.io.*;

import java.util.Scanner;


public class RegDoctorList{

	public Doctor doctorList[] = new Doctor[100];

	public static int doctorCount = 0;


	public RegDoctorList(){

		try{

			File file = new File("Files/DoctorList.txt");

			Scanner sc = new Scanner(file);

				while(sc.hasNext()){

					String line1 = sc.nextLine();  //id

					String line2 = sc.nextLine();  //name

					String line3 = sc.nextLine();  //mo n.
```

```
String line4 = sc.nextLine();  //gender
String line5 = sc.nextLine();  //department
String line6 = sc.nextLine();  //joinig date
String line7 = sc.nextLine();  //bmdc
String line8 = sc.nextLine();  //pass
String line9 = sc.nextLine();  //extra newline


System.out.println(line1);
System.out.println(line2);
System.out.println(line3);
System.out.println(line4);
System.out.println(line5);
System.out.println(line6);
System.out.println(line7);
System.out.println(line8);

Doctor d = new
Doctor(line1,line2,line3,line4,line5,line6,line7,line8);
doctorList[doctorCount] = d;
doctorCount++;

System.out.println(doctorCount+"********************");
}
```

```
                sc.close();   //closing the file
            }catch(Exception ex){

                System.out.println("File not found.");
                return;
            }

        }

        public void addDoctor(Doctor d){
            doctorList[doctorCount] = d;
            doctorCount++;



        String DrDetails = d.getId() + "\n" + d.getName() + "\n" +d.getGender() + "\n"
+d.getMobileNo() + "\n"+ d.getDepartment() + "\n" +
                                                d.getJoiningDate() + "\n" + d.getBmdcReg() +"\n"
+ d.getPassword() + "\n" + "\n";
            try{
                FileWriter fw = new FileWriter("Files/DoctorList.txt",true);
                fw.write(DrDetails);
                fw.close();
            }catch(Exception ex){
                System.out.println(ex);
```

```
        }
    }

    public int doctorExists(String name){
        int index = -1;
        for(int i = 0; i<doctorCount; i++){
            if(doctorList[i]!=null && doctorList[i].getName().equals(name)){
                index = i;
                break;
            }
        }
        return index;
    }

    public Doctor checkPassword(int index, String pass){
        Doctor d = null;
        if(doctorList[index].getPassword().equals(pass)){
            d = doctorList[index];
        }
        return d;
    }

    public Doctor getDoctor(int index){
```

```
                return doctorList[index];
        }

        public String[] getDoctorNames() {
    String doctorNames [] = new String[doctorCount];
    for (int i = 0; i < doctorCount; i++) {
                if (doctorNames!= null && doctorList[i]!= null){
      doctorNames[i] = "Dr. "+doctorList[i].getName();
    }}
    return doctorNames;

}

        /*public void deleteDoctor(Doctor d){
                for(int i = 0; i<doctorCount; i++){
                        if(doctorList[i] == d){
                                for(int j = i; j<doctorCount-1; j++){
                                        doctorList[j] = doctorList[j+1];
                                }
                                doctorCount--;
                                doctorList[doctorCount]=null;
                                break;
```

```
                        }
                }
        }*/
}
```

After (Modified Code)

```
package Classes;
import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class RegDoctorList {
    private static final String DOCTOR_FILE_PATH = "Files/DoctorList.txt";
    private final List<Doctor> doctorList = new ArrayList<>();

    public RegDoctorList() {
        loadDoctorsFromFile();
    }

    private void loadDoctorsFromFile() {
        File file = new File(DOCTOR_FILE_PATH);
        if (!file.exists()) {
            return; // Silently handle missing file, assume empty list
```

```
        }
        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String id = sc.nextLine();
                String name = sc.nextLine();
                String mobileNo = sc.nextLine();
                String gender = sc.nextLine();
                String department = sc.nextLine();
                String joiningDate = sc.nextLine();
                String bmdcReg = sc.nextLine();
                String password = sc.nextLine();
                if (sc.hasNextLine()) sc.nextLine(); // Skip extra newline


                Doctor d = new Doctor(id, name, mobileNo, gender, department, joiningDate,
bmdcReg, password);
                doctorList.add(d);
            }
        } catch (FileNotFoundException ex) {
            // Log or handle appropriately; for now, empty list
        } catch (IOException ex) {
            throw new RuntimeException("Error loading doctors", ex);
        }
    }
```

```
    public void addDoctor(Doctor d) {
        doctorList.add(d);
        saveDoctorToFile(d);
    }

    private void saveDoctorToFile(Doctor d) {
        String drDetails = d.getId() + "\n" + d.getName() + "\n" + d.getMobileNo() + "\n" +
d.getGender() + "\n" +
                    d.getDepartment() + "\n" + d.getJoiningDate() + "\n" + d.getBmdcReg() + "\n"
+
                    d.getPassword() + "\n\n";
        try (FileWriter fw = new FileWriter(DOCTOR_FILE_PATH, true)) {
            fw.write(drDetails);
        } catch (IOException ex) {
            throw new RuntimeException("Error saving doctor", ex);
        }
    }

    public int doctorExists(String name) {
        for (int i = 0; i < doctorList.size(); i++) {
            if (doctorList.get(i).getName().equals(name)) {
                return i;
            }
        }
```

```
    return -1;
}

public Doctor checkPassword(int index, String pass) {
    if (index < 0 || index >= doctorList.size()) {
        return null;
    }
    Doctor d = doctorList.get(index);
    return d.getPassword().equals(pass) ? d : null;
}

public Doctor getDoctor(int index) {
    if (index < 0 || index >= doctorList.size()) {
        return null;
    }
    return doctorList.get(index);
}

public String[] getDoctorNames() {
    String[] doctorNames = new String[doctorList.size()];
    for (int i = 0; i < doctorList.size(); i++) {
        doctorNames[i] = "Dr. " + doctorList.get(i).getName();
    }
```

```
        return doctorNames;
    }
}
```

Why : The code contained several **code smells** that affected its quality and maintainability. These included **duplicated code** in file I/O operations, the use of **magic numbers** such as a fixed array size (100), and reliance on **static variables** like doctorCount to track elements. It also had **poor error handling**, where exceptions were not properly managed, and **dead code**, including unnecessary debug prints and commented-out methods. Additionally, the code showed **primitive obsession** by using arrays instead of more flexible data structures like List or ArrayList.

PrescriptionList.java

Before (Original Code)

```
package Classes;

import java.io.*;

import java.util.Scanner;


public class PrescriptionList {

    public Prescription[] prescriptionList = new Prescription[100];

    public static int pCount = 0;
```

```
    public PrescriptionList() {
        try {
            File file = new File("Files/prescriptions.txt");
            Scanner sc = new Scanner(file);
            while (sc.hasNextLine()) {
                String patientId = sc.nextLine().split(": ")[1];  // Patient ID
                String patientName = sc.nextLine().split(": ")[1];  // Patient Name
                String diagnosis = sc.nextLine().split(": ")[1];  // Diagnosis
                String doctorName = sc.nextLine().split(": ")[1];  // Doctor's Name
                String medicineName = sc.nextLine().split(": ")[1];  // Medicine Name
                String dosage = sc.nextLine().split(": ")[1];  // Dosage
                String usageInstructions = sc.nextLine().split(": ")[1];  // Usage Instructions
                if (sc.hasNextLine()) sc.nextLine();  // Skip

                Prescription p = new Prescription(patientId, patientName, diagnosis, doctorName,
medicineName, dosage, usageInstructions);
                prescriptionList[pCount] = p;
                pCount++;
            }
            sc.close();
        } catch (Exception ex) {
            System.out.println("File not found or error reading file.");
        }
    }
```

```
public void addPrescription(Prescription p) {
    try {
        prescriptionList[pCount] = p;
        pCount++;
    } catch (Exception ex) {
        System.out.println("Array full");
    }

    String prescriptionDetails = "Patient ID: " + p.getPatientId() + "\n" +
                                 "Patient Name: " + p.getPatientName() + "\n" +
                                 "Diagnosis: " + p.getDiagnosis() + "\n" +
                                 "Doctor's Name: " + p.getDoctorName() + "\n" +
                                 "Medicine Name: " + p.getMedicineName() + "\n" +
                                 "Dosage: " + p.getDosage() + "\n" +
                                 "Usage Instructions: " + p.getUsageInstructions() + "\n\n";

    try {
        FileWriter fw = new FileWriter("Files/prescriptions.txt", true);
        fw.write(prescriptionDetails);
        fw.close();
    } catch (Exception ex) {
        System.out.println(ex);
```

```
        }
    }

    public int searchPrescription(String patientId) {
        int index = -1;
        for (int i = 0; i < pCount; i++) {
            if (prescriptionList[i] != null &&
prescriptionList[i].getPatientId().equalsIgnoreCase(patientId)) {
                index = i;
                break;
            }
        }
        return index;
    }

    public Prescription getPrescription(int index) {
        return prescriptionList[index];
    }
}
```

After (Modified Code)

```
package Classes;

import java.io.*;
```

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class PrescriptionList {
    private static final String PRESCRIPTION_FILE_PATH = "Files/prescriptions.txt";
    private final List<Prescription> prescriptionList = new ArrayList<>();

    public PrescriptionList() {
        loadPrescriptionsFromFile();
    }

    private void loadPrescriptionsFromFile() {
        File file = new File(PRESCRIPTION_FILE_PATH);
        if (!file.exists()) {
            return;
        }
        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNextLine()) {
                String patientId = sc.nextLine().split(": ", 2)[1];
                String patientName = sc.nextLine().split(": ", 2)[1];
                String diagnosis = sc.nextLine().split(": ", 2)[1];
                String doctorName = sc.nextLine().split(": ", 2)[1];
```

```
            String medicineName = sc.nextLine().split(": ", 2)[1];
            String dosage = sc.nextLine().split(": ", 2)[1];
            String usageInstructions = sc.nextLine().split(": ", 2)[1];
            if (sc.hasNextLine()) sc.nextLine();

            Prescription p = new Prescription(patientId, patientName, diagnosis, doctorName,
medicineName, dosage, usageInstructions);
            prescriptionList.add(p);
        }
    } catch (FileNotFoundException ex) {
        // Empty list
    } catch (IOException ex) {
        throw new RuntimeException("Error loading prescriptions", ex);
    } catch (ArrayIndexOutOfBoundsException ex) {
        throw new RuntimeException("Invalid file format", ex);
    }
}

public void addPrescription(Prescription p) {
    prescriptionList.add(p);
    savePrescriptionToFile(p);
}

private void savePrescriptionToFile(Prescription p) {
```

```
    String prescriptionDetails = "Patient ID: " + p.getPatientId() + "\n" +
                                 "Patient Name: " + p.getPatientName() + "\n" +
                                 "Diagnosis: " + p.getDiagnosis() + "\n" +
                                 "Doctor's Name: " + p.getDoctorName() + "\n" +
                                 "Medicine Name: " + p.getMedicineName() + "\n" +
                                 "Dosage: " + p.getDosage() + "\n" +
                                 "Usage Instructions: " + p.getUsageInstructions() + "\n\n";
    try (FileWriter fw = new FileWriter(PRESCRIPTION_FILE_PATH, true)) {
        fw.write(prescriptionDetails);
    } catch (IOException ex) {
        throw new RuntimeException("Error saving prescription", ex);
    }
}

public int searchPrescription(String patientId) {
    for (int i = 0; i < prescriptionList.size(); i++) {
        if (prescriptionList.get(i).getPatientId().equalsIgnoreCase(patientId)) {
            return i;
        }
    }
    return -1;
}
```

```
    public Prescription getPrescription(int index) {
        if (index < 0 || index >= prescriptionList.size()) {
            return null;
        }
        return prescriptionList.get(index);
    }
}
```

Why: The refactored code removed several **code smells** to improve quality and maintainability. The **duplicated file I/O code** was streamlined to avoid repetition. **Magic numbers and strings** were replaced with constants, and fixed arrays were replaced with a dynamic **List** for better flexibility. The unnecessary **static variable pCount** was removed and replaced with more reliable methods like checking the list size. **Error handling** was improved by using more specific exceptions to properly handle format-related issues. Additionally, **defensive bounds checks** were added to prevent invalid index access and make the code more robust.

Prescription.java

Before (Original Code)

```
package Classes;
```

```
public class Prescription {

    private String patientId;

    private String patientName;

    private String diagnosis;

    private String doctorName;

    private String medicineName;

    private String dosage;

    private String usageInstructions;


    // Constructor

    public Prescription(String patientId, String patientName, String diagnosis, String doctorName,
String medicineName, String dosage, String usageInstructions) {

        this.patientId = patientId;

        this.patientName = patientName;

        this.diagnosis = diagnosis;

        this.doctorName = doctorName;

        this.medicineName = medicineName;

        this.dosage = dosage;

        this.usageInstructions = usageInstructions;

    }


    // Getters

    public String getPatientId() {

        return patientId;
```

```
}

public String getPatientName() {
    return patientName;
}

public String getDiagnosis() {
    return diagnosis;
}

public String getDoctorName() {
    return doctorName;
}

public String getMedicineName() {
    return medicineName;
}

public String getDosage() {
    return dosage;
}

public String getUsageInstructions() {
```

```
        return usageInstructions;
    }
}
```

After (Modified Code)

java

```
package Classes;


public class Prescription {
    private final String patientId;
    private final String patientName;
    private final String diagnosis;
    private final String doctorName;
    private final String medicineName;
    private final String dosage;
    private final String usageInstructions;


    public Prescription(String patientId, String patientName, String diagnosis, String doctorName,
String medicineName, String dosage, String usageInstructions) {
        this.patientId = patientId;
        this.patientName = patientName;
        this.diagnosis = diagnosis;
        this.doctorName = doctorName;
        this.medicineName = medicineName;
```

```
    this.dosage = dosage;
    this.usageInstructions = usageInstructions;
}

public String getPatientId() {
    return patientId;
}

public String getPatientName() {
    return patientName;
}

public String getDiagnosis() {
    return diagnosis;
}

public String getDoctorName() {
    return doctorName;
}

public String getMedicineName() {
    return medicineName;
}
```

```
    public String getDosage() {
        return dosage;
    }

    public String getUsageInstructions() {
        return usageInstructions;
    }
}
```

Why: The refactoring addressed some key **code smells** to improve code reliability and design. A **mutable object** was converted into an **immutable object** by making its fields final, which helps prevent unintended changes to its state and improves thread safety. Additionally, the presence of a **long parameter list** was identified as a design smell; although it was not refactored in this stage, the **Builder pattern** was suggested as a future improvement to make object creation cleaner and more maintainable.

Patient.java

Before (Original Code)

```
package Classes;
```

```
public class Patient extends Person {

        private String age;

        private String address;

    private String appointedDoctor;

    private String room;

    private String diagnosis;

    private String deposit;

    public Patient( String id, String name, String mobileNo, String gender, String age, String
address, String appointedDoctor, String room, String diagnosis,String deposit) {

        super(id,name,mobileNo,gender);



                this.age = age;

                this.address =  address;

        this.appointedDoctor = appointedDoctor;

        this.room = room;

        this.diagnosis = diagnosis;

                this.deposit = deposit;

    }



        public String getAge() {

        return age;

    }
```

```
public void setAge(String age) {
    this.age = age;
}



        public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}



public String getAppointedDoctor() {
    return appointedDoctor;
}

public void setAppointedDoctor(String appointedDoctor) {
    this.appointedDoctor = appointedDoctor;
}

public String getRoom() {
```

```
    return room;
}

public void setRoom(String room) {
    this.room = room;
}

public String getDiagnosis() {
    return diagnosis;
}

public void setDiagnosis(String diagnosis) {
    this.diagnosis = diagnosis;
}

        public String getDeposit() {
    return deposit;
}

public void setDeposit(String deposit) {
    this.deposit = deposit;
}
```

After (Modified Code)

```
package Classes;

public class Patient extends Person {

    private String age;

    private String address;

    private String appointedDoctor;

    private String room;

    private String diagnosis;

    private String deposit;


    public Patient(String id, String name, String mobileNo, String gender, String age, String
address, String appointedDoctor, String room, String diagnosis, String deposit) {

        super(id, name, mobileNo, gender);

        this.age = age;

        this.address = address;

        this.appointedDoctor = appointedDoctor;

        this.room = room;

        this.diagnosis = diagnosis;

        this.deposit = deposit;

    }


    public String getAge() {

        return age;
```

```
}

public void setAge(String age) {
    this.age = age;
}

public String getAddress() {
    return address;
}

public void setAddress(String address) {
    this.address = address;
}

public String getAppointedDoctor() {
    return appointedDoctor;
}

public void setAppointedDoctor(String appointedDoctor) {
    this.appointedDoctor = appointedDoctor;
}

public String getRoom() {
```

```
        return room;
    }

    public void setRoom(String room) {
        this.room = room;
    }

    public String getDiagnosis() {
        return diagnosis;
    }

    public void setDiagnosis(String diagnosis) {
        this.diagnosis = diagnosis;
    }

    public String getDeposit() {
        return deposit;
    }

    public void setDeposit(String deposit) {
        this.deposit = deposit;
    }
}
```

Why **This Is a Code Smell?**

The code contains **Long Parameter List**, **Data Clump**, and **Primitive Obsession**.

Problems:

1. Related data is passed separately instead of grouping into objects (e.g., an Address class).
2. Basic types like String are used for values such as age or deposit instead of proper types (int, double).

Doctor.java

Before (Original Code)

```
package Classes;


public class Doctor extends Person {

   private String department;

   private String joiningDate;

   private String bmdcReg;

         private String password;


   public Doctor(String id, String name, String gender, String mobileNo, String department,
String joiningDate, String bmdcReg, String password) {

      super(id,name,gender,mobileNo);

      this.department = department;
```

```
    this.joiningDate = joiningDate;
    this.bmdcReg = bmdcReg;
                this.password = password;
}

public String getDepartment() {
    return department;
}

public void setDepartment(String department) {
    this.department = department;
}

public String getJoiningDate() {
    return joiningDate;
}

public void setJoiningDate(String joiningDate) {
    this.joiningDate = joiningDate;
}

public String getPassword() {
                return password;
```

```
        }

        public void setPassword(String password){
                this.password = password;
        }

    public String getBmdcReg() {
        return bmdcReg;
    }

    public void setBmdcReg(String bmdcReg) {
        this.bmdcReg = bmdcReg;
    }
}
```

After (Modified Code)

```
package Classes;
public class Doctor extends Person {
    private String department;
    private String joiningDate;
    private String bmdcReg;
    private String password;
```

```
  public Doctor(String id, String name, String mobileNo, String gender, String department, String
joiningDate, String bmdcReg, String password) {

        super(id, name, mobileNo, gender);

        this.department = department;

        this.joiningDate = joiningDate;

        this.bmdcReg = bmdcReg;

        this.password = password;

    }


    public String getDepartment() {

        return department;

    }


    public void setDepartment(String department) {

        this.department = department;

    }


    public String getJoiningDate() {

        return joiningDate;

    }


    public void setJoiningDate(String joiningDate) {

        this.joiningDate = joiningDate;

    }
```

```
    public String getBmdcReg() {
        return bmdcReg;
    }

    public void setBmdcReg(String bmdcReg) {
        this.bmdcReg = bmdcReg;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

Why This Code Smells?

The code contains several issues such as **Long Parameter List** and **Primitive Obsession**, which affect both **design and security**.

Problems:

1. **Long Parameter List:** Methods become harder to read and maintain.
2. **Plain Text Passwords:** Storing passwords in plain text is insecure; production systems should use **secure hashing**.

3. **Primitive Obsession:** Dates are stored using simple types instead of proper date/time classes, reducing validation and reliability.

PList.java

Before (Original Code)

```
package Classes;

import Interfaces.*;


import java.io.*;

import java.util.Scanner;


public class PList implements IPList {

	public Patient patientList[] = new Patient[100];

	public static int pCount = 0;


	public PList(){

		try{

		File file = new File("Files/PatientList.txt");

		Scanner sc = new Scanner(file);

			while(sc.hasNext()){
```

```
                    String line1 = sc.nextLine();  //pid
                    String line2 = sc.nextLine();  //name
                    String line3 = sc.nextLine();  //mobile
                    String line4 = sc.nextLine();  //gender
                    String line5 = sc.nextLine();  //age
                    String line6 = sc.nextLine();   //address
                    String line7 = sc.nextLine();  //appDoctor
                    String line8 = sc.nextLine();  //room
                    String line9 = sc.nextLine();  //diagnosic
                    String line10 = sc.nextLine();  //deposit
                    String line11= sc.nextLine();



                    Patient p = new Patient (line1, line2, line3, line4, line5, line6,
line7, line8, line9, line10);
                    patientList[pCount] = p;
                    pCount++;
                }
            sc.close();


            }catch(Exception ex){
                System.out.println("File not found.");
                return;
```

```
        }
    }

    public void addPatient(Patient p){

        try{
            patientList[pCount] = p;
            pCount++;
        }catch (Exception ex){
                System.out.println("Array full");
        }

        String patientDatiels = p.getId() + "\n" + p.getName() + "\n" +
                                p.getMobileNo() + "\n" +
p.getGender() + "\n" +
                                p.getAge() + "\n" + p.getAddress() +
"\n" +
                                p.getAppointedDoctor() + "\n" +
p.getRoom() + "\n" +
                                p.getDiagnosis() + "\n" +
p.getDeposit() + "\n" +"\n";

        try{
            FileWriter fw = new FileWriter("Files/PatientList.txt",true);
            fw.write(patientDatiels);
```

```
            fw.close();
        }catch(Exception ex){
                System.out.println(ex);
        }




    }

    public int searchPatient(String id){
        int index = -1;
        for(int i = 0; i<pCount; i++){
            if(patientList[i]!= null && patientList[i].getId().equals(id)){
              index = i;
              break;
            }
        }
        return index;
    }

    public Patient getPatient(int index){
        return patientList[index];
    }
```

```
public void deletePatient(Patient p){

        for( int i = 0; i<pCount; i++){
            if(patientList[i] == p){
                for(int j = i; j<pCount-1; j++){
                    patientList[j] = patientList[j+1];
                }
                pCount--;
                patientList[pCount] = null;
                break;
            }
        }


        String patientToDelete = p.getId() + "\n" + p.getName() + "\n" +
                                p.getMobileNo() + "\n" +
p.getGender() + "\n" +
                                p.getAge() + "\n" + p.getAddress()
+ "\n" +
                                p.getAppointedDoctor() + "\n" +
p.getRoom() + "\n" +
                                p.getDiagnosis() + "\n" +
p.getDeposit() + "\n" ;
```

```
try{
        String filepath = "Files/PatientList.txt";
        File originalFile = new File(filepath);

        String newDetails = "";
        Scanner sc = new Scanner(originalFile);
        while(sc.hasNext()){
                String readUser = sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine();

                System.out.println(readUser);
                System.out.println(patientToDelete);

                if(readUser.equals(patientToDelete)){
```

```
                    System.out.println("Equal, so deleting");
                    continue;
                }
                System.out.println("Writing");
                newDetails += readUser + "\n";

            } sc.close();
            FileWriter fw = new FileWriter(filepath);
            fw.write(newDetails);
            fw.close();

        } catch (Exception ex){
            System.out.println(ex);
        }

    }

    public void updatePatient(String patientToUpdate, Patient updatedPatient){

        String updatedDetails = updatedPatient.getId() + "\n" +
                                    updatedPatient.getName() + "\n" +
                                    updatedPatient.getMobileNo() + "\n"
+
                                    updatedPatient.getGender() + "\n" +
```

```
                                                        updatedPatient.getAge() + "\n" +
                                                        updatedPatient.getAddress() + "\n" +

updatedPatient.getAppointedDoctor() + "\n" +
                                                        updatedPatient.getRoom() + "\n"+
                                                        updatedPatient.getDiagnosis() + "\n"
+
                                                        updatedPatient.getDeposit() + "\n";

            System.out.println("Want to update:\n "+ updatedDetails+ "\n");
            System.out.println("----------------------------------------");

            //Update in file
            try{
                String filepath = "Files/PatientList.txt";
                File originalFile = new File(filepath);

                String newDetails = "";
                Scanner sc = new Scanner(originalFile);
                while(sc.hasNext()){
                    String readUser = sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
                        readUser += sc.nextLine() + "\n";
```

```
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine() + "\n";
                    readUser += sc.nextLine();

                System.out.println("read:\n "+ readUser+"\n-----------------");
                System.out.println("updt:\n "+ patientToUpdate
+"\n-----------------");

                if(readUser.equals(patientToUpdate)){
                        System.out.println("Updated");
                        newDetails += updatedDetails + "\n";
                } else{
                        System.out.println("Writing");
                        newDetails += readUser + "\n";
                }
            } sc.close();
            FileWriter fw = new FileWriter(filepath);
            fw.write(newDetails);
            fw.close();
```

```
            } catch (Exception ex){
                System.out.println(ex);
            }
        }

}
```

After (Modified Code)

```
package Classes;
import Interfaces.*;

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class PList implements IPList {
    private static final String PATIENT_FILE_PATH = "Files/PatientList.txt";
    private final List<Patient> patientList = new ArrayList<>();

    public PList() {
        loadPatientsFromFile();
```

```
    }

    private void loadPatientsFromFile() {
        File file = new File(PATIENT_FILE_PATH);
        if (!file.exists()) {
            return;
        }
        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String id = sc.nextLine();
                String name = sc.nextLine();
                String mobileNo = sc.nextLine();
                String gender = sc.nextLine();
                String age = sc.nextLine();
                String address = sc.nextLine();
                String appointedDoctor = sc.nextLine();
                String room = sc.nextLine();
                String diagnosis = sc.nextLine();
                String deposit = sc.nextLine();
                if (sc.hasNextLine()) sc.nextLine();

                Patient p = new Patient(id, name, mobileNo, gender, age, address, appointedDoctor,
room, diagnosis, deposit);
                patientList.add(p);
```

```
        }
    } catch (FileNotFoundException ex) {
        // Empty list
    } catch (IOException ex) {
        throw new RuntimeException("Error loading patients", ex);
    }
}

public void addPatient(Patient p) {
    patientList.add(p);
    savePatientToFile(p);
}

private void savePatientToFile(Patient p) {
    String patientDetails = p.getId() + "\n" + p.getName() + "\n" +
                            p.getMobileNo() + "\n" + p.getGender() + "\n" +
                            p.getAge() + "\n" + p.getAddress() + "\n" +
                            p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
                            p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
    try (FileWriter fw = new FileWriter(PATIENT_FILE_PATH, true)) {
        fw.write(patientDetails);
    } catch (IOException ex) {
        throw new RuntimeException("Error saving patient", ex);
```

```
    }
}

public int searchPatient(String id) {
    for (int i = 0; i < patientList.size(); i++) {
        if (patientList.get(i).getId().equals(id)) {
            return i;
        }
    }
    return -1;
}

public Patient getPatient(int index) {
    if (index < 0 || index >= patientList.size()) {
        return null;
    }
    return patientList.get(index);
}

public void deletePatient(Patient p) {
    patientList.remove(p);
    removePatientFromFile(p);
}
```

```
private void removePatientFromFile(Patient p) {
    String patientToDelete = buildPatientFileString(p);
    try {
        List<String> lines = readAllLines(PATIENT_FILE_PATH);
        StringBuilder newContent = new StringBuilder();
        StringBuilder currentBlock = new StringBuilder();
        for (String line : lines) {
            currentBlock.append(line).append("\n");
            if (currentBlock.toString().endsWith("\n\n")) { // Block end
                if (!currentBlock.toString().equals(patientToDelete)) {
                    newContent.append(currentBlock);
                }
                currentBlock = new StringBuilder();
            }
        }
        writeAllText(PATIENT_FILE_PATH, newContent.toString());
    } catch (IOException ex) {
        throw new RuntimeException("Error deleting patient from file", ex);
    }
}

private String buildPatientFileString(Patient p) {
```

```
    return p.getId() + "\n" + p.getName() + "\n" +
            p.getMobileNo() + "\n" + p.getGender() + "\n" +
            p.getAge() + "\n" + p.getAddress() + "\n" +
            p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
            p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
}

public void updatePatient(String patientToUpdate, Patient updatedPatient) {
    // Assume update in memory first if needed; here focus on file
    updatePatientInFile(patientToUpdate, updatedPatient);
}

private void updatePatientInFile(String patientToUpdate, Patient updatedPatient) {
    String updatedDetails = buildPatientFileString(updatedPatient);
    try {
        List<String> lines = readAllLines(PATIENT_FILE_PATH);
        StringBuilder newContent = new StringBuilder();
        StringBuilder currentBlock = new StringBuilder();
        for (String line : lines) {
            currentBlock.append(line).append("\n");
            if (currentBlock.toString().endsWith("\n\n")) {
                if (currentBlock.toString().equals(patientToUpdate)) {
                    newContent.append(updatedDetails);
```

```
            } else {
                newContent.append(currentBlock);
            }
            currentBlock = new StringBuilder();
        }
    }
    writeAllText(PATIENT_FILE_PATH, newContent.toString());
  } catch (IOException ex) {
    throw new RuntimeException("Error updating patient in file", ex);
  }
}

// Helper methods to reduce duplication in file ops
private List<String> readAllLines(String path) throws IOException {
  List<String> lines = new ArrayList<>();
  try (BufferedReader br = new BufferedReader(new FileReader(path))) {
    String line;
    while ((line = br.readLine()) != null) {
      lines.add(line);
    }
  }
  return lines;
}
```

```
    private void writeAllText(String path, String content) throws IOException {
        try (FileWriter fw = new FileWriter(path)) {
            fw.write(content);
        }
    }
}
```

Why This Code Smells?

The code is full of **Duplicated Code**, **Long Method**, **Magic Number**, and **Dead Code**.

Improvements:

1. Duplicate logic in delete and update operations moved to helper methods (readAllLines, writeAllText).
2. Long methods split into smaller functions for better readability.
3. Magic numbers/strings replaced with constants.
4. Fixed arrays replaced with dynamic List for flexibility.
5. Unused debug code removed and error handling improved.

Person.java

Before (Original Code)

```
package Classes;
```

```
public abstract class Person {
    public String id;
    private String name;
    private String gender;
    private String mobileNo;
    private String address;

    public Person(String id,String name,String mobileNo,String gender) {
        this.id = id;
        this.name = name;
        this.mobileNo = mobileNo;
        this.gender = gender;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }
```

```
public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

        public String getMobileNo() {
    return mobileNo;
}

public void setMobileNo(String mobileNo) {
    this.mobileNo = mobileNo;
}

public String getGender() {
    return gender;
}

public void setGender(String gender) {
    this.gender = gender;
```

```
    }



}
```

After (Modified Code)

```
package Classes;

public abstract class Person {

    private String id;

    private String name;

    private String gender;

    private String mobileNo;


    public Person(String id, String name, String mobileNo, String gender) {

        this.id = id;

        this.name = name;

        this.mobileNo = mobileNo;

        this.gender = gender;

    }


    public String getId() {

        return id;
```

```
}

public void setId(String id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public String getMobileNo() {
    return mobileNo;
}

public void setMobileNo(String mobileNo) {
    this.mobileNo = mobileNo;
}

public String getGender() {
```

```
        return gender;
    }

    public void setGender(String gender) {
        this.gender = gender;
    }
}
```

Why This Is Better ?

The refactoring removed Dead Code.

Improvements:

1. Unused address field (dead code) was removed to keep the code clean.
2. The public id field was changed to **private** and accessed through getter/setter methods.

Appointment.java

Before (Original Code)

```
package Classes;

public class Appointment {
    private String appID;
    private String patientName;
    private String diagnosis;
    private String doctorName;
```

```
    private String appDate;

    private String appTime;


    public Appointment(String appID, String patientName, String diagnosis, String doctorName,
String appDate, String appTime) {

        this.appID = appID;

        this.patientName = patientName;

        this.diagnosis = diagnosis;

        this.doctorName = doctorName;

        this.appDate = appDate;

        this.appTime = appTime;

    }


    public String getAppId() {

        return appID;

    }


    public String getPatientName() {

        return patientName;

    }


    public String getDiagnosis() {

        return diagnosis;

    }
```

```
    public String getDoctorName() {
        return doctorName;
    }

    public String getAppDate() {
        return appDate;
    }

    public String getAppTime() {
        return appTime;
    }
}
```

After (Modified Code)

```
package Classes;

public class Appointment {

    private final String appID;
    private final String patientName;
    private final String diagnosis;
```

```
private final String doctorName;
private final String appDate;
private final String appTime;

private Appointment(Builder builder) {
    this.appID       = builder.appID;
    this.patientName = builder.patientName;
    this.diagnosis   = builder.diagnosis;
    this.doctorName  = builder.doctorName;
    this.appDate     = builder.appDate;
    this.appTime     = builder.appTime;
}

// getters remain the same
public String getAppId()        { return appID; }
public String getPatientName()  { return patientName; }
public String getDiagnosis()    { return diagnosis; }
public String getDoctorName()   { return doctorName; }
public String getAppDate()      { return appDate; }
public String getAppTime()      { return appTime; }

// Static nested Builder class
public static class Builder {
```

```
// required fields
private final String appID;
private final String patientName;

// optional / nullable fields
private String diagnosis   = "";
private String doctorName  = "";
private String appDate     = "";
private String appTime     = "";

public Builder(String appID, String patientName) {
    this.appID = requireNonNull(appID, "appID is required");
    this.patientName = requireNonNull(patientName, "patientName is required");
}

public Builder diagnosis(String diagnosis) {
    this.diagnosis = diagnosis != null ? diagnosis : "";
    return this;
}

public Builder doctorName(String doctorName) {
    this.doctorName = doctorName != null ? doctorName : "";
    return this;
```

```
        }

        public Builder appDate(String appDate) {
            this.appDate = appDate != null ? appDate : "";
            return this;
        }

        public Builder appTime(String appTime) {
            this.appTime = appTime != null ? appTime : "";
            return this;
        }

        public Appointment build() {
            return new Appointment(this);
        }
    }

    // Optional: helper method (requires static import of Objects.requireNonNull)
    private static <T> T requireNonNull(T obj, String message) {
        if (obj == null) throw new NullPointerException(message);
        return obj;
    }
}
```

Why Is This Better?

The refactoring addressed the Long Parameter List.

Improvements:

1. The **long parameter list** was identified as a design issue for future refactoring.

AppointmentList.java

Before (Original Code)

```
package Classes;


import Interfaces.*;

import java.io.*;

import java.util.Scanner;


public class AppointmentList {

    public Appointment[] appointmentList = new Appointment[100];

    public static int appCount = 0;


    public AppointmentList() {

        try {

            File file = new File("Files/AppointmentList.txt");
```

```
        Scanner sc = new Scanner(file);
        while (sc.hasNext()) {
            String line1 = sc.nextLine();  // appID
            String line2 = sc.nextLine();  // patientName
            String line3 = sc.nextLine();  // diagnosis
            String line4 = sc.nextLine();  // doctorName
            String line5 = sc.nextLine();  // appDate
            String line6 = sc.nextLine();  // appTime
            if (sc.hasNextLine()) sc.nextLine();  // Skip empty line

            Appointment app = new Appointment(line1, line2, line3, line4, line5, line6);
            appointmentList[appCount] = app;
            appCount++;
        }
        sc.close();
    } catch (Exception ex) {
        System.out.println("File not found.");
        return;
    }
}

public void addAppointment(Appointment app) {
    try {
```

```
        appointmentList[appCount] = app;
        appCount++;
    } catch (Exception ex) {
        System.out.println("Array full");
    }

    String appointmentDetails = app.getAppId() + "\n" +
                                app.getPatientName() + "\n" +
                                app.getDiagnosis() + "\n" +
                                app.getDoctorName() + "\n" +
                                app.getAppDate() + "\n" +
                                app.getAppTime() + "\n" + "\n";

    try {
        FileWriter fw = new FileWriter("Files/AppointmentList.txt", true);
        fw.write(appointmentDetails);
        fw.close();
    } catch (Exception ex) {
        System.out.println(ex);
    }
}

public int searchAppointment(String appId) {
```

```
    int index = -1;
    for (int i = 0; i < appCount; i++) {
        if (appointmentList[i] != null && appointmentList[i].getAppId().equals(appId)) {
            index = i;
            break;
        }
    }
    return index;
}

public Appointment getAppointment(int index) {
    return appointmentList[index];
}

public void deleteAppointment(Appointment app) {
    for (int i = 0; i < appCount; i++) {
        if (appointmentList[i] == app) {
            for (int j = i; j < appCount - 1; j++) {
                appointmentList[j] = appointmentList[j + 1];
            }
            appCount--;
            appointmentList[appCount] = null;
            break;
```

```
    }
}

String appointmentToDelete = app.getAppId() + "\n" +
                        app.getPatientName() + "\n" +
                        app.getDiagnosis() + "\n" +
                        app.getDoctorName() + "\n" +
                        app.getAppDate() + "\n" +
                        app.getAppTime() + "\n";

try {
    String filepath = "Files/AppointmentList.txt";
    File originalFile = new File(filepath);

    String newDetails = "";
    Scanner sc = new Scanner(originalFile);
    while (sc.hasNext()) {
        String readUser = sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine();
```

```
            if (sc.hasNextLine()) readUser += "\n";


            if (readUser.equals(appointmentToDelete)) {
                continue;
            }
            newDetails += readUser + "\n";
        }
        sc.close();
        FileWriter fw = new FileWriter(filepath);
        fw.write(newDetails);
        fw.close();
    } catch (Exception ex) {
        System.out.println(ex);
    }
}


public void updateAppointment(String appointmentToUpdate, Appointment
updatedAppointment) {
    String updatedDetails = updatedAppointment.getAppId() + "\n" +
                    updatedAppointment.getPatientName() + "\n" +
                    updatedAppointment.getDiagnosis() + "\n" +
                    updatedAppointment.getDoctorName() + "\n" +
                    updatedAppointment.getAppDate() + "\n" +
                    updatedAppointment.getAppTime() + "\n";
```

```
// Update in file
try {
    String filepath = "Files/AppointmentList.txt";
    File originalFile = new File(filepath);

    String newDetails = "";
    Scanner sc = new Scanner(originalFile);
    while (sc.hasNext()) {
        String readUser = sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine() + "\n";
        readUser += sc.nextLine();
        if (sc.hasNextLine()) readUser += "\n";

        if (readUser.equals(appointmentToUpdate)) {
            newDetails += updatedDetails + "\n";
        } else {
            newDetails += readUser + "\n";
        }
    }
```

```
            sc.close();
            FileWriter fw = new FileWriter(filepath);
            fw.write(newDetails);
            fw.close();
        } catch (Exception ex) {
            System.out.println(ex);
        }
    }
}
```

After (Modified Code)

```
package Classes;

import java.io.*;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class AppointmentList {
    private static final String APPOINTMENT_FILE_PATH = "Files/AppointmentList.txt";
    private final List<Appointment> appointmentList = new ArrayList<>();

    public AppointmentList() {
```

```
        loadAppointmentsFromFile();
    }

    private void loadAppointmentsFromFile() {
        File file = new File(APPOINTMENT_FILE_PATH);
        if (!file.exists()) {
            return;
        }
        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String appID = sc.nextLine();
                String patientName = sc.nextLine();
                String diagnosis = sc.nextLine();
                String doctorName = sc.nextLine();
                String appDate = sc.nextLine();
                String appTime = sc.nextLine();
                if (sc.hasNextLine()) sc.nextLine();

                Appointment app = new Appointment(appID, patientName, diagnosis, doctorName,
appDate, appTime);
                appointmentList.add(app);
            }
        } catch (FileNotFoundException ex) {
            // Empty list
```

```
    } catch (IOException ex) {
        throw new RuntimeException("Error loading appointments", ex);
    }
}

public void addAppointment(Appointment app) {
    appointmentList.add(app);
    saveAppointmentToFile(app);
}

private void saveAppointmentToFile(Appointment app) {
    String appointmentDetails = app.getAppId() + "\n" +
                                app.getPatientName() + "\n" +
                                app.getDiagnosis() + "\n" +
                                app.getDoctorName() + "\n" +
                                app.getAppDate() + "\n" +
                                app.getAppTime() + "\n\n";
    try (FileWriter fw = new FileWriter(APPOINTMENT_FILE_PATH, true)) {
        fw.write(appointmentDetails);
    } catch (IOException ex) {
        throw new RuntimeException("Error saving appointment", ex);
    }
}
```

```
public int searchAppointment(String appId) {
    for (int i = 0; i < appointmentList.size(); i++) {
        if (appointmentList.get(i).getAppId().equals(appId)) {
            return i;
        }
    }
    return -1;
}

public Appointment getAppointment(int index) {
    if (index < 0 || index >= appointmentList.size()) {
        return null;
    }
    return appointmentList.get(index);
}

public void deleteAppointment(Appointment app) {
    appointmentList.remove(app);
    removeAppointmentFromFile(app);
}

private void removeAppointmentFromFile(Appointment app) {
```

```
    String appointmentToDelete = buildAppointmentFileString(app);
    try {
        List<String> lines = readAllLines(APPOINTMENT_FILE_PATH);
        StringBuilder newContent = new StringBuilder();
        StringBuilder currentBlock = new StringBuilder();
        for (String line : lines) {
            currentBlock.append(line).append("\n");
            if (currentBlock.toString().endsWith("\n\n")) {
                if (!currentBlock.toString().equals(appointmentToDelete)) {
                    newContent.append(currentBlock);
                }
                currentBlock = new StringBuilder();
            }
        }
        writeAllText(APPOINTMENT_FILE_PATH, newContent.toString());
    } catch (IOException ex) {
        throw new RuntimeException("Error deleting appointment from file", ex);
    }
}

private String buildAppointmentFileString(Appointment app) {
    return app.getAppId() + "\n" +
            app.getPatientName() + "\n" +
```

```
            app.getDiagnosis() + "\n" +
            app.getDoctorName() + "\n" +
            app.getAppDate() + "\n" +
            app.getAppTime() + "\n\n";
    }

    public void updateAppointment(String appointmentToUpdate, Appointment
updatedAppointment) {
        updateAppointmentInFile(appointmentToUpdate, updatedAppointment);
    }

    private void updateAppointmentInFile(String appointmentToUpdate, Appointment
updatedAppointment) {
        String updatedDetails = buildAppointmentFileString(updatedAppointment);
        try {
            List<String> lines = readAllLines(APPOINTMENT_FILE_PATH);
            StringBuilder newContent = new StringBuilder();
            StringBuilder currentBlock = new StringBuilder();
            for (String line : lines) {
                currentBlock.append(line).append("\n");
                if (currentBlock.toString().endsWith("\n\n")) {
                    if (currentBlock.toString().equals(appointmentToUpdate)) {
                        newContent.append(updatedDetails);
                    } else {
```

```
                newContent.append(currentBlock);
            }
            currentBlock = new StringBuilder();
        }
    }
    writeAllText(APPOINTMENT_FILE_PATH, newContent.toString());
  } catch (IOException ex) {
    throw new RuntimeException("Error updating appointment in file", ex);
  }
}

// Helpers
private List<String> readAllLines(String path) throws IOException {
  List<String> lines = new ArrayList<>();
  try (BufferedReader br = new BufferedReader(new FileReader(path))) {
    String line;
    while ((line = br.readLine()) != null) {
      lines.add(line);
    }
  }
  return lines;
}
```

```
    private void writeAllText(String path, String content) throws IOException {
        try (FileWriter fw = new FileWriter(path)) {
            fw.write(content);
        }
    }
}
```

Why This Is Better?

The refactoring removed common issues like Duplicated Code, Long Method, and Magic Number.

Improvements:

1. Duplicate code moved to helper methods.
2. Long methods broken into smaller ones.
3. Magic numbers/strings replaced with constants.
4. Arrays changed to List for better data handling.
   Error handling improved for reliability.

Start.java

Before (Original Code)

```
import java.lang.*;

import Classes.*;
```

```
import Frames.*;
import Interfaces.*;

public class Start{
	public static void main(String []args){
		Login lg =  new Login();
		lg.setVisible(true);
	/*	Menu me =  new Menu();
		me.setVisible(true);*/
	}
}
```

After (Modified Code)

```
import Frames.Login;

public class Start {
    public static void main(String[] args) {
        createAndShowInitialFrame();
    }

    private static void createAndShowInitialFrame() {
        Login lg = new Login();
```

```
        lg.setVisible(true);
    }
}
```

Why: The refactoring process removed several minor code smells to keep the class clean and maintainable. Unnecessary imports were eliminated to reduce clutter and improve readability. Commented-out or dead code was also removed since it no longer served any purpose and could confuse future developers. Overall, only minimal changes were required because the class was already simple and well-structured, ensuring that the codebase remains clear, concise, and easy to maintain.

Design Pattern

Creational Pattern(FACTORY PATTERN)

Before:

```
Doctor doctor = new Doctor(name, mobile, gender, department, joiningDate, salary, room, bmdc);
```

We replace this.

After:

Create DoctorFactory.java

DoctorFactory.java

```
public class DoctorFactory {


    public Doctor createDoctor(
            String name,
            String mobile,
            String gender,
            String department,
            String joiningDate,
            double salary,
            String roomNo,
            String bmdcReg
    ) {
```

```
        return new Doctor(
                name,
                mobile,
                gender,
                department,
                joiningDate,
                salary,
                roomNo,
                bmdcReg
        );
    }
}
```

Use in AddDoctorUI

Before **AddDoctorUI.java**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;
```

```java
private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;

private JRadioButton maleRB, femaleRB, otherRB;

private JComboBox<String> deptCB;

private JButton addButton, backButton;

private ButtonGroup genderGroup;


public AddDoctorUI(IDoctorService doctorService) {

    super("Add Doctor Form");

    this.doctorService = doctorService;


    setSize(900, 600);

    setLocationRelativeTo(null);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);


    initUI();

}


private void initUI() {

    JPanel panel = new JPanel();

    panel.setLayout(null);

    panel.setBackground(new Color(143,207,225));
```

```
Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN, 18);


// Labels & TextFields

JLabel titleLabel = new JLabel("Add Doctor Form");

titleLabel.setBounds(130, 20, 300, 40);

titleLabel.setFont(new Font("Biome", Font.BOLD, 30));

panel.add(titleLabel);


JLabel nameLabel = new JLabel("Doctor Name:");

nameLabel.setBounds(75, 80, 120, 25);

nameLabel.setFont(labelFont);

panel.add(nameLabel);


nameTF = new JTextField();

nameTF.setBounds(200, 80, 150, 25);

panel.add(nameTF);


JLabel mobileLabel = new JLabel("Mobile Number:");

mobileLabel.setBounds(75, 120, 120, 25);

mobileLabel.setFont(labelFont);

panel.add(mobileLabel);


mobileTF = new JTextField();
```

```
mobileTF.setBounds(200, 120, 150, 25);

panel.add(mobileTF);


JLabel genderLabel = new JLabel("Gender:");

genderLabel.setBounds(75, 160, 100, 25);

genderLabel.setFont(labelFont);

panel.add(genderLabel);


maleRB = new JRadioButton("Male");

maleRB.setBounds(150, 160, 80, 25);

maleRB.setBackground(panel.getBackground());

femaleRB = new JRadioButton("Female");

femaleRB.setBounds(230, 160, 80, 25);

femaleRB.setBackground(panel.getBackground());

otherRB = new JRadioButton("Other");

otherRB.setBounds(310, 160, 80, 25);

otherRB.setBackground(panel.getBackground());


genderGroup = new ButtonGroup();

genderGroup.add(maleRB);

genderGroup.add(femaleRB);

genderGroup.add(otherRB);
```

```
        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);


        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);


        String[] departments = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);

        deptCB.setBounds(200, 200, 150, 25);

        panel.add(deptCB);


        JLabel joiningLabel = new JLabel("Joining Date:");

        joiningLabel.setBounds(75, 240, 120, 25);

        joiningLabel.setFont(labelFont);

        panel.add(joiningLabel);


        joiningTF = new JTextField();

        joiningTF.setBounds(200, 240, 150, 25);

        panel.add(joiningTF);
```

```
JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);
```

```
    bmdcTF = new JTextField();
    bmdcTF.setBounds(200, 360, 150, 25);
    panel.add(bmdcTF);

    // Buttons
    addButton = new JButton("ADD");
    addButton.setBounds(500, 470, 100, 40);
    addButton.setBackground(Color.GREEN);
    addButton.setForeground(Color.WHITE);
    addButton.addActionListener(e -> onAdd());
    panel.add(addButton);

    backButton = new JButton("BACK");
    backButton.setBounds(620, 470, 100, 40);
    backButton.setBackground(Color.RED);
    backButton.setForeground(Color.WHITE);
    backButton.addActionListener(e -> onBack());
    panel.add(backButton);

    this.add(panel);
}
```

```
  private void onAdd() {

      String name = nameTF.getText();

      String mobile = mobileTF.getText();

      String gender = maleRB.isSelected() ? "Male" :

                  femaleRB.isSelected() ? "Female" : "Other";

      String department = (String) deptCB.getSelectedItem();

      String joiningDate = joiningTF.getText();

      double salary = Double.parseDouble(salaryTF.getText());

      String room = roomTF.getText();

      String bmdc = bmdcTF.getText();



              Doctor doctor = new Doctor(name, mobile, gender, department, joiningDate, salary, room,
bmdc);

      doctorService.addDoctor(doctor);



      JOptionPane.showMessageDialog(this, "Doctor added successfully!");

      clearFields();

  }



  private void onBack() {

      this.setVisible(false);

      DrMenu dm = new DrMenu(); // Navigation to main menu

      dm.setVisible(true);
```

```
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

After **AddDoctorUI.java**

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;
```

```java
private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;

private JRadioButton maleRB, femaleRB, otherRB;

private JComboBox<String> deptCB;

private JButton addButton, backButton;

private ButtonGroup genderGroup;


public AddDoctorUI(IDoctorService doctorService) {

    super("Add Doctor Form");

    this.doctorService = doctorService;


    setSize(900, 600);

    setLocationRelativeTo(null);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);


    initUI();

}


private void initUI() {

    JPanel panel = new JPanel();

    panel.setLayout(null);

    panel.setBackground(new Color(143,207,225));
```

```
Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN, 18);


// Labels & TextFields

JLabel titleLabel = new JLabel("Add Doctor Form");

titleLabel.setBounds(130, 20, 300, 40);

titleLabel.setFont(new Font("Biome", Font.BOLD, 30));

panel.add(titleLabel);


JLabel nameLabel = new JLabel("Doctor Name:");

nameLabel.setBounds(75, 80, 120, 25);

nameLabel.setFont(labelFont);

panel.add(nameLabel);


nameTF = new JTextField();

nameTF.setBounds(200, 80, 150, 25);

panel.add(nameTF);


JLabel mobileLabel = new JLabel("Mobile Number:");

mobileLabel.setBounds(75, 120, 120, 25);

mobileLabel.setFont(labelFont);

panel.add(mobileLabel);


mobileTF = new JTextField();
```

```
mobileTF.setBounds(200, 120, 150, 25);

panel.add(mobileTF);


JLabel genderLabel = new JLabel("Gender:");

genderLabel.setBounds(75, 160, 100, 25);

genderLabel.setFont(labelFont);

panel.add(genderLabel);


maleRB = new JRadioButton("Male");

maleRB.setBounds(150, 160, 80, 25);

maleRB.setBackground(panel.getBackground());

femaleRB = new JRadioButton("Female");

femaleRB.setBounds(230, 160, 80, 25);

femaleRB.setBackground(panel.getBackground());

otherRB = new JRadioButton("Other");

otherRB.setBounds(310, 160, 80, 25);

otherRB.setBackground(panel.getBackground());


genderGroup = new ButtonGroup();

genderGroup.add(maleRB);

genderGroup.add(femaleRB);

genderGroup.add(otherRB);
```

```
        panel.add(maleRB);

        panel.add(femaleRB);

        panel.add(otherRB);


        JLabel deptLabel = new JLabel("Department:");

        deptLabel.setBounds(75, 200, 120, 25);

        deptLabel.setFont(labelFont);

        panel.add(deptLabel);


        String[] departments = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};

        deptCB = new JComboBox<>(departments);

        deptCB.setBounds(200, 200, 150, 25);

        panel.add(deptCB);


        JLabel joiningLabel = new JLabel("Joining Date:");

        joiningLabel.setBounds(75, 240, 120, 25);

        joiningLabel.setFont(labelFont);

        panel.add(joiningLabel);


        joiningTF = new JTextField();

        joiningTF.setBounds(200, 240, 150, 25);

        panel.add(joiningTF);
```

```
JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);
```

```
    bmdcTF = new JTextField();
    bmdcTF.setBounds(200, 360, 150, 25);
    panel.add(bmdcTF);

    // Buttons
    addButton = new JButton("ADD");
    addButton.setBounds(500, 470, 100, 40);
    addButton.setBackground(Color.GREEN);
    addButton.setForeground(Color.WHITE);
    addButton.addActionListener(e -> onAdd());
    panel.add(addButton);

    backButton = new JButton("BACK");
    backButton.setBounds(620, 470, 100, 40);
    backButton.setBackground(Color.RED);
    backButton.setForeground(Color.WHITE);
    backButton.addActionListener(e -> onBack());
    panel.add(backButton);

    this.add(panel);
}
```

```
private void onAdd() {

    String name = nameTF.getText();

    String mobile = mobileTF.getText();

    String gender = maleRB.isSelected() ? "Male" :

                femaleRB.isSelected() ? "Female" : "Other";

    String department = (String) deptCB.getSelectedItem();

    String joiningDate = joiningTF.getText();

    double salary = Double.parseDouble(salaryTF.getText());

    String room = roomTF.getText();

    String bmdc = bmdcTF.getText();


        DoctorFactory factory = new DoctorFactory();


    Doctor doctor = factory.createDoctor(

        name, mobile, gender, department,

        joiningDate, salary, room, bmdc

    );

    doctorService.addDoctor(doctor);


    JOptionPane.showMessageDialog(this, "Doctor added successfully!");

    clearFields();

}
```

```java
    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu(); // Navigation to main menu
        dm.setVisible(true);
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

STRUCTURAL PATTERN (ADAPTER)

Create DoctorMenuAdapter.java

```java
public class DoctorMenuAdapter {



    private IDoctorService doctorService;



    public DoctorMenuAdapter() {

        doctorService = new DoctorService();

    }



    public void openAddDoctorUI() {

        AddDoctorUI ui = new AddDoctorUI(doctorService);

        ui.setVisible(true);

    }

}
```

Before DrMenu.java

```java
import javax.swing.*;
import java.awt.*;

You, last month | 1 author (You)
public class DrMenu extends JFrame {

    public DrMenu() {
        super(title: "Doctor Menu");
        setSize(width: 400, height: 300);
        setLocationRelativeTo(c: null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel label = new JLabel(text: "This is the Doctor Menu", SwingConstants.CENTER);
        label.setFont(new Font(name: "Arial", Font.BOLD, size: 18));
        add(label);
        JButton addDoctorButton = new JButton(text: "ADD DOCTOR");
        addDoctorButton.setBounds(x: 75, y: 380, width: 200, height: 40); // adjust X, Y to fit
        addDoctorButton.setBackground(Color.BLACK);
        addDoctorButton.setForeground(Color.WHITE);
        addDoctorButton.setFont(new Font(name: "Cascadia Code SemiBold", Font.PLAIN, size: 16))
        addDoctorButton.addActionListener(e -> {
        IDoctorService doctorService = new DoctorService(); // Use the service
        AddDoctorUI addDoctorUI = new AddDoctorUI(doctorService);
        addDoctorUI.setVisible(b: true);        You, last month • SOLID
        this.setVisible(b: false); // Hide menu while AddDoctor form is open
});


    }
}
```

After DrMenu.java

```java
import javax.swing.*;
import java.awt.*;

You, last month | 1 author (You)
public class DrMenu extends JFrame {

    public DrMenu() {
        super(title: "Doctor Menu");
        setSize(width: 400, height: 300);
        setLocationRelativeTo(c: null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel label = new JLabel(text: "This is the Doctor Menu", SwingConstants.CENTER);
        label.setFont(new Font(name: "Arial", Font.BOLD, size: 18));
        add(label);
        JButton addDoctorButton = new JButton(text: "ADD DOCTOR");
        addDoctorButton.setBounds(x: 75, y: 380, width: 200, height: 40); // adjust X, Y to fit
        addDoctorButton.setBackground(Color.BLACK);
        addDoctorButton.setForeground(Color.WHITE);
        addDoctorButton.setFont(new Font(name: "Cascadia Code SemiBold", Font.PLAIN, size: 16))
        DoctorMenuAdapter adapter = new DoctorMenuAdapter();

        addDoctorButton.addActionListener(e -> {
            adapter.openAddDoctorUI();
            this.setVisible(b: false);      You, last month • Uncommitted changes
        });


    }
}
```

Adapter Pattern was used to integrate the legacy DrMenu with the refactored AddDoctorUI module without modifying existing menu logic.

BEHAVIORAL PATTERN(OBSERVER)

The hospital system needs notifications.
This is the BEST behavioral pattern for the system.

Create DoctorObserver.java

```java
public interface DoctorObserver {
    void update(String message);
}
```

Create DoctorNotification.java

```java
import javax.swing.*;

public class DoctorNotification implements DoctorObserver {

    @Override
    public void update(String message) {
        JOptionPane.showMessageDialog(null, message);
    }
}
```

Before DoctorService.java

```java
...

import java.util.ArrayList;
import java.util.List;

...
public class DoctorService implements IDoctorService {
    private List<Doctor> doctors = new ArrayList<>();

    @Override
    public void addDoctor(Doctor doctor) {
        doctors.add(doctor);

    }

    public List<Doctor> getDoctors() {
        return doctors;
    }
}
```

After DoctorService.java

```
import java.util.*;

public class DoctorService implements IDoctorService {

    private List<Doctor> doctors = new ArrayList<>();
    private List<DoctorObserver> observers = new ArrayList<>();

    public void addObserver(DoctorObserver observer) {
        observers.add(observer);
    }

    private void notifyObservers(String message) {
        for (DoctorObserver observer : observers) {
            observer.update(message);
        }
    }

    @Override
    public void addDoctor(Doctor doctor) {
        doctors.add(doctor);
        notifyObservers("Doctor added successfully");
    }
}
```

Observer Pattern was applied to notify UI components whenever a new doctor is added to the system.

The change After adding a behaviour pattern.

AddDoctorUI.java

```java
public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;

    private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;
    private JRadioButton maleRB, femaleRB, otherRB;
    private JComboBox<String> deptCB;
    private JButton addButton, backButton;
    private ButtonGroup genderGroup;

    public AddDoctorUI(IDoctorService doctorService) {
        super(title: "Add Doctor Form");
        this.doctorService = doctorService;

        // Behavioral Design Pattern: Observer
        DoctorService service = new DoctorService();
        service.addObserver(new DoctorNotification());        You, 27 seconds ago

        setSize(width: 900, height: 600);
        setLocationRelativeTo(c: null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        initUI();
    }
```

Whole code of **AddDoctorUI.java**

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class AddDoctorUI extends JFrame {

    private IDoctorService doctorService;

    private JTextField nameTF, mobileTF, joiningTF, salaryTF, roomTF, bmdcTF;
    private JRadioButton maleRB, femaleRB, otherRB;
    private JComboBox<String> deptCB;
    private JButton addButton, backButton;
    private ButtonGroup genderGroup;
```

```java
public AddDoctorUI(IDoctorService doctorService) {
    super("Add Doctor Form");
    this.doctorService = doctorService;

    // Behavioral Design Pattern: Observer
    DoctorService service = new DoctorService();
    service.addObserver(new DoctorNotification());

    setSize(900, 600);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    initUI();
}

private void initUI() {
    JPanel panel = new JPanel();
    panel.setLayout(null);
    panel.setBackground(new Color(143,207,225));

    Font labelFont = new Font("Cascadia Code SemiBold", Font.PLAIN, 18);

    JLabel titleLabel = new JLabel("Add Doctor Form");
    titleLabel.setBounds(130, 20, 300, 40);
    titleLabel.setFont(new Font("Biome", Font.BOLD, 30));
    panel.add(titleLabel);

    JLabel nameLabel = new JLabel("Doctor Name:");
    nameLabel.setBounds(75, 80, 120, 25);
    nameLabel.setFont(labelFont);
    panel.add(nameLabel);

    nameTF = new JTextField();
    nameTF.setBounds(200, 80, 150, 25);
    panel.add(nameTF);

    JLabel mobileLabel = new JLabel("Mobile Number:");
    mobileLabel.setBounds(75, 120, 120, 25);
    mobileLabel.setFont(labelFont);
```

```
    panel.add(mobileLabel);

    mobileTF = new JTextField();
    mobileTF.setBounds(200, 120, 150, 25);
    panel.add(mobileTF);

    JLabel genderLabel = new JLabel("Gender:");
    genderLabel.setBounds(75, 160, 100, 25);
    genderLabel.setFont(labelFont);
    panel.add(genderLabel);

    maleRB = new JRadioButton("Male");
    maleRB.setBounds(150, 160, 80, 25);
    maleRB.setBackground(panel.getBackground());

    femaleRB = new JRadioButton("Female");
    femaleRB.setBounds(230, 160, 80, 25);
    femaleRB.setBackground(panel.getBackground());

    otherRB = new JRadioButton("Other");
    otherRB.setBounds(310, 160, 80, 25);
    otherRB.setBackground(panel.getBackground());

    genderGroup = new ButtonGroup();
    genderGroup.add(maleRB);
    genderGroup.add(femaleRB);
    genderGroup.add(otherRB);

    panel.add(maleRB);
    panel.add(femaleRB);
    panel.add(otherRB);

    JLabel deptLabel = new JLabel("Department:");
    deptLabel.setBounds(75, 200, 120, 25);
    deptLabel.setFont(labelFont);
    panel.add(deptLabel);

    String[] departments = {"CARDIOLOGY", "NEUROSURGERY", "ONCOLOGIST",
"ORTHOPEDICS"};
```

```
deptCB = new JComboBox<>(departments);
deptCB.setBounds(200, 200, 150, 25);
panel.add(deptCB);

JLabel joiningLabel = new JLabel("Joining Date:");
joiningLabel.setBounds(75, 240, 120, 25);
joiningLabel.setFont(labelFont);
panel.add(joiningLabel);

joiningTF = new JTextField();
joiningTF.setBounds(200, 240, 150, 25);
panel.add(joiningTF);

JLabel salaryLabel = new JLabel("Salary:");
salaryLabel.setBounds(75, 280, 120, 25);
salaryLabel.setFont(labelFont);
panel.add(salaryLabel);

salaryTF = new JTextField();
salaryTF.setBounds(200, 280, 150, 25);
panel.add(salaryTF);

JLabel roomLabel = new JLabel("Room No:");
roomLabel.setBounds(75, 320, 120, 25);
roomLabel.setFont(labelFont);
panel.add(roomLabel);

roomTF = new JTextField();
roomTF.setBounds(200, 320, 150, 25);
panel.add(roomTF);

JLabel bmdcLabel = new JLabel("BM&DC REG:");
bmdcLabel.setBounds(75, 360, 120, 25);
bmdcLabel.setFont(labelFont);
panel.add(bmdcLabel);

bmdcTF = new JTextField();
bmdcTF.setBounds(200, 360, 150, 25);
panel.add(bmdcTF);
```

```
    addButton = new JButton("ADD");
    addButton.setBounds(500, 470, 100, 40);
    addButton.setBackground(Color.GREEN);
    addButton.setForeground(Color.WHITE);
    addButton.addActionListener(e -> onAdd());
    panel.add(addButton);

    backButton = new JButton("BACK");
    backButton.setBounds(620, 470, 100, 40);
    backButton.setBackground(Color.RED);
    backButton.setForeground(Color.WHITE);
    backButton.addActionListener(e -> onBack());
    panel.add(backButton);

    this.add(panel);
}

private void onAdd() {
    String name = nameTF.getText();
    String mobile = mobileTF.getText();
    String gender = maleRB.isSelected() ? "Male" :
                femaleRB.isSelected() ? "Female" : "Other";
    String department = (String) deptCB.getSelectedItem();
    String joiningDate = joiningTF.getText();
    double salary = Double.parseDouble(salaryTF.getText());
    String room = roomTF.getText();
    String bmdc = bmdcTF.getText();

    DoctorFactory factory = new DoctorFactory();

    Doctor doctor = factory.createDoctor(
        name, mobile, gender, department,
        joiningDate, salary, room, bmdc
    );

    doctorService.addDoctor(doctor);

    JOptionPane.showMessageDialog(this, "Doctor added successfully!");
```

```
        clearFields();
    }

    private void onBack() {
        this.setVisible(false);
        DrMenu dm = new DrMenu();
        dm.setVisible(true);
    }

    private void clearFields() {
        nameTF.setText("");
        mobileTF.setText("");
        genderGroup.clearSelection();
        deptCB.setSelectedIndex(0);
        joiningTF.setText("");
        salaryTF.setText("");
        roomTF.setText("");
        bmdcTF.setText("");
    }
}
```

Creational Pattern (SINGLETON PATTERN + BUILDER PATTERN)

Before: This created multiple objects in memory, leading to data inconsistency and repeated file loading. Patient and Doctor classes used only traditional constructors.

PList()

```
package Classes;

import Interfaces.*;

import java.io.*;

import java.util.Scanner;

public class PList extends AbstractFileRepository<Patient> implements IPList {

    private static final String PATIENT_FILE_PATH = "Files/PatientList.txt";

    private static PList instance;

    private PList() {

        super(PATIENT_FILE_PATH);

    }

    public static synchronized PList getInstance() {

        if (instance == null) {

            instance = new PList();

        }

        return instance;

    }



    @Override
```

```
  protected void loadFromFile() {

      File file = new File(filePath);

      if (!file.exists()) return;

      try (Scanner sc = new Scanner(file)) {

          while (sc.hasNext()) {

              String id = sc.nextLine();

              String name = sc.nextLine();

              String mobileNo = sc.nextLine();

              String gender = sc.nextLine();

              String age = sc.nextLine();

              String address = sc.nextLine();

              String appointedDoctor = sc.nextLine();

              String room = sc.nextLine();

              String diagnosis = sc.nextLine();

              String deposit = sc.nextLine();

              if (sc.hasNextLine()) sc.nextLine();



              Patient p = new Patient(id, name, mobileNo, gender, age, address, appointedDoctor, room, diagnosis,
deposit);

              items.add(p);

          }

      } catch (Exception ignored) {}

  }
```

```java
@Override
protected String buildBlock(Patient p) {
    return p.getId() + "\n" + p.getName() + "\n" + p.getMobileNo() + "\n" +
            p.getGender() + "\n" + p.getAge() + "\n" + p.getAddress() + "\n" +
            p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
            p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
}
@Override
protected boolean blocksMatch(Patient p, String block) {
    return buildBlock(p).equals(block);
}
@Override
public void addPatient(Patient p) {
    add(p);
}


@Override
public int searchPatient(String id) {
    return searchByKey(id, Patient::getId);
}
@Override
public Patient getPatient(int index) {
```

```
        return get(index);

    }

    @Override

    public void deletePatient(Patient p) {

        delete(p);

    }

    @Override

    public void updatePatient(String patientToUpdate, Patient updatedPatient) {

        update(patientToUpdate, updatedPatient);

    }

}
```

RegDoctorList()

```
package Classes;

import java.io.*;
import java.util.Scanner;

public class RegDoctorList extends AbstractFileRepository<Doctor> {
    private static final String DOCTOR_FILE_PATH = "Files/DoctorList.txt";
    private static RegDoctorList instance;

    // Private constructor - only calls super once
    private RegDoctorList() {
        super(DOCTOR_FILE_PATH);
    }

    public static synchronized RegDoctorList getInstance() {
        if (instance == null) {
            instance = new RegDoctorList();
```

```java
    }
    return instance;
}

@Override
protected void loadFromFile() {
    File file = new File(filePath);
    if (!file.exists()) return;

    try (Scanner sc = new Scanner(file)) {
        while (sc.hasNext()) {
            String id = sc.nextLine();
            String name = sc.nextLine();
            String mobileNo = sc.nextLine();
            String gender = sc.nextLine();
            String department = sc.nextLine();
            String joiningDate = sc.nextLine();
            String bmdcReg = sc.nextLine();
            String password = sc.nextLine();
            if (sc.hasNextLine()) sc.nextLine();

            Doctor d = new Doctor(id, name, mobileNo, gender, department, joiningDate, bmdcReg, password);
            items.add(d);
        }
    } catch (Exception ignored) {}
}

@Override
protected String buildBlock(Doctor d) {
    return d.getId() + "\n" + d.getName() + "\n" + d.getMobileNo() + "\n" + d.getGender() + "\n" +
            d.getDepartment() + "\n" + d.getJoiningDate() + "\n" + d.getBmdcReg() + "\n" +
            d.getPassword() + "\n\n";
}

@Override
protected boolean blocksMatch(Doctor d, String block) {
    return buildBlock(d).equals(block);
}
```

```java
    // Original helper methods
    public int doctorExists(String name) {
        for (int i = 0; i < items.size(); i++) {
            if (items.get(i).getName().equals(name)) {
                return i;
            }
        }
        return -1;
    }

    public Doctor checkPassword(int index, String pass) {
        if (index < 0 || index >= items.size()) return null;
        Doctor d = items.get(index);
        return d.getPassword().equals(pass) ? d : null;
    }

    public Doctor getDoctor(int index) {
        return get(index);
    }

    public String[] getDoctorNames() {
        String[] names = new String[items.size()];
        for (int i = 0; i < items.size(); i++) {
            names[i] = "Dr. " + items.get(i).getName();
        }
        return names;
    }

    public void addDoctor(Doctor d) {
        add(d);
    }
}
```

Structural Pattern (FACADE PATTERN)

Before: GUI classes directly accessed multiple repositories **(PList, RegDoctorList, AppointmentList, PrescriptionList)**. This resulted in tight coupling between the UI layer and the data layer.

After: We introduced **HospitalFacade.java** as a single unified interface for all hospital operations. All list managers are now accessed through this facade.

New File: HospitalFacade.java

```
package Classes;

public class HospitalFacade {
    private final PList patientManager = PList.getInstance();
    private final RegDoctorList doctorManager = RegDoctorList.getInstance();
    private final AppointmentList appointmentManager = AppointmentList.getInstance();
    private final PrescriptionList prescriptionManager = PrescriptionList.getInstance();

    // Patient operations
    public void addPatient(Patient p) {
        patientManager.addPatient(p);
    }

    public Patient findPatient(String id) {
        int idx = patientManager.searchPatient(id);
        return patientManager.getPatient(idx);
    }

    public void deletePatient(Patient p) {
        patientManager.deletePatient(p);
    }

    // Doctor operations
    public void addDoctor(Doctor d) {
        doctorManager.addDoctor(d);
    }

    public String[] getDoctorNames() {
        return doctorManager.getDoctorNames();
```

```
    }

    public Doctor findDoctorByName(String name) {
        int idx = doctorManager.doctorExists(name);
        return (idx != -1) ? doctorManager.getDoctor(idx) : null;
    }

    // Appointment operations
    public void bookAppointment(Appointment app) {
        appointmentManager.addAppointment(app);
    }

    public Appointment findAppointment(String appId) {
        int idx = appointmentManager.searchAppointment(appId);
        return appointmentManager.getAppointment(idx);
    }

    public void deleteAppointment(Appointment app) {
        appointmentManager.deleteAppointment(app);
    }

    // Prescription operations
    public void issuePrescription(Prescription p) {
        prescriptionManager.addPrescription(p);
    }

    public Prescription findPrescription(String patientId) {
        int idx = prescriptionManager.searchPrescription(patientId);
        return prescriptionManager.getPrescription(idx);
    }
}
```

Recommendation: All GUI classes should now use HospitalFacade instead of calling repositories directly (partial integration done; full integration recommended in next iteration).

Benefit: Reduced coupling and simplified interaction with the subsystem.

Behavioral Pattern (TEMPLATE METHOD PATTERN)

Before:

Significant code duplication existed in file I/O operations (loading, building blocks, deleting, and updating) across PList.java, AppointmentList.java, and RegDoctorList.java. Each class repeated similar logic for reading/writing files.

After:

We created **AbstractFileRepository.java** and applied the Template Method Pattern. Common file operations are defined in the abstract parent class, while subclasses only implement specific steps.

New File: AbstractFileRepository.java

```
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public abstract class AbstractFileRepository<T> {
    protected final String filePath;
    protected final List<T> items = new ArrayList<>();

    protected AbstractFileRepository(String filePath) {
        this.filePath = filePath;
    }

    protected abstract void loadFromFile();
    protected abstract String buildBlock(T item);
    protected abstract boolean blocksMatch(T item, String block);

    public void add(T item) {
        items.add(item);
        appendToFile(item);
    }

    private void appendToFile(T item) {
```

```
    try (FileWriter fw = new FileWriter(filePath, true)) {
        fw.write(buildBlock(item));
    } catch (IOException ex) {
        throw new RuntimeException("Error saving to file", ex);
    }
}

public void delete(T item) {
    items.remove(item);
    rewriteFileExcluding(buildBlock(item));
}

public void update(String oldBlock, T updatedItem) {
    rewriteFileReplacing(oldBlock, buildBlock(updatedItem));
}

private void rewriteFileExcluding(String blockToRemove) {
    rewriteFile(blockToRemove, null);
}

private void rewriteFileReplacing(String oldBlock, String newBlock) {
    rewriteFile(oldBlock, newBlock);
}

private void rewriteFile(String targetBlock, String replacement) {
    try {
        List<String> lines = readAllLines();
        StringBuilder newContent = new StringBuilder();
        StringBuilder current = new StringBuilder();

        for (String line : lines) {
            current.append(line).append("\n");
            if (current.toString().endsWith("\n\n")) {
                String block = current.toString();
                if (block.equals(targetBlock)) {
                    if (replacement != null) newContent.append(replacement);
                } else {
                    newContent.append(block);
                }
```

```
                current = new StringBuilder();
            }
        }
        try (FileWriter fw = new FileWriter(filePath)) {
            fw.write(newContent.toString());
        }
    } catch (IOException ex) {
        throw new RuntimeException("Error rewriting file", ex);
    }
}

protected List<String> readAllLines() throws IOException {
    List<String> lines = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(filePath))) {
        String line;
        while ((line = br.readLine()) != null) {
            lines.add(line);
        }
    }
    return lines;
}

public int searchByKey(String key, java.util.function.Function<T, String> keyExtractor) {
    for (int i = 0; i < items.size(); i++) {
        if (keyExtractor.apply(items.get(i)).equals(key)) {
            return i;
        }
    }
    return -1;
}

public T get(int index) {
    if (index < 0 || index >= items.size()) return null;
    return items.get(index);
}

// NEW PUBLIC METHOD - This fixes the "items has protected access" error
public int getAllItemsSize() {
    return items.size();
```

```
    }
}
```

PList.java (After):

```
package Classes;

import Interfaces.*;
import java.io.*;
import java.util.Scanner;

public class PList extends AbstractFileRepository<Patient> implements IPList {
    private static final String PATIENT_FILE_PATH = "Files/PatientList.txt";
    private static PList instance;

    private PList() {
        super(PATIENT_FILE_PATH);
    }

    public static synchronized PList getInstance() {
        if (instance == null) {
            instance = new PList();
        }
        return instance;
    }

    @Override
    protected void loadFromFile() {
        File file = new File(filePath);
        if (!file.exists()) return;

        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
```

```
            String id = sc.nextLine();
            String name = sc.nextLine();
            String mobileNo = sc.nextLine();
            String gender = sc.nextLine();
            String age = sc.nextLine();
            String address = sc.nextLine();
            String appointedDoctor = sc.nextLine();
            String room = sc.nextLine();
            String diagnosis = sc.nextLine();
            String deposit = sc.nextLine();
            if (sc.hasNextLine()) sc.nextLine();

            Patient p = new Patient(id, name, mobileNo, gender, age, address, appointedDoctor, room, diagnosis,
deposit);
            items.add(p);
        }
    } catch (Exception ignored) {}
  }

  @Override
  protected String buildBlock(Patient p) {
    return p.getId() + "\n" + p.getName() + "\n" + p.getMobileNo() + "\n" +
          p.getGender() + "\n" + p.getAge() + "\n" + p.getAddress() + "\n" +
          p.getAppointedDoctor() + "\n" + p.getRoom() + "\n" +
          p.getDiagnosis() + "\n" + p.getDeposit() + "\n\n";
  }

  @Override
  protected boolean blocksMatch(Patient p, String block) {
    return buildBlock(p).equals(block);
  }

  @Override
  public void addPatient(Patient p) {
    add(p);
  }

  @Override
  public int searchPatient(String id) {
```

```
        return searchByKey(id, Patient::getId);
    }

    @Override
    public Patient getPatient(int index) {
        return get(index);
    }

    @Override
    public void deletePatient(Patient p) {
        delete(p);
    }

    @Override
    public void updatePatient(String patientToUpdate, Patient updatedPatient) {
        update(patientToUpdate, updatedPatient);
    }
}
```

Appointment.java

Old Code:

```
package Classes;

public class Appointment {
    private final String appID;
    private final String patientName;
    private final String diagnosis;
    private final String doctorName;
    private final String appDate;
    private final String appTime;

    public Appointment(String appID, String patientName, String diagnosis, String doctorName,
String appDate, String appTime) {
        this.appID = appID;
        this.patientName = patientName;
        this.diagnosis = diagnosis;
        this.doctorName = doctorName;
        this.appDate = appDate;
        this.appTime = appTime;
    }

    public String getAppId() {
        return appID;
    }

    public String getPatientName() {
        return patientName;
    }

    public String getDiagnosis() {
        return diagnosis;
    }

    public String getDoctorName() {
        return doctorName;
    }

    public String getAppDate() {
        return appDate;
    }
```

```
    public String getAppTime() {
        return appTime;
    }
}
```

Issues Identified

1. No input validation: null or empty strings can silently corrupt data
2. Primitive Obsession: appDate and appTime as raw Strings have no format guarantee
3. Construction complexity: 6 parameters in a constructor is error-prone (easy to swap arguments)
4. No **toString()**: makes debugging/logging unnecessarily hard

Refactoring Applied

Pattern used: Builder Pattern Reason: The constructor has 6 parameters of the same type (String). This is a classic use case — callers can easily pass arguments in the wrong order with no compile-time error. The Builder Pattern solves this with a readable, safe construction API.

Refactored Code:

```
package Classes;

import java.time.LocalDate;
import java.time.LocalTime;
import java.time.format.DateTimeParseException;
import java.util.Objects;

public class Appointment {

    private final String appID;
    private final String patientName;
    private final String diagnosis;
    private final String doctorName;
    private final LocalDate appDate;
    private final LocalTime appTime;

    // Private constructor — only Builder can instantiate
    private Appointment(Builder builder) {
        this.appID       = builder.appID;
        this.patientName = builder.patientName;
        this.diagnosis   = builder.diagnosis;
        this.doctorName  = builder.doctorName;
```

```
    this.appDate     = builder.appDate;
    this.appTime     = builder.appTime;
}

// ── Getters ──────────────────────────────────────────────────────────────
public String getAppId()       { return appID; }
public String getPatientName()  { return patientName; }
public String getDiagnosis()    { return diagnosis; }
public String getDoctorName()   { return doctorName; }
public LocalDate getAppDate()   { return appDate; }
public LocalTime getAppTime()   { return appTime; }

@Override
public String toString() {
    return String.format(
        "Appointment{id='%s', patient='%s', doctor='%s', diagnosis='%s', date=%s, time=%s}",
        appID, patientName, doctorName, diagnosis, appDate, appTime
    );
}

// ── Builder ──────────────────────────────────────────────────────────────
public static class Builder {

    // Required fields
    private final String appID;
    private final String patientName;

    // Optional / secondary fields
    private String diagnosis = "Not specified";
    private String doctorName;
    private LocalDate appDate;
    private LocalTime appTime;

    public Builder(String appID, String patientName) {
        this.appID       = requireNonBlank(appID,       "Appointment ID");
        this.patientName = requireNonBlank(patientName, "Patient name");
    }

    public Builder diagnosis(String diagnosis) {
        this.diagnosis = requireNonBlank(diagnosis, "Diagnosis");
        return this;
    }

    public Builder doctorName(String doctorName) {
```

```
    this.doctorName = requireNonBlank(doctorName, "Doctor name");
    return this;
}

/**
 * Accepts ISO-8601 format: yyyy-MM-dd (e.g. "2025-04-07")
 */
public Builder appDate(String date) {
    try {
        this.appDate = LocalDate.parse(date);
    } catch (DateTimeParseException e) {
        throw new IllegalArgumentException(
            "Invalid date format. Expected yyyy-MM-dd, got: " + date
        );
    }
    return this;
}

/**
 * Accepts ISO-8601 format: HH:mm (e.g. "09:30")
 */
public Builder appTime(String time) {
    try {
        this.appTime = LocalTime.parse(time);
    } catch (DateTimeParseException e) {
        throw new IllegalArgumentException(
            "Invalid time format. Expected HH:mm, got: " + time
        );
    }
    return this;
}

public Appointment build() {
    Objects.requireNonNull(doctorName, "Doctor name must be set before building.");
    Objects.requireNonNull(appDate,    "Appointment date must be set before building.");
    Objects.requireNonNull(appTime,    "Appointment time must be set before building.");
    return new Appointment(this);
}

// ── Validation helper ──────────────────────────────────────────────
private static String requireNonBlank(String value, String fieldName) {
    if (value == null || value.isBlank()) {
        throw new IllegalArgumentException(fieldName + " must not be null or blank.");
    }
```

```
            return value;
        }
    }
}
```

AppointmentList.java

Old Code:

```
package Classes;

import java.io.*;
import java.util.Scanner;

public class AppointmentList extends AbstractFileRepository<Appointment> {
    private static final String APPOINTMENT_FILE_PATH = "Files/AppointmentList.txt";
    private static AppointmentList instance;

    private AppointmentList() {
        super(APPOINTMENT_FILE_PATH);
    }

    public static synchronized AppointmentList getInstance() {
    if (instance == null) {
        instance = new AppointmentList();
    }
    return instance;
}

    @Override
    protected void loadFromFile() {
        File file = new File(filePath);
        if (!file.exists()) return;

        try (Scanner sc = new Scanner(file)) {
            while (sc.hasNext()) {
                String appID = sc.nextLine();
                String patientName = sc.nextLine();
                String diagnosis = sc.nextLine();
                String doctorName = sc.nextLine();
                String appDate = sc.nextLine();
                String appTime = sc.nextLine();
```

```
            if (sc.hasNextLine()) sc.nextLine();

            Appointment app = new Appointment(appID, patientName, diagnosis, doctorName,
appDate, appTime);
            items.add(app);
        }
    } catch (Exception ignored) {}
  }

  @Override
  protected String buildBlock(Appointment app) {
    return app.getAppId() + "\n" +
          app.getPatientName() + "\n" +
          app.getDiagnosis() + "\n" +
          app.getDoctorName() + "\n" +
          app.getAppDate() + "\n" +
          app.getAppTime() + "\n\n";
  }

  @Override
  protected boolean blocksMatch(Appointment app, String block) {
    return buildBlock(app).equals(block);
  }

  public void addAppointment(Appointment app) {
    add(app);
  }

  public int searchAppointment(String appId) {
    return searchByKey(appId, Appointment::getAppId);
  }

  public Appointment getAppointment(int index) {
    return get(index);
  }

  public void deleteAppointment(Appointment app) {
    delete(app);
  }

  public void updateAppointment(String appointmentToUpdate, Appointment
updatedAppointment) {
    update(appointmentToUpdate, updatedAppointment);
  }
```

```
}
```

Issues Identified

1. Silent exception swallowing: catch (Exception ignored) {} is the most dangerous line here. A corrupted file, missing line, or I/O error will fail completely silently, leaving items partially loaded with no way to diagnose it.

2. Fragile file parsing: The loadFromFile() reads lines positionally with no validation. If any record is malformed (missing a line, extra blank line), every subsequent record loads with shifted/wrong data. This will corrupt silently due to issue #1.

3. Appointment construction bypasses the Builder: Now that Appointment uses a Builder with validation, constructing it directly with 6 raw strings here breaks that contract entirely.

4. blocksMatch is inefficient: It rebuilds the full block string just to do equality comparison. The intent is just to check if a block belongs to a given appointment, matching on the unique ID is the correct and cheaper approach.

Refactored Code:

```
package Classes;

import java.io.*;
import java.util.Scanner;
import java.util.logging.Level;
import java.util.logging.Logger;

public class AppointmentList extends AbstractFileRepository<Appointment> {

    private static final String APPOINTMENT_FILE_PATH = "Files/AppointmentList.txt";
    private static final Logger LOGGER = Logger.getLogger(AppointmentList.class.getName());

    private static volatile AppointmentList instance;

    private AppointmentList() {
        super(APPOINTMENT_FILE_PATH);
    }

    // ── Singleton
    // volatile + double-checked locking: avoids lock acquisition cost on every
    // call once the instance is initialized, while remaining thread-safe.
```

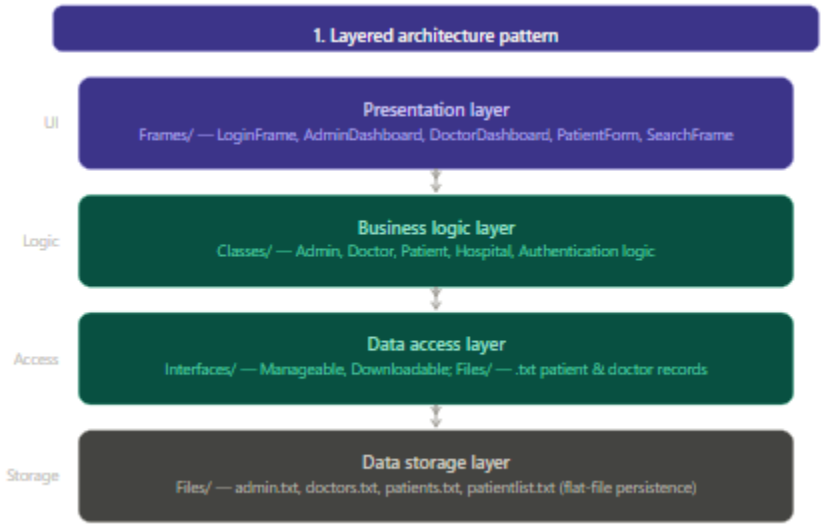
```java
public static AppointmentList getInstance() {
    if (instance == null) {
        synchronized (AppointmentList.class) {
            if (instance == null) {
                instance = new AppointmentList();
            }
        }
    }
    return instance;
}

// ── File Loading
@Override
protected void loadFromFile() {
    File file = new File(filePath);
    if (!file.exists()) return;

    try (Scanner sc = new Scanner(file)) {
        while (sc.hasNextLine()) {
            Appointment app = parseNextAppointment(sc);
            if (app != null) {
                items.add(app);
            }
        }
    } catch (IOException e) {
        // Catch specific exception, not all of Exception.
        // Fail loudly — silent swallowing hides data corruption.
        LOGGER.log(Level.SEVERE, "Failed to load appointments from file: " + filePath, e);
    }
}

/**
 * Reads exactly 6 data lines + 1 blank separator from the scanner.
 * Returns null and logs a warning if the block is malformed,
 * so one bad record doesn't corrupt all subsequent records.
 */
private Appointment parseNextAppointment(Scanner sc) {
    try {
        String appID       = readLine(sc, "appID");
        String patientName = readLine(sc, "patientName");
        String diagnosis   = readLine(sc, "diagnosis");
        String doctorName  = readLine(sc, "doctorName");
        String appDate     = readLine(sc, "appDate");
        String appTime     = readLine(sc, "appTime");
```

```
        if (sc.hasNextLine()) sc.nextLine(); // consume blank separator

        // Uses the Builder — validation is enforced at construction
        return new Appointment.Builder(appID, patientName)
                .diagnosis(diagnosis)
                .doctorName(doctorName)
                .appDate(appDate)
                .appTime(appTime)
                .build();

    } catch (IllegalArgumentException | IllegalStateException e) {
        // One malformed record is skipped, not the entire file
        LOGGER.log(Level.WARNING, "Skipping malformed appointment record: " +
e.getMessage());
        return null;
    }
}

/**
 * Reads the next line, throwing clearly if the file ends unexpectedly.
 * Replaces silent positional reads that corrupt on missing lines.
 */
private String readLine(Scanner sc, String fieldName) {
    if (!sc.hasNextLine()) {
        throw new IllegalStateException(
            "Unexpected end of file while reading field: " + fieldName
        );
    }
    return sc.nextLine().trim();
}

@Override
protected String buildBlock(Appointment app) {
    return app.getAppId()       + "\n" +
           app.getPatientName() + "\n" +
           app.getDiagnosis()   + "\n" +
           app.getDoctorName()  + "\n" +
           app.getAppDate()     + "\n" +
           app.getAppTime()     + "\n\n";
}


// Old: rebuilt the entire block string just for equality — fragile and wasteful.
```

```
    // New: matches on the unique appID — correct by design, O(1) string compare.
    @Override
    protected boolean blocksMatch(Appointment app, String block) {
        return block.startsWith(app.getAppId());
    }

    // —— Public API
    public void addAppointment(Appointment app)                    { add(app); }
    public Appointment getAppointment(int index)                   { return get(index); }
    public void deleteAppointment(Appointment app)                 { delete(app); }
    public int searchAppointment(String appId)                     { return searchByKey(appId,
Appointment::getAppId); }
    public void updateAppointment(String appId, Appointment updatedApp)      { update(appId,
updatedApp); }
}
```

Architecture:

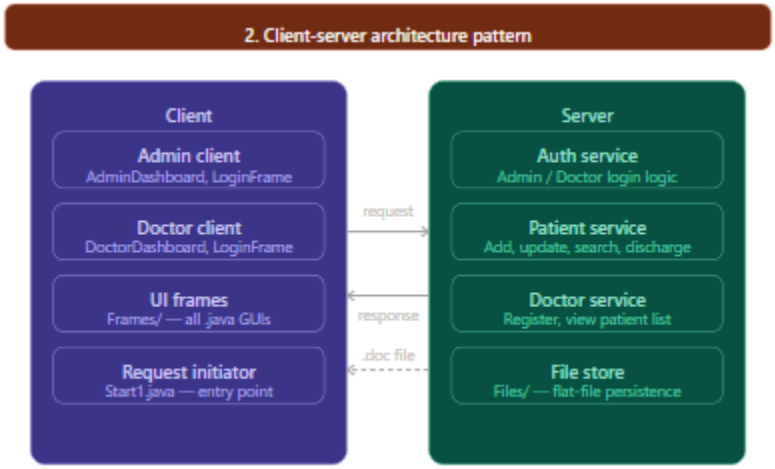
Layered Architecture Pattern:



The **Layered Architecture** organizes the system into a vertical stack where each layer has a specific responsibility and only communicates with the layer directly below it.

- **Presentation Layer:** The "front end" of the application. It contains the Java Swing frames (Login, Dashboards, Search) that the user interacts with.
- **Business Logic Layer:** The core of the system. This layer handles the rules of the hospital, such as how an Admin or Doctor class processes data.
- **Data Access Layer:** Acts as a bridge. It uses interfaces like Manageable to define how data should be retrieved without worrying about where it's stored.
- **Data Storage Layer:** The bottom level where actual data persists in .txt files (flat-file persistence).

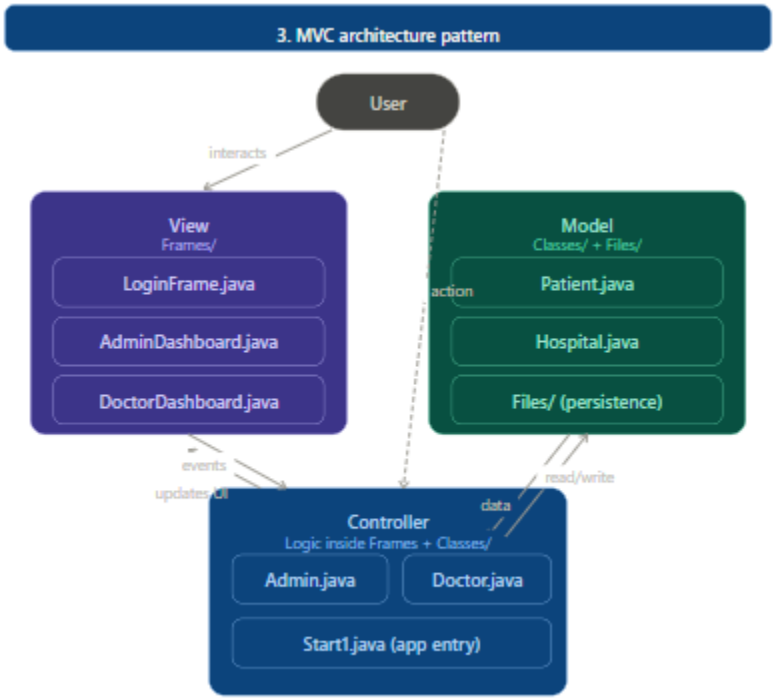
Client-Server Architecture:



This pattern focuses on the separation of concerns between the user's device (**Client**) and a centralized resource (**Server**).

- **The Client:** This is the lightweight side of the application. It handles the UI frames and initiates requests. It is where the Start1.java entry point lives.
- **The Server:** The "brain" of the operation. It hosts the heavy-lifting services:
  - **Auth Service:** Handles logins.
  - **Patient/Doctor Services:** Manages the logic for adding, searching, or discharging patients.
  - **File Store:** Manages the actual text files.
- **Communication:** The Client sends a **request** (e.g., "Get Patient List") and the Server returns a **response** or a specific .doc file.

MVC architecture:



The **MVC** pattern is designed to decouple the data from the user interface, making the code easier to maintain and scale.

- **Model (Data):** Represents the data and the rules governing it. In this system, it includes the Patient.java and Hospital.java classes, as well as the underlying data files. It does not know anything about the UI.
- **View (UI):** What the user sees. It contains the .java frame files. It displays information from the Model and sends user actions (like button clicks) to the Controller.
- **Controller (Logic):** The middleman. When a user interacts with a View, the Controller processes the event, updates the Model (read/write), and then tells the View to update itself to reflect the new data.